

Тестирование ПО

◆ **Тестирование** – поиск несоответствий между фактическим и ожидаемым результатом.

—SDLC

Этапы разработки ПО:

1 Инициация/Идея

Разработка и сбор требований

Дизайн

Разработка

Тестирование

Ввод в эксплуатацию

Вывод из эксплуатации

—Обязанности тестировщика

Разработка тестовой документации

проведение теста

оформление баг-репорта

Навыки тестировщика

Хард – ОС на уровне продвинутого пользователя, англ, языки программирования, веб-технологии, приложения, гейм-дев.

Софт – внимательность, усидчивость, коммуникабельность, ответственность

Есть 2 стороны:

1. Бизнес – хочет качественный продукт, прибыль

2. Пользователь – хочет качественный продукт, безопасность

10:54

🔗 SDLC: Как создаётся ПО — путь от идеи до запуска

◆SDLC (Software Development Life Cycle) — это пошаговый процесс создания программного обеспечения, где каждый этап (требования → дизайн → код → тест → запуск) передаёт результат следующему.

◆Фазы SDLC:

1. Сбор требований (requirements) — QA изучает SRS (документ с требованиями), задаёт вопросы, ищет неясности.

2. Дизайн (design) — QA понимает архитектуру, чтобы знать, что и как тестировать.

3. Разработка (coding) — разработчики пишут код, QA готовит тест-кейсы (сценарии проверки).

4. Тестирование (testing) — QA проверяет функционал, заводит баги (ошибки).

5. Развертывание (deployment) — продукт выкладывают в прод (реальную среду), QA проверяет, что всё работает.

6. Поддержка (maintenance) — QA проверяет фиксы (исправления), обновления и улучшения.

◆Пример

- Что тестируем: кнопка «Заказать» в приложении доставки еды

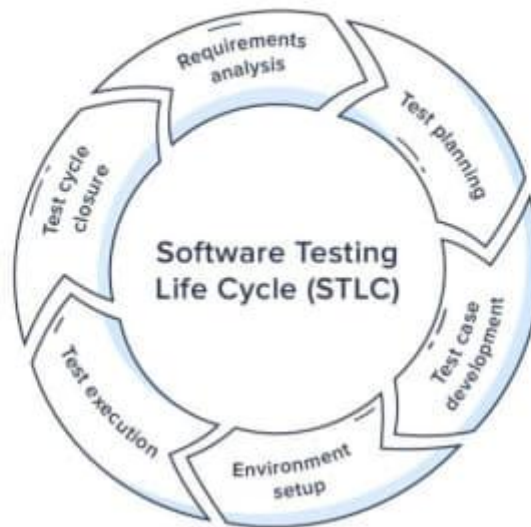
- Что проверяем: добавляется ли товар в корзину, считается ли итоговая сумма, приходит ли подтверждение

- Возможный баг: при нажатии «Заказать» без выбора адреса приложение зависает вместо показа ошибки «Укажите адрес доставки»

🕒 Итог за 10 секунд:

- Суть: SDLC — это последовательность шагов, чтобы сделать ПО качественным и в срок
- Когда использовать: всегда, когда участвуешь в проекте — чтобы понимать свою роль и сроки
- Частая ошибка: начинать тестировать, не уточнив требования (баг будет не в коде, а в понимании)
- Лучшая практика: задавать вопросы на этапе требований — дешевле исправить ошибку на бумаге, чем в коде
- Исключение: в прототипах и хакатонах этапы идут параллельно или сокращаются
- Предостережение: не игнорируй документ SRS (спецификацию требований) — это твой главный ориентир при проверке

Фазы STLC



Жизненный цикл тестирования ПО (STLC – Software Testing Lifecycle)

STLC: Как тестировщик проверяет ПО — от требований до отчёта

◆ **STLC (Software Testing Life Cycle)** — это последовательность шагов тестирования, где каждый этап (анализ требований → план → тест-кейсы → среда → запуск тестов → отчёт) направлен на поиск багов и подтверждение качества продукта.

◆ Фазы STLC

- 1 **Анализ требований (Requirement Analysis)**. Читаешь требования от аналитика, задаёшь вопросы, если что-то неясно
- 2 **Планирование тестирования (Test Planning)**. Составляешь план: что, чем и когда тестировать, какие нужны браузеры/устройства
- 3 **Разработка тест-кейсов (Test Case Development)**. Пишешь тест-кейсы (пошаговые инструкции для проверки) в тест-менеджере (TestRail, Qase)
- 4 **Настройка тестовой среды (Test Environment Setup)**. Договариваешься с DevOps, чтобы подняли тестовый сервер с нужной версией продукта
- 5 **Выполнение тестов (Test Execution)**. Запускаешь тесты, фиксируешь баги в трекере (Jira), перепроверяешь исправления
- 6 **Завершение цикла испытаний (Test Cycle Closure)**. Готовишь итоговый отчёт: сколько тестов прошло, какие баги остались, можно ли выпускать в продакшен.

◆ Пример

- Что тестируем: форма регистрации в мобильном приложении

- Что проверяем: принимается ли корректный email, показывается ли ошибка при слабом пароле, не дублируется ли аккаунт при повторной отправке
- Возможный баг: при вводе email с пробелом в конце (user@email.com_) система создаёт аккаунт, но письмо с подтверждением не приходит — пользователь не может войти

👉 Итог за 10 секунд:

- Суть: STLC — это дорожная карта тестировщика, чтобы ничего не упустить и найти баги до пользователя
- Когда использовать: в любом проекте, где есть тестирование — чтобы работать системно, а не «тыкать наугад»
- Частая ошибка: начинать писать тест-кейсы, не уточнив требования (потом придётся переделывать)
- Лучшая практика: задавать вопросы на этапе анализа требований — баг, найденный на бумаге, стоит \$1, в коде — \$100
- Исключение: в экстренных хотфиксах (срочных исправлениях) этапы сжимают до «проверил баг → отдал»
- Предостережение: не пропускай настройку тестовой среды — тесты на «неправильной» версии дадут ложные результаты

□ 7 принципов тестирования, которые должен знать каждый QA



1. Тестирование демонстрирует наличие дефектов (Testing shows presence of defects) — тесты могут показать, что баги есть, но не могут доказать, что их нет. Даже если всё прошло успешно, ошибки всё равно могут остаться.

2. Исчерпывающее тестирование недостижимо (Exhaustive testing is not possible) — невозможно проверить все комбинации данных, действий и условий из-за их огромного количества.

3. Раннее тестирование (Early testing) — тестирование нужно начинать как можно раньше (на этапе требований), чтобы находить ошибки до разработки.

4. Скопление дефектов (Defect clustering) — большинство багов обычно сосредоточено в небольшом количестве модулей.

5. Парадокс пестицида (Pesticide paradox) — если постоянно запускать одни и те же тесты, они перестают находить новые баги.

6. Тестирование зависит от контекста (Testing is context dependent) — подход к тестированию зависит от типа продукта, рисков и целей.

7. Заблуждение об отсутствии ошибок (Absence of errors fallacy) — даже если багов нет, продукт может быть бесполезным для пользователя.

◆Как это выглядит в работе по шагам:

QA получает требования и начинает анализировать их до разработки (Early testing)

Выделяет зоны риска и фокусируется на них (Defect clustering)

Понимает, что все сценарии покрыть нельзя → выбирает приоритетные (Exhaustive testing)

Пишет тест-кейсы и регулярно обновляет их (Pesticide paradox)

Проверяет продукт с учётом его типа (например, сайт ≠ банковская система) (Context dependent)

Находит баги и фиксирует их, но не делает вывод “всё идеально” (Presence of defects)

Проверяет, решает ли продукт задачу пользователя (Absence of errors fallacy)

◆Пример

Реальная ситуация:

- что тестируем: интернет-магазин
- что проверяем: оформление заказа
- какой возможен баг: кнопка “Оплатить” не работает при выборе определённого способа оплаты

Дополнительно:

QA может не проверить все способы оплаты (невозможно покрыть всё)

Основные баги будут в корзине и оплате (кластеризация)

Если тестировать только один сценарий — новые баги не найдутся (парадокс пестицида)

Даже без багов сайт может быть неудобным → пользователь уйдёт

☛ Итог за 10 секунд:

- Суть: тестирование показывает баги, но не доказывает их отсутствие
- Когда использовать: при планировании и выполнении любого тестирования
- Частая ошибка: пытаться протестировать всё подряд
- Лучшая практика: тестировать рано и фокусироваться на рисках
- Исключение: критические системы (например, медицина) требуют максимального покрытия
- Предостережение: нельзя считать продукт качественным только потому, что багов не нашли

Тестирование (Testing) — процесс проверки продукта и документов для выявления дефектов и оценки качества.

Контроль качества (QC, Quality Control) — действия по проверке готового продукта и результатов работы на соответствие требованиям.

Обеспечение качества (QA, Quality Assurance) — процессы и правила, которые предотвращают появление дефектов ещё до разработки.

Управление качеством (Quality Management) — это скоординированные действия по руководству и контролю организации в отношении качества, которые включают в себя:

1. Установление политики качества и целей качества
2. Планирование качества (quality planning)
3. Контроль качества (quality control)
4. Обеспечение качества (quality assurance)
5. Повышение качества (quality improvement)

Верификация (Verification) — проверка, что продукт сделан правильно по требованиям (без запуска кода).

Валидация (Validation) — проверка, что продукт решает задачу пользователя (с запуском кода).

12:52

Тестирование ПО

◆Цели тестирования

1. QA проверяет требования, дизайн и код → оценивает их качество
2. Сравнивает: все ли требования реализованы в продукте
3. Запускает продукт → проверяет, работает ли он так, как ожидают пользователи
4. Находит баги → фиксирует их в баг-трекере (например, Jira)

5. Анализирует, насколько продукт готов к релизу
6. Передаёт отчёт команде и менеджеру для принятия решения
7. Оценивает риски: что может сломаться после релиза
8. Проверяет соответствие стандартам и требованиям (например, безопасность, законы)

☞ Итог за 10 секунд:

- QA — предотвращает баги, QC — проверяет качество, Testing — находит дефекты
- Когда использовать:

QA — до и во время разработки, QC и Testing — после реализации фич

- Частая ошибка:

путать QA и тестирование (QA — это не только тесты)

- Лучшая практика:

строить процессы (QA) + проверять продукт (QC + Testing)

- Исключение:

в маленьких командах один человек может совмещать QA, QC и Testing

- Предостережение:

если нет QA-процессов — количество багов будет расти с каждым релизом

[#qa](#) [#qc](#) [#testing](#) [#verification](#) [#validation](#) [#softwaretesting](#) [#manualqa](#)

13:21

Тестирование ПО



Уровни тестирования

Пирамида тестирования + уровни и виды тестов



Пирамида тестирования (Test Pyramid) — модель, которая показывает распределение тестов по уровням: больше простых (unit), меньше сложных (E2E). Тест должен находиться на уровне тестируемого объекта.

- ◆ Типизация тестирования по уровням на котором оно проводится. Всего 3 уровня тестирования.

1 Unit / Component testing (компонентное, модульное или юнит тестирование) — проверка одного изолированного элемента (функция, метод, класс) без зависимостей.

△ Обычно делает разработчик.

2 Integration testing (интеграционное тестирование) — проверка взаимодействия нескольких модулей или систем.

— тестирование интеграции компонентов (отдельные модули одного приложения взаимодействуют между собой)

— системное интеграционное тестирование (взаимодействие всех систем приложения, либо взаимодействие разных систем м. собой или тест. интерфейсов с помощью кот взаимодействует система)

. ★ тест. интерфейсов, 3 вида:

— API – интерфейс программирования приложений, набор методов для доступа к функциям другой программы (внедрение платежных ресурсов,)

— CLI – интерфейс командной строки, обычная командная строка

— GUI – управление предоставляется с помощью графического интерфейса (пример то что видим когда открываем страницу браузера)

3 System testing (системное тестирование) — проверка всей системы целиком как «черного ящика» (без знания кода) с целью проверки исходным требованиям.

#qa #testing #testpyramid #unittest #integrationtesting #виды тестирования

1 April 2026

15:26

Тестирование ПО

□ Виды тестов по уровням

→ Acceptance testing (приемочное тестирование) — проверка, готов ли продукт к принятию заказчиком. (проход по всем функциям, последний этап перед релизом).

Типы приемочного тестирования

— Пользовательское приемочное тестирование UAT – тестирование группой пользователей

— Эксплуатационное – выполняется пользователями либо администратором в реальной рабочей среде. (тестирование безопасности, резервного копирования, восстановления и т.д)

— Тестирование на соответствие контракту – соответствие ГОСТам, стандартам, GDPR – требования к защите пользовательских данных.

— Альфа тестирование – эксплуатационное тестирование которое проводят другие разработчики но не в рамках нашей компании

— Бета тестирование – производится на внешней стороне, без участия разработчиков. Обычно проводится группой пользователей, чтобы получить отзыв до выхода на рынок.

◆ Виды тестирования по позитивности тестовых сценариев

— Positive testing (позитивное тестирование) — проверка работы при корректных действиях и данных.

— Negative testing (негативное тестирование) — проверка поведения при ошибочных действиях или данных.

Destructive testing (деструктивное тестирование) — проверка системы на отказ путем доведения до предела.

E2E (End-to-End, сквозное тестирование) — проверка полного пользовательского сценария от начала до конца.

♦Как это выглядит в работе по шагам:

1. QA начинает с unit-тестов → проверяются отдельные функции
2. Объединяет модули → проверяет их взаимодействие (integration)
3. Тестирует всю систему целиком (system testing)
4. Проверяет пользовательские сценарии от начала до конца (E2E)
5. Проверяет, принимает ли заказчик продукт (acceptance)
6. Выполняет позитивные тесты → проверяет нормальные сценарии
7. Выполняет негативные тесты → вводит ошибки и невалидные данные
8. Проводит деструктивные тесты → нагружает систему до отказа

♦Пример

Реальная ситуация:

- что тестируем: интернет-магазин
- что проверяем: покупку товара

Разбор:

Unit: проверяем функцию расчета цены

Integration: проверяем работу корзины + оплаты

System: тестируем весь сайт целиком

E2E: пользователь регистрируется → добавляет товар → покупает

Positive: вводим корректные данные → покупка успешна

Negative: вводим неверную карту → ошибка оплаты

Destructive: создаем сильную нагрузку → сайт падает

- возможный баг: заказ оформляется, но деньги не списываются

☞ **Итог за 10 секунд:**

- Суть: чем ниже уровень теста — тем он быстрее и дешевле, чем выше — тем сложнее и дороже
- Когда использовать: всегда комбинировать уровни тестирования в пирамиде
- Частая ошибка: делать только E2E тесты и игнорировать unit
- Любое тестирование необходимо начинать с позитивных проверок.
- Нельзя объединять позитивные и негативные тесты, так как это затрудняет локализацию дефектов.
- Предостережение: нельзя покрывать всю логику только UI-тестами

Классификация тестирования по степени важности — это разделение тестов по тому, насколько критична проверяемая функция для работы продукта.

☐ **Smoke testing (дымовое тестирование)** — проверка базовой функциональности, без которой продукт нельзя использовать. Если smoke падает → тестирование останавливается.

★ **Sanity testing (санити тестирование)** — в ISTQB считается синонимом smoke, но иногда: smoke — на нестабильном билде, sanity — на стабильном перед релизом.

☐ **Critical path testing (тестирование критического пути)** — проверка основных пользовательских сценариев (то, что делают 90% пользователей ежедневно). Может быть позитивным или негативным.

📖 **Extended testing (расширенное тестирование)** — проверка всей функциональности, включая редкие и нестандартные сценарии.

□ Типы тестирования по цели

📖 **New feature test** – тестирование качества новой функциональности, обычно тестируется по всем уровням (смоук, критического пути, расширенное)

📖 **Regression testing (регрессионное тестирование)** — проверка, что изменения не сломали уже работающий функционал. Проводится в каждом новом билде. Может включать проверку багов, может не покрывать все приложение, а конкретные его части. Часто автоматизируют, так как лучше проводить несколько раз.

★ Что включать в регрессионное тестирование?

- тесты которые покрывают тестирование безопасности либо критически важных функций для бизнеса.
- часто меняющиеся области
- функции с высокой вероятностью ошибки

📖 **Re-test (ретест)** — повторная проверка конкретного бага после его исправления.

★ Как это выглядит в работе по шагам:

1. QA получает новый билд (версию продукта)
2. Проводит smoke → проверяет базовый сценарий (например: логин → покупка)
3. Если smoke упал → тестирование останавливается
4. Если smoke прошёл → тестирует основные пользовательские сценарии (critical path)
5. Проверяет новые и изменённые функции
6. После фикса бага делает ретест → проверяет именно этот баг
7. Перед релизом запускает регрессию → проверяет стабильность всей системы
8. Дополнительно выполняет extended → проверяет редкие сценарии и крайние случаи

◆ Пример

Реальная ситуация:

- что тестируем: интернет-магазин
- что проверяем: покупку товара

Разбор:

- * Smoke: пользователь может войти → найти товар → купить
- * Critical path: покупка разными способами оплаты
- * Extended: покупка с редкой валютой или нестандартным адресом
- * Retest: исправили баг оплаты → проверяем только его
- * Regression: проверяем, не сломались ли корзина и доставка
- возможный баг: после фикса оплаты перестала работать корзина

🎯 Итог за 10 секунд:

- Суть: smoke — базовая проверка, regression — проверка после изменений, retest — проверка фикса
- Когда использовать: smoke — на каждом билде, regression — перед релизом, retest — после фикса
- Частая ошибка: путать ретест и регрессию
- Лучшая практика: сначала ретест, потом регрессия
- Исключение: в маленьких проектах регрессия может быть частичной
- Предостережение: нельзя пропускать smoke — можно тестировать “сломанный” продукт



Методы тестирования

◆ Методы тестирования по доступу к коду

1 **Black-box testing (тестирование чёрного ящика)** — тестирование без знания внутреннего кода и структуры, только через интерфейс с точки зрения пользователя.

2 **Gray-box testing (тестирование серого ящика)** — тестирование с частичным знанием системы (например, API, DevTools, понимание логики).

3 **White-box testing (тестирование белого ящика)** — тестирование с полным доступом к коду, архитектуре и внутренней логике.

18:48

◆ Функциональное vs нефункциональное тестирование — что и как проверяем

— **Функциональное тестирование (functional testing)** — проверка функций системы: выполняет ли она то, что должна по требованиям. *Отвечает на вопрос: что делает система.*

— **Нефункциональное тестирование (non-functional testing)** — проверка характеристик качества: удобство (usability), производительность (performance), безопасность (security), надежность. *Отвечает на вопрос: как система это делает.*

→ Делится на:

1 Тестирование на отказ и восстановление

2 Тестирование производительности (под нагрузкой). Есть отдельные специалисты по этому виду.

Включает в себя:

— Нагрузочное

— Стресс

— Тест стабильности – при длительной работе

— Объемное – при увеличенных объемах данных

3 Тест удобства использования

4 Безопасности

5 Установки

6 Конфигурационное

— кроссплатформенное

— кроссбраузерное

7 Локализации (i10n) – язык, культура, дата, правовые особенности, цвета

8 Интернационализации (i18n) – насколько продукт м. адаптироваться под другие локалии. чтение справа-налево, вертикально и т.д

9тестирование GUI – требования к графическому интерфейсу

10 Тест доступности (для людей с ограниченными возможностями инвалидов, для водителей, для мам и т.д.)

◆ Классификация по степени автоматизации

— **Manual testing (ручное тестирование)** — выполнение тестов человеком без использования автоскриптов. QA сам проходит шаги и проверяет результат.

— **Automated testing (автоматизированное тестирование)** — выполнение тестов с помощью кода и инструментов (например, автотестов), где проверки запускаются без участия человека.

18:48

◆ Статическое vs динамическое тестирование

— **Static testing (статическое тестирование)** — проверка без запуска кода. Анализируются артефакты (результаты работы): требования, документация, код (через code review — проверка кода коллегами), тестовые данные.

— **Dynamic testing (динамическое тестирование)** — проверка с запуском кода. Оценивается реальное поведение системы во время работы.

→Пример:

Static: читаем требования → поле email должно быть обязательным

Static: смотрим код → проверка email отсутствует

Dynamic: запускаем приложение → вводим пустой email

Проверяем поведение → регистрация проходит (ошибка)

• возможный баг: система позволяет зарегистрироваться без email

18:49

◆Классификация по степени формализации

— **Scripted testing (тестирование по тест-кейсам)** — полностью формализованный подход: тестирование выполняется строго по заранее написанным тест-кейсам (пошаговые инструкции + ожидаемый результат).

— **Exploratory testing (исследовательское тестирование)** — частично формализованный подход: есть общее понимание и базовый план, но сценарии уточняются прямо во время тестирования.

— **Ad hoc testing (интуитивное тестирование)** — неформализованный подход: нет документации, тестирование строится на опыте и интуиции тестировщика (experience-based testing).

18:55

◆Классификация по целям и задачам

Installation testing (инсталляционное тестирование) — проверка процесса установки, обновления и удаления приложения, чтобы убедиться, что продукт корректно устанавливается и работает после установки.

✦ Включает в себя следующие процессы:

Установка ПО

Удаление ПО

Обновление ПО

Откат на предыдущую версию

Повторный запуск установки после возникновения ошибки или исправления уже возникших проблем
Автоматическая установка
Установка отдельного компонента из общего пакета программ

— **Usability testing (тестирование удобства использования)** — проверка того, насколько пользователю понятно, легко и приятно работать с продуктом. Относится к нефункциональному тестированию

✦ *Что нужно тестировать:*

Общая доступность
Скорость, производительность
Удобство навигации и интерфейс
Плавность

— **Тестирование доступности (accessibility testing, A11Y)** — тестирование, направленное на исследование пригодности продукта к использованию людьми с ограниченными возможностями (слабым зрением и т.д.). Относится к нефункциональному тестированию.

✦ *Что можно тестировать:*

Использование вспомогательных технологий в ПО (распознавание речи, экранная клавиатура и лупа, скринридеры)
Возможность использовать приложение одной рукой
Настройки специальной цветопередачи
Наличие понятных инструкций и руководства пользователя
Доступность сайта можно тестировать специальными инструментами, например, WAVE, TAV, Accessibility Valet, Accessibility Developer Tools.

— **Security testing (тестирование безопасности)** — проверка, может ли система защитить данные и функции от несанкционированного доступа (когда у пользователя нет прав). Относится к нефункциональному тестированию.

✦ *От чего нужно защищать ПО:*

SQL-инъекции
XSS-инъекции
Перехват трафика
Брутфорсинг (полный перебор данных для получения доступа)

— **Internationalization testing (интернационализация, i18n)** — проверка готовности продукта к работе на разных языках и в разных культурах (без привязки к конкретной стране).

— **Localization testing (локализация, l10n)** — проверка, что продукт корректно адаптирован под конкретный язык и страну (переводы, форматы, культура).

i18n: можно переключить язык на немецкий

l10n: перевод корректный → "Buy" = "Kaufen"

19:00

☞ **Итог за 10 секунд виды тестирования:**

— *Functional testing (функциональное тестирование)* — проверяем, что делает система

— *Non-functional testing (нефункциональное тестирование)* — проверяем, как система это делает

— **Black-box testing (черный ящик)** — тестируем без знания внутренней логики системы

— **Gray-box testing (серый ящик)** — тестируем с частичным пониманием системы (например, API)

- White-box testing (белый ящик) — тестируем с полным доступом к коду и логике
- *Smoke testing (дымовое тестирование)* — проверяем базовую критичную функциональность
- *Critical path testing (тестирование критического пути)* — проверяем основные пользовательские сценарии
- Extended testing (расширенное тестирование) — проверяем всю функциональность и редкие кейсы
- Positive testing (позитивное тестирование) — проверяем корректные сценарии
- Negative testing (негативное тестирование) — проверяем ошибки и невалидные данные
- Destructive testing (деструктивное тестирование) — проверяем пределы системы и точку отказа
- Manual testing (ручное тестирование) — тесты выполняет человек
- Automated testing (автоматизированное тестирование) — тесты выполняются автоматически
- Static testing (статическое тестирование) — проверяем без запуска кода
- Dynamic testing (динамическое тестирование) — проверяем с запуском кода
- Scripted testing (по тест-кейсам) — тестируем по заранее описанным шагам
- Exploratory testing (исследовательское) — тестируем с доработкой сценариев по ходу
- Ad hoc testing (интуитивное) — тестируем без документации, на опыте
- Installation testing (инсталляционное тестирование) — проверяем установку, обновление и удаление
- Usability testing (тестирование удобства) — проверяем, насколько продукт понятен и удобен
- Accessibility testing (тестирование доступности) — проверяем доступность для людей с ограничениями
- Security testing (тестирование безопасности) — проверяем, можно ли взломать систему или получить чужие данные
- i18n — готовность к разным языкам,
- l10n — корректная адаптация под конкретный язык
- Re-testing (ретест) — проверяем, что баг исправлен
- Regression testing (регрессионное тестирование) — проверяем, что изменения не сломали систему

◆Классификация по целям и задачам (ч2)

- **Тестирование совместимости (compatibility testing, interoperability testing)** – проверка, может ли приложение корректно работать в разных средах: устройства, браузеры, операционные системы, сеть. Относится к нефункциональному тестированию.
Включает в себя:
 - 1 **Configuration testing (конфигурационное тестирование)** — проверка работы приложения на разных конфигурациях (железо, ОС, сеть).
 - 2 **Cross-browser testing (кросс-браузерное тестирование)** — проверка работы в разных браузерах и их версиях.
 - 3 **Cross-platform testing (кросс-платформенное тестирование)** — проверка работы на разных операционных системах.
- **Reliability testing (тестирование надёжности)** — проверка, может ли система стабильно работать без сбоев в течение времени или количества операций.

— **Recoverability testing (тестирование восстанавливаемости)** — проверка, может ли система восстановиться после сбоя и вернуть данные и работоспособность.

— **Failover testing (тестирование отказоустойчивости)** — проверка, может ли система продолжить работу при сбое за счёт резервных механизмов (переключение на запасные ресурсы).

— **Performance testing (тестирование производительности)** — проверка скорости и стабильности работы системы при разной нагрузке (сколько пользователей, данных, запросов). Относится к нефункциональному тестированию.

→ Подвиды:

Нагрузочное тестирование (load testing, capacity testing)

Тестирование масштабируемости (scalability testing)

Объёмное тестирование (volume testing)

Стрессовое тестирование (stress testing)

Конкурентное тестирование (concurrency testing)

— **Load testing (нагрузочное тестирование)** — проверка работы системы при нормальной и немного повышенной нагрузке (в пределах допустимого).

— **Scalability testing (тестирование масштабируемости)** — проверка, как система улучшает производительность при добавлении ресурсов (серверов, памяти).

— **Volume testing (объёмное тестирование)** — проверка работы с большим количеством данных.

— **Stress testing (стрессовое тестирование)** — проверка поведения системы при экстремальной нагрузке или нехватке ресурсов.

— **Concurrency testing (конкурентное тестирование)** — проверка работы при большом количестве одновременных запросов.

14:55

Нефункциональное тестирование

В практике тестирования принято деление тестирования на тестирование функционального показателя качества, называемое «функциональным тестированием», и тестирование других показателей качества, называемое «нефункциональным тестированием».

14:57

⚠ Двоякая природа тестирования безопасности и производительности

Существует трактовка, по которой к *функциональному тестированию* можно отнести **тестирование безопасности и тестирование производительности**, но только в тех случаях, когда они представляют основную функциональность приложения, а не просто являются дополнительными характеристиками.

Например, защита денежных транзакций, функции антивируса, возможность одновременной работы над задачей несколькими пользователями.

! Использовать с осторожностью и только, если спросят про такие кейсы. В стандартах это не упоминается.

Нейросети для разработчика

Есть полезная утилита [Copilot](#) от компании Microsoft. Это утилита, которая обучилась на других

репозиториях, чтобы помогать разработчикам и ускорять написание кода.

В РФ доступен бесплатный аналог от Сбера. Вот инструкция, как его установить:

Гигакод – войти и привязать устройство, выбираем среду и скачиваем zip архив с установщиком vsx, затем в vs code нужно установить этот плагин через меню плагинов (в меню выбрать установка из vsx).

Затем на нижней панели справа появляется кнопка подключить Gigcode, нажимаем и устройство привязано. (для личного использования бесплатно).

Tabnine – ИИ работает с локальными данными и подстраивается под проект используя локальный контекст. Есть бесплатная базовая версия. Можно писать промпт на создание функции в комментариях кода.

ReplitAi – онлайн редактор внутри страницы браузера. Можно включать пояснения прямо в коде. Можно задавать вопросы или просить подсказки по коду внутри браузера.

Create – быстрый прототип на react отдельных компонентов. Создает сразу визуал и код, который затем можно экспортировать. Может использовать разные UI фреймворки (например Chakra UI). Возможна генерация по скриншоту.

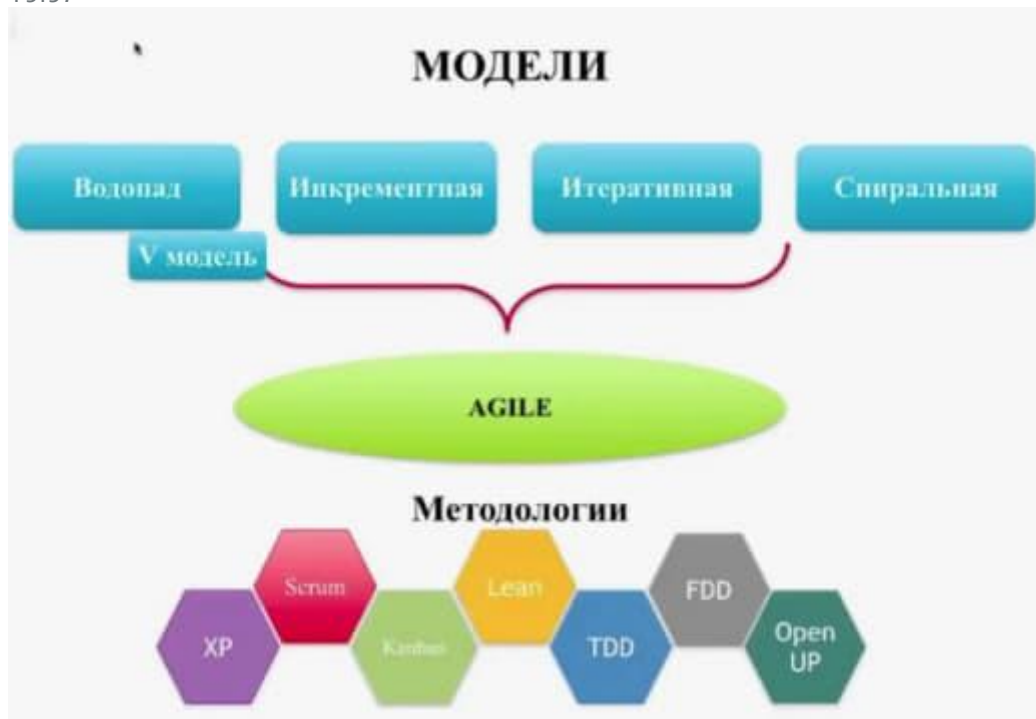
#код

15:57

Тестирование ПО

Модели разработки ПО

15:57



17:13

Тестирование ПО

❖ **Модель разработки ПО (Software Development Model, SDM)** — это структура, которая определяет этапы разработки (анализ, разработка, тестирование и т.д.), их порядок и взаимодействие между ними.

🐉 **Waterfall model (водопадная модель)** — разработка "по этапам без возврата". Линейная модель разработки, где каждая фаза выполняется один раз и строго после предыдущей (требования →

дизайн → разработка → тестирование → релиз). Возврат назад не предусмотрен: ошибка = начинать заново.

◆ **V-model (V-образная модель)** — модель разработки, где каждому этапу разработки соответствует этап тестирования, и тестирование планируется заранее. Это развитие waterfall, но с ранним вовлечением QA.

◆ **Iterative / Incremental model (итерационная инкрементальная модель)** — модель, где продукт создаётся по частям (инкрементам), а этапы разработки повторяются много раз (итерации). Каждая итерация даёт рабочий результат с добавленной функциональностью.

◆ **Spiral model (спиральная модель)** — модель разработки, в которой проект развивается итерациями, но главный фокус — анализ и снижение рисков на каждом этапе.

◆ **Agile model (гибкая модель разработки)** — подход к разработке ПО, основанный на Agile-манифесте, где приоритет — люди, работающий продукт, сотрудничество с заказчиком и готовность к изменениям.

Ключевые принципы:

Люди и взаимодействие важнее процессов

Работающий продукт важнее документации

Сотрудничество с заказчиком важнее контракта

Готовность к изменениям важнее плана

🕒 Итог за 10 секунд:

— **Waterfall model (водопадная модель)** – разработка идёт строго по этапам без возврата

— **V-model (V-образная модель)** –

тестирование планируется заранее и идёт параллельно разработке

— **Iterative / Incremental model (итерационная инкрементальная модель)** – продукт развивается по частям, каждая итерация добавляет функциональность

— **Spiral model (спиральная модель)** – разработка идёт по итерациям с постоянным анализом рисков

— **Agile model (гибкая модель разработки)** – разработка маленькими шагами с постоянной обратной связью

17:15

★ Примеры моделей разработки

◆ **Waterfall (водопадная модель)** — сделали всё сразу → потом тестируем

☞ Пример: разработали весь интернет-магазин → только в конце QA начинает тестирование

◆ **V-model (V-образная модель)** — тестирование планируется заранее

☞ Пример: при написании требований сразу продумали, как будем проверять оплату → потом по этим тестам проверили

◆ **Iterative / Incremental (итерационная инкрементальная)** — продукт делается по частям

☞ Пример:

Итерация 1 → регистрация

Итерация 2 → корзина

Итерация 3 → оплата

◆ **Spiral (спиральная модель)** — сначала анализ рисков → потом разработка

☞ Пример: есть риск, что система не выдержит нагрузку → сначала делаем прототип → тестируем → потом разрабатываем

◆ **Agile (гибкая модель)** — короткие циклы + постоянные улучшения

☞ Пример:

Спринт 1 → сделали оплату

Спринт 2 → улучшили UX

Спринт 3 → добавили новые функции

☞ **Итог за 10 секунд:**

- Waterfall — всё сразу → тест в конце
- V-model — тесты планируются заранее
- Iterative — делаем по частям
- Spiral — через анализ рисков
- Agile — быстро, гибко, с фидбеком

17:15

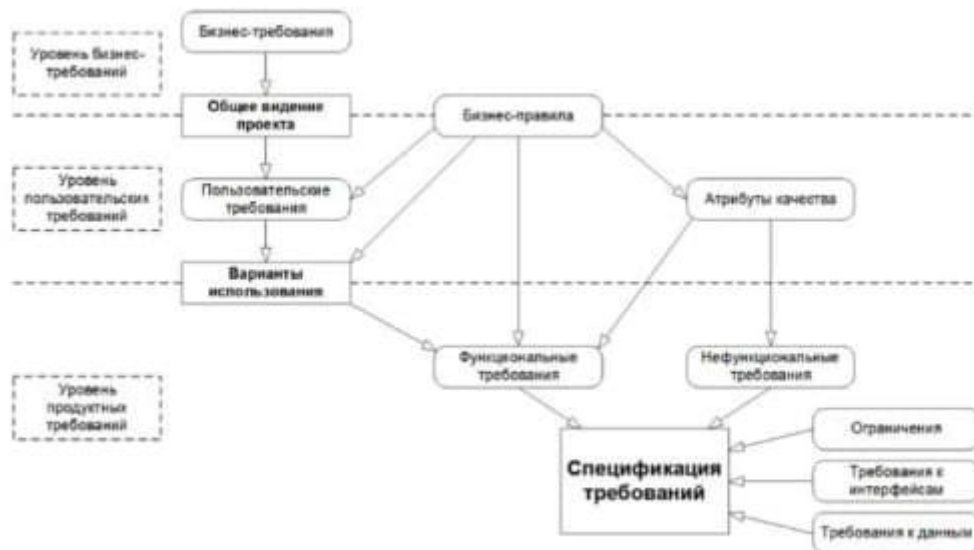


Рисунок 2.2.е — Уровни и типы требований

■ Уровни и типы требований

◆ **Requirement (требование)** — это обязательное условие или критерий, которому должна соответствовать система. Без выполнения требования продукт считается неправильным.

18:50

■ Уровни и типы требований

◆ **Requirements (требования)** — это набор условий и правил, которым должна соответствовать система.

Делятся на уровни: бизнес → пользователь → продукт (функциональные и нефункциональные).

✦ Все требования собираются в документ **SRS (Software Requirements Specification** — спецификация требований).

◆ **Business requirements (бизнес-требования)** — описывают цель продукта: зачем он нужен и какую пользу приносит.

◆ **User requirements (пользовательские требования)** — описывают, что пользователь должен уметь делать в системе (через user stories, use cases).

◆ **Functional requirements (функциональные требования)** — описывают, что именно делает система (логика, действия, проверки).

◆ **Non-functional requirements (нефункциональные требования)** — описывают, как система работает (скорость, безопасность, удобство и т.д.).

◆ **Спецификация требований (software requirements specification, SRS)** объединяет в себе описание всех требований уровня продукта и может представлять собой весьма объёмный документ (сотни и тысячи страниц).

✦ С остальными уровнями и типами требований вы можете ознакомиться [здесь](#),

→ **Как это выглядит в работе по шагам:**

1. Бизнес формулирует цель → зачем нужен продукт (business)
2. Аналитик описывает сценарии → что будет делать пользователь (user)
3. Команда превращает это в конкретные действия системы (functional)
4. Добавляют ограничения качества → скорость, безопасность (non-functional)
5. Все требования собираются в SRS-документ
6. QA изучает требования → пишет тест-кейсы
7. Проверяет соответствие системы требованиям
8. Находит несоответствия → фиксирует баги

◆ **Пример**

- что тестируем: интернет-магазин
- что проверяем: покупку товара

Разбор:

Business: увеличить продажи

User: пользователь должен купить товар

Functional: кнопка “Купить” оформляет заказ

Non-functional: страница загружается за 2 секунды

возможный баг: кнопка работает (functional ok), но грузится 10 секунд (non-functional ошибка)

☞ **Итог за 10 секунд:**

- Суть: требования идут сверху вниз — от цели бизнеса к конкретным действиям системы
- Когда использовать: при анализе и тестировании любого продукта
- Частая ошибка: проверять только функциональные требования
- Лучшая практика: тестировать и функциональные, и нефункциональные требования
- Исключение: на ранних этапах нефункциональные требования могут быть частично описаны
- Предостережение: если требования неполные — баги неизбежны

[#требования](#) [#requirements](#)

6 April 2026

17:50

Тестирование ПО

◆ **Неявные требования (implicit requirements)** — требования, которые не прописаны явно в документации, но ожидаются по умолчанию на основе логики, стандартов, опыта и контекста продукта. В таком случае необходимо изучать неявные требования из других источников:

Законы, регламенты, инструкции

Список задач, существующие тесты и баг-репорты

Руководство пользователя

Реклама продукта

Интервью с командой и заказчиками

Чаты и email-переписка

Прототип, дизайн-макет

Конкурентный анализ, личный опыт

✦ Это далеко не весь перечень, подробнее [здесь](#)

◆ **Как это выглядит в работе по шагам:**

1. QA получает задачу → требований нет или они неполные
2. Изучает законы и стандарты → что обязательно должно быть
3. Смотрит существующие тесты и баги → как система уже работает
4. Читает руководство пользователя → ожидаемое поведение
5. Анализирует дизайн и прототип → как должен выглядеть интерфейс
6. Общается с командой и заказчиком → уточняет детали
7. Сравнивает с конкурентами → как реализовано у других
8. Формирует свои проверки → на основе опыта и логики

◆ **Пример**

- что тестируем: форму регистрации
- что проверяем: поведение системы

Разбор:

В требованиях не написано, что email обязателен

Но по логике и опыту → без email регистрация невозможна

QA проверяет → регистрация проходит без email

- возможный баг: система позволяет зарегистрироваться без обязательного поля

☞ **Итог за 10 секунд:**

- Суть: неявные требования — это то, что “и так должно работать”
- Когда использовать: когда требований мало или они неполные
- Частая ошибка: тестировать только по документации
- Лучшая практика: использовать опыт, логику и дополнительные источники
- Исключение: строго регламентированные проекты (там всё прописано)
- Предостережение: игнорирование неявных требований = пропущенные баги

17:51

◆ **Свойства качественных требований** — это критерии, которые показывают, насколько требование написано правильно и пригодно для разработки и тестирования.

◆ **Виды свойства качественных требований:**

- **Completeness (завершённость)** — требование содержит всю нужную информацию, ничего не пропущено.
- **Atomicity (атомарность)** — одно требование = одна логика, его нельзя делить без потери смысла.
- **Consistency (непротиворечивость)** — требования не конфликтуют друг с другом и сами с собой.
- **Unambiguousness/clearness (однозначность)** — требование понятно только в одном смысле, без

двусмысленности.

- **Feasibility (выполнимость)** — требование можно реализовать технически и в рамках сроков/бюджета.
- **Obligatoriness & up-to-date (обязательность и актуальность)** — требование действительно нужно и не устарело.
- **Traceability (прослеживаемость)** — можно связать требование с тестами, кодом и другими требованиями.
- **Modifiability (модифицируемость)** — требование можно легко изменить без “поломки” системы требований.
- **Ranking (приоритизация)** — у требования есть приоритет, важность и срочность.
- **Correctness & verifiability (корректность и проверяемость)** — требование написано правильно и его можно проверить тестами.

◆ Как это выглядит в работе по шагам:

1. QA читает требование
2. Проверяет полноту → хватает ли информации
3. Проверяет атомарность → нет ли “2 в 1”
4. Проверяет противоречия → с другими требованиями
5. Проверяет ясность → нет ли двусмысленных слов
6. Оценивает выполнимость → реально ли сделать
7. Проверяет актуальность и приоритет
8. Проверяет, можно ли написать тест-кейс → если нет → требование плохое

◆ Пример

- что тестируем: регистрацию пользователя
- что проверяем: качество требования

Разбор:

Требование: “система должна быстро обрабатывать запросы” → ✗ непонятно (неоднозначность)

Исправление: “ответ системы ≤ 2 секунды при 1000 пользователей” → ✓ можно протестировать

возможный баг: из-за плохого требования разработчик реализует “как понял” → система работает неправильно

☞ Итог за 10 секунд:

- Суть: хорошее требование = понятное, проверяемое и без противоречий
- Когда использовать: при анализе требований перед тестированием
- Частая ошибка: принимать расплывчатые требования
- Лучшая практика: если нельзя написать тест-кейс → требование нужно уточнить
- Исключение: на ранних этапах требования могут быть неидеальными
- Предостережение: плохие требования = баги ещё до разработки

#требования

18:12

Тестирование ПО

Документирование требований

◆ **Use case (вариант использования)** — сценарий, как пользователь (актор — пользователь или система) взаимодействует с системой для достижения цели. Обычно отображается в виде диаграммы: актер (человек), система (прямоугольник), функции (овалы).

◆ **User story (пользовательская история)** — краткое описание функции от лица пользователя по

шаблону:

“As a [роль], I want [действие], so that [цель]”. Обязательно содержит acceptance criteria (критерии приемки — конкретные условия, по которым проверяется готовность).

Москир (макет) — визуальная модель интерфейса (как будет выглядеть экран), используется для проверки frontend (UI — интерфейса).

◆Как это выглядит в работе по шагам:

Аналитик описывает use case → общий сценарий работы пользователя

Формирует user stories → разбивает на маленькие задачи

Добавляет acceptance criteria → конкретные условия проверки

Дизайнер делает москир → как выглядит интерфейс

QA изучает user story и критерии

QA смотрит москир → проверяет UI

QA пишет тест-кейсы на основе acceptance criteria

QA тестирует → проверяет соответствие требованиям

◆Пример

- что тестируем: регистрацию пользователя

Разбор:

User story:

“As a user, I want to register, so that I can use the service”

Acceptance criteria:

пароль ≥ 8 символов, email валидный

Москир: форма регистрации с полями

QA проверяет:

- есть ли поля
- соответствуют ли макету
- выполняются ли критерии

- возможный баг: поле есть, но пароль можно ввести из 3 символов

👉 Итог за 10 секунд:

- Суть: use case — сценарий, user story — задача, москир — внешний вид
- Когда использовать: при разработке и тестировании функциональности
- Частая ошибка: игнорировать acceptance criteria
- Лучшая практика: писать тесты на основе критериев приемки
- Исключение: маленькие проекты могут обходиться без use case
- Предостережение: без четких требований тестирование становится хаотичным

✈ Подробнее можно прочитать [здесь](#) и [здесь](#).

18:17

□ Как QA работает с требованиями — идеал vs реальность

✦ **Процесс работы с требованиями** — это последовательность действий QA по проверке, уточнению и подготовке требований к тестированию до начала разработки.

◆Как это выглядит в работе по шагам:

1. Аналитик или Product Owner пишет требования + макеты
2. QA проверяет требования → полнота, ясность, логика

3. Команда обсуждает требования (Backlog Refinement — встреча для уточнения)
4. QA находит ошибки → отправляет на доработку
5. Повторяют проверку → пока требования не станут качественными
6. После согласования → требования идут в разработку
7. QA заранее пишет тест-кейсы и чек-листы (до разработки)
8. Во время спринта требования не меняются

◆Пример

- что тестируем: регистрацию

Разбор:

Требование: “пользователь регистрируется”

QA уточняет → какие поля, какие ограничения

Пишет тест-кейсы до разработки

Разработка начинается уже с понятными требованиями

- возможный баг: если требования не уточнили → разработчик реализует “как понял”

☞ Итог за 10 секунд:

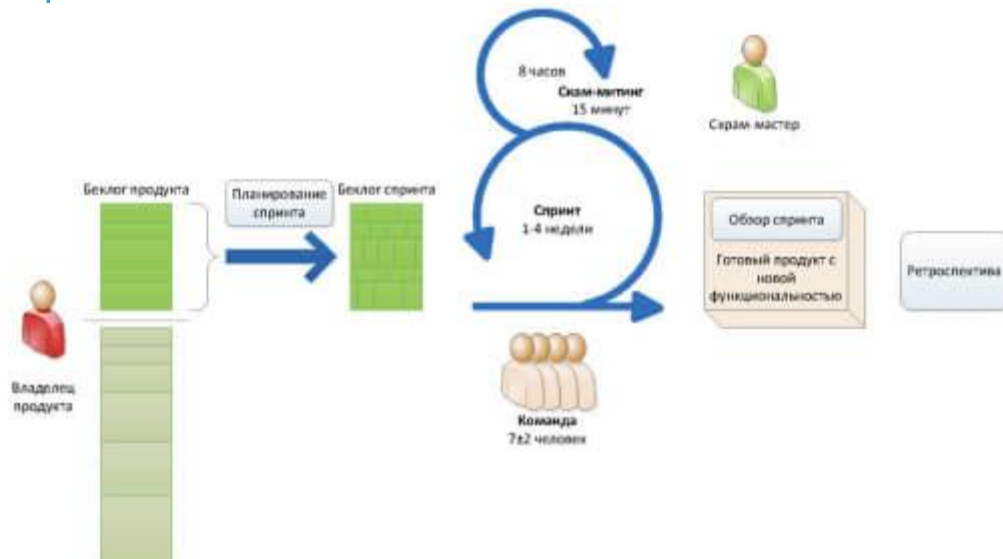
- Суть: QA должен проверить и уточнить требования ДО разработки
- Когда использовать: перед началом любого спринта
- Частая ошибка: писать тесты после разработки
- Лучшая практика: сначала требования → потом тесты → потом код
- Исключение: в неидеальных проектах требования уточняются по ходу
- Предостережение: плохие требования = баги на всех этапах

#требования

7 April 2026

15:52

Тестирование ПО



◆ **Scrum** — это легкий фреймворк (набор правил и процессов), который помогает команде создавать продукт по частям и адаптироваться к изменениям.

Ценности Scrum — принципы, без которых Scrum не работает:

Commitment (приверженность) — команда отвечает за результат

Focus (фокус) — команда сосредоточена на целях спринта

Openness (открытость) — все проблемы и прогресс обсуждаются открыто

Respect (уважение) — участники уважают друг друга как профессионалов

Courage (смелость) — команда не боится принимать решения и решать сложные задачи

17:24

Тестирование ПО

👤 Роли в Scrum

◆ **Scrum Team** — небольшая (до 10 человек), кросс-функциональная (все нужные навыки внутри) и самоуправляемая команда, работающая над одной целью — **Product Goal**.

— **Developers** — участники команды, которые создают готовый продукт (Increment — результат спринта). Сюда входят разработчики, QA, аналитики.

— **Product Owner (PO)** — отвечает за ценность продукта и управляет списком задач (Product Backlog).

— **Scrum Master (SM)** — отвечает за правильное применение Scrum и помогает команде работать эффективно.

◆Что появилось / уточнилось (актуальные дополнения 2024–2026):

Нет новых ролей в официальном Scrum

→ роли остались те же

Появился Scrum Guide Expansion Pack (2025)

☞ это НЕ новая версия, а дополнение ()

Он добавляет:

— больше фокуса на ценность (value), а не просто задачи

— участие stakeholders (заинтересованных лиц)

— акцент на результат, а не только “сделано”.

Расширилось понимание ролей (на практике):

Scrum Master → уже не просто “фасилитатор”, а системный коуч

Product Owner → думает про стратегию и бизнес-ценность

Developers → полностью отвечают за результат (включая QA)

☞ Но это эволюция практики, а не изменение ролей

◆Как это выглядит сейчас в реальной работе:

Scrum Team = единая команда без разделений

QA работает внутри Developers

Все отвечают за результат, а не “свою часть”

Фокус сместился:

✗ не “закрыть задачу”

✓ а “дать ценность пользователю”

Scrum Master помогает не только команде, но и организации

Product Owner работает ближе к бизнесу и метрикам

◆Как это выглядит в работе по шагам:

1. Product Owner формирует Product Goal (цель продукта)

2. PO наполняет и приоритизирует Product Backlog (список задач)

3. Scrum Team выбирает задачи на спринт
4. Developers (включая QA) планируют работу (Sprint Backlog)
5. Разрабатывают и тестируют функциональность
6. Scrum Master помогает убрать препятствия
7. Команда ежедневно синхронизируется (Daily Scrum)
8. В конце спринта получают готовый Increment

◆Пример:

- что тестируем: функцию оплаты
- что проверяем: процесс работы команды

Разбор:

- * PO ставит задачу → “реализовать оплату”
- * Developers делают и тестируют
- * QA проверяет внутри команды
- * SM помогает убрать блокеры

- возможный баг: если QA не участвует в процессе → баги находят слишком поздно

👉 Итог за 10 секунд:

- Суть: PO — что делать, Developers — делают, SM — помогает
- Когда использовать: в Scrum-командах
- Частая ошибка: выделять QA вне Developers
- Лучшая практика: QA работает внутри команды
- Исключение: в старых моделях QA может быть отдельно
- Предостережение: отсутствие ролей = потеря ответственности

#scrum

17:34

📅 События Scrum — как проходит работа внутри спринта

◆ **Sprint** — фиксированный цикл (до 1 месяца), внутри которого создается готовый результат (Increment). Все события Scrum происходят внутри Sprint.

◆ **Sprint Planning** — планирование спринта: команда решает, что делать и как делать.

→ В ходе *Sprint Planning* рассматриваются следующие темы:

- Почему этот Sprint ценен? Определение Sprint Goal
- Что может быть готово в этом Sprint?
- Как будет выполняться выбранная работа?

◆ **Sprint Goal** — главная цель спринта, это краткое описание того, какую ценность команда должна создать в текущем Sprint.

◆ **Product Backlog** — это упорядоченный список всех задач, требований и улучшений продукта. За него отвечает Product Owner. Элементы называются PBI (Product Backlog Items).

◆ **Sprint Backlog** — это набор задач (Product Backlog Items) + план их выполнения, которые команда выбрала для текущего Sprint, чтобы достичь Sprint Goal. (*Sprint Backlog = Sprint Goal + выбранные элементы Product Backlog + плюс план их реализации*)

◆ **Daily Scrum** — ежедневная встреча (15 минут) для проверки прогресса и корректировки плана.

◆ **Sprint Review** — демонстрация результата и сбор обратной связи от заказчика.

◆ **Sprint Retrospective** — анализ процесса и поиск улучшений.

📖 *Актуально (Scrum Guide 2020–2026):*

* Sprint = контейнер всех событий

* 3 вопроса на Daily **не обязательны** (убраны из гайда)

(★ см комментарии)

◆ **Как это выглядит в работе по шагам:**

1. Начинается Sprint → фиксируем цель (Sprint Goal)
2. Sprint Planning → выбираем задачи и планируем работу
3. Каждый день → Daily Scrum → проверяем прогресс
4. Developers (включая QA) разрабатывают и тестируют
5. В конце → Sprint Review → показываем результат
6. Получаем обратную связь → корректируем план
7. Проводим Retrospective → улучшаем процесс
8. Начинается новый Sprint

◆ **Пример**

- что тестируем: функцию оплаты
- что проверяем: процесс Scrum

Разбор:

- * Planning → выбрали задачу “сделать оплату”
- * Daily → обсуждаем прогресс
- * QA тестирует в процессе
- * Review → показываем заказчику
- * Retrospective → обсуждаем ошибки

- возможный баг: команда не синхронизируется → задача не завершена к концу спринта

🕒 **Итог за 10 секунд:**

- Суть: Sprint = цикл, внутри которого планируем → делаем → проверяем → улучшаем
- Когда использовать: в Scrum-проектах
- Частая ошибка: пропускать ретроспективу
- Лучшая практика: тестировать внутри спринта, а не в конце
- Исключение: маленькие команды могут упрощать процесс
- Предостережение: без событий Scrum команда теряет контроль над процессом

#scrum события #sprint

18:14

Тестирование ПО

📦 **Артефакты Scrum — что команда реально создает**

◆ **Scrum Artifacts (артефакты Scrum)** — это ключевые элементы, которые делают работу прозрачной и показывают прогресс: Product Backlog, Sprint Backlog и Increment.

— **Product Backlog** — упорядоченный список всех задач и требований продукта. Постоянно обновляется и является единственным источником работы команды. Уточнение (Refinement) — разбиение и детализация задач.

— **Sprint Backlog** — план спринта: Sprint Goal (зачем), выбранные задачи (что) и план реализации (как). Создается Developers и обновляется в процессе спринта.

— **Increment** — готовый к использованию результат работы (часть продукта), который добавляется к предыдущим версиям и приносит ценность.

♦Как это выглядит в работе по шагам:

1. Product Owner формирует Product Backlog
2. Команда уточняет задачи (Refinement) → делает их понятными
3. На Sprint Planning выбирают задачи
4. Формируют Sprint Backlog (цель + задачи + план)
5. Developers (включая QA) реализуют и тестируют
6. Постоянно обновляют Sprint Backlog
7. Получают Increment → готовый результат
8. Показывают его на Sprint Review

♦Пример

- что тестируем: интернет-магазин
- что проверяем: процесс Scrum

Разбор:

- * Product Backlog → “сделать оплату”
- * Sprint Backlog → план: кнопка + API + тесты
- * Increment → рабочая оплата

QA:

- проверяет, что функция реально работает
- возможный баг: функция сделана, но не готова к использованию → это HE Increment

☞ Итог за 10 секунд:

- Суть: Product Backlog — что делать, Sprint Backlog — план, Increment — результат
- Когда использовать: в каждом Scrum-проекте
- Частая ошибка: считать Increment “недоделанной функцией”
- Лучшая практика: Increment должен быть полностью готов
- Исключение: можно делать несколько Increment за спринт
- Предостережение: если нет прозрачности → команда теряет контроль

#scrum

18:29

♦**Estimation (оценка)** — это прогноз усилий, которые понадобятся для выполнения задачи (Product Backlog Item). Используется для планирования спринтов и понимания сроков релиза. Существует множество способов оценки усилий Скрам-командой, но при этом всегда используются относительные единицы: например, Стори Поинты. Обычно оценка проводится в рамках Уточнения (Refinement, Груминга) Бэклога Продукта.

Story Points (стори поинты) — это условная единица, которая показывает относительную сложность задачи с учетом объема работы, рисков и неопределенности. Это не время, а “насколько сложно сделать”. Часто используются числа Фибоначчи (1, 2, 3, 5, 8, 13...).

Planning Poker (покер-планирование) — это способ оценки задач, при котором каждый участник команды независимо выбирает сложность задачи (в story points), а потом команда обсуждает различия и приходит к общему решению.

♦Как это выглядит в работе по шагам:

1. Команда берет задачу из Product Backlog
2. Обсуждает требования → что нужно сделать
3. Каждый участник выбирает оценку (story points)
4. Все одновременно показывают оценки
5. Если оценки разные → обсуждают причины
6. Приходят к общей оценке
7. Записывают её в backlog
8. Используют оценки для планирования спринта

♦Пример

- что тестируем: задачу “реализовать оплату”

Разбор:

- * Один ставит 3 → думает, что просто
- * Другой ставит 8 → знает про сложную интеграцию
- * Обсуждают → находят скрытые сложности
- * Итог → ставят 8

- возможный баг: недооценка задачи → не успевают в спринт

👉 Итог за 10 секунд:

- Суть: оценка — это не время, а сложность задачи
- Когда использовать: при планировании задач
- Частая ошибка: переводить story points в часы
- Лучшая практика: оценивать командой через обсуждение
- Исключение: маленькие задачи можно не оценивать подробно
- Предостережение: плохая оценка = сорванные сроки

Подробнее [здесь](#)

[#scrum](#) [#planning](#) [poker](#) [#storypoints](#) [#estimation](#)

19:06

Тестирование ПО

📊 Velocity и Capacity — сколько команда может сделать

Velocity (скорость команды) — это количество работы (в story points), которое команда реально завершила за спринт. Считается только по полностью готовым задачам (Done).

Capacity (вместимость спринта) — это сколько работы команда планирует взять в спринт (в story points), исходя из доступных ресурсов.

[Источник](#)

👉 Итог за 10 секунд:

- Суть: Capacity — план, Velocity — факт
- Когда использовать: при планировании спринтов
- Частая ошибка: путать эти понятия
- Лучшая практика: ориентироваться на прошлый Velocity
- Исключение: новая команда → нет исторических данных
- Предостережение: игнорирование Velocity = срыв сроков

12:44

Тестирование ПО

★ Дополнительные термины Scrum — что часто спрашивают на работе

◆ **MVP (Minimum Viable Product)** — минимально жизнеспособный продукт, версия с базовым функционалом, которая позволяет быстро получить обратную связь от пользователей.

◆ **Stakeholder (стейкхолдер)** — заинтересованное лицо, которое влияет на продукт или дает обратную связь (заказчик, бизнес, пользователи).

◆ **Sprint Board (Scrum-доска)** — инструмент для визуализации задач спринта (To Do / In Progress / Done).

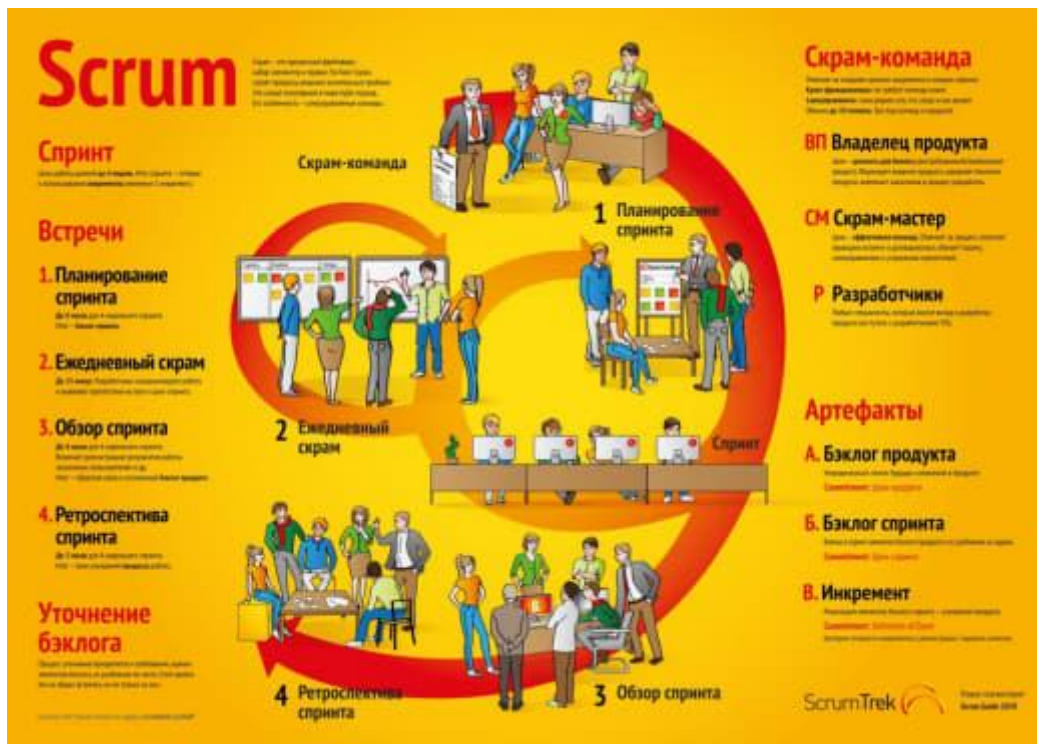
◆ Как это выглядит в работе по шагам:

1. Команда делает MVP → минимальную версию продукта
2. Показывает его Stakeholders → получает обратную связь
3. Product Owner обновляет Product Backlog
4. Команда берет задачи в Sprint
5. Все задачи отображаются на Sprint Board
6. Developers (включая QA) двигают задачи по колонкам
7. Команда видит прогресс в реальном времени
8. В конце спринта показывает результат

🕒 Итог за 10 секунд:

- Суть: MVP — минимум для старта, Stakeholder — даёт фидбек, Board — показывает прогресс
- Когда использовать: в каждом Scrum-проекте
- Частая ошибка: делать “идеальный продукт” вместо MVP
- Лучшая практика: быстро делать MVP и получать обратную связь
- Исключение: критические системы требуют полной версии сразу
- Предостережение: без визуализации задач команда теряет контроль

12:49



Шпаргалка

#scrum

13:27

Тестирование ПО

23	24	25	26	27	28	29
09:00 Daily Scrum 10:00 Sprint Planning	09:00 Daily Scrum	09:00 Daily Scrum 14:00 Refinement	09:00 Daily Scrum 15:00 Bug Triage	09:00 Daily Scrum		
30	31	1 week	2	3	4	5
09:00 Daily Scrum	09:00 Daily Scrum	09:00 Daily Scrum 14:00 Refinement	09:00 Daily Scrum	09:00 Daily Scrum 16:00 Sprint Review 17:00 Retrospective		

Как Scrum выглядит в реальной работе — календарь спринта

1. **Sprint Planning** (в начале спринта)
 - команда выбирает задачи и планирует работу
2. **Daily Scrum** (каждый день, 15 минут)
 - обсуждают прогресс и блокеры
3. **Backlog Refinement** (1 раз в неделю или чаще)
 - уточняют требования + задают вопросы + оценивают задачи
4. **Разработка и тестирование (ежедневно)**
 - Developers (включая QA) делают задачи
5. **Bug Triage** (по необходимости)
 - приоритизируют баги
6. **Sprint Review** (в конце спринта)
 - показывают результат заказчику
7. **Demo** (часто вместе с Review)
 - демонстрируют функциональность
8. **Sprint Retrospective** (в конце спринта)
 - обсуждают, как улучшить процесс

16:10

Тестирование ПО

Техники тест-дизайна

◆ Техники тест-дизайна

Это способы подбора тестовых данных и сценариев, чтобы проверить систему максимально эффективно с минимальным количеством тестов.

◆ **Equivalence Partitioning (классы эквивалентности)** — техника тестирования, при которой входные данные делятся на группы (классы), внутри которых система ведет себя одинаково. Проверяем один представитель из группы вместо всех значений.

◆ **Boundary Value Analysis (граничные значения)** — техника тестирования, при которой проверяются значения на границах диапазонов, где чаще всего возникают ошибки.

18:46

Тестирование ПО

□ Эквивалентное разбиение и граничные значения

◆ **Эквивалентное разбиение (Equivalence Partitioning)** — это деление входных данных на группы, внутри которых система работает одинаково.

Граничные значения (Boundary Values) — это значения на стыке этих групп, где чаще всего происходят ошибки.

◆ Как это выглядит в работе по шагам:

1. QA анализирует требование (например: ввод числа от 1 до 100)
2. Делит данные на классы:
валидные (1-100)
невалидные (<1, >100)
3. Выбирает по одному значению из каждого класса
4. Определяет границы: 1 и 100
5. Проверяет значения на границах и рядом:
0, 1, 2
99, 100, 101
6. Пишет тест-кейсы
7. Выполняет тестирование
8. Анализирует ошибки

☞ Итог за 10 секунд:

- Суть: делим данные на группы и тестируем каждую + границы
- Когда использовать: любые поля с диапазонами и правилами
- Частая ошибка: не проверяют значения 17/18 и 54/55
- Лучшая практика: классы + границы + ± 1 + невалидные типы
- Исключение: когда значения фиксированные (enum, boolean)
- Предостережение: нельзя тестировать только “середину”

[#Эквивалентное разбиение](#) [#граничные значения](#) [#equivalencepartitioning](#) [#boundarytesting](#) [#тестирование](#)

18:56

□ Анализ граничных значений (где реально ломается система)

◆ Анализ граничных значений (Boundary Value Analysis)

Метод тест-дизайна, при котором проверяются значения на границах диапазонов и рядом с ними, потому что именно там чаще всего возникают ошибки (по ISTQB и практике QA).

✦ Как это выглядит в работе по шагам

1. QA анализирует диапазоны из требований
пример: 0-17, 18-54, 55+
2. QA определяет границы каждого диапазона
0 и 17
18 и 54
55
3. QA выбирает значения на границе
0, 17, 18, 54, 55
4. QA добавляет значения рядом с границей (± 1):
-1, 0, 1
17, 18
54, 55, 56
5. QA проверяет нижнюю границу и значение до неё
пример: -1 → ошибка, 0 → корректно
6. QA проверяет верхнюю границу и значение после неё
пример: 17 → 50%, 18 → 25%
7. QA повторяет это для всех диапазонов
каждый диапазон = отдельная проверка
8. QA формирует тест-кейсы с ожидаемыми результатами

✦ Пример (реальный кейс):

- что тестируем

Поле "Возраст" при расчёте скидки

- что проверяем

Корректность перехода между диапазонами

- разбор

0-17 → 50%

18-54 → 25%

55+ → 75%

Проверка границы:

17 → 50%

18 → 25%

Проверка рядом:

16 → 50%

19 → 25%

- возможный баг

Возраст 17 даёт 25% из-за ошибки в условии ($\geq$ / $>$ перепутаны).

👉 Итог за 10 секунд:

- Суть: тестируем границы и значения рядом с ними
- Когда использовать: любые диапазоны чисел (возраст, сумма, рейтинг)
- Частая ошибка: проверяют только границу, но не ± 1 рядом
- Лучшая практика: граница + до неё + после неё
- Исключение: когда нет упорядоченных данных (например, строки)
- Предостережение: нельзя полагаться только на эквивалентное разбиение

13:24

□ Попарное тестирование (Pairwise) — как сократить 96 тестов до 16

◆ Попарное тестирование (Pairwise Testing / All Pairs)

Техника тест-дизайна, при которой тестируются все комбинации по парам параметров, а не полный перебор, потому что большинство багов возникает при взаимодействии двух факторов.

#Попарное тестирование

15:46

Тестирование ПО

□ Попарное тестирование (Pairwise testing)

◆ Попарное тестирование (Pairwise testing)

Техника тест-дизайна, при которой тестируются все возможные комбинации **пар параметров** (входных данных), чтобы покрыть максимум сценариев с минимальным числом тестов. Основана на том, что большинство багов возникает при взаимодействии **двух факторов**, а не всех сразу.

◆ Пример:

Фильтр интернет-магазина:

- Цвет: красный / синий
- Размер: S / M
- Доставка: курьер / самовывоз

Всего комбинаций = $2 \times 2 \times 2 = 8$

Попарное тестирование даст ~4-6 тестов, но покроет все пары:
(цвет+размер), (цвет+доставка), (размер+доставка)

◆ Виды (подходы формирования пар)

— All Pairs (алгоритмический метод)

Автоматически генерирует набор тестов, покрывающий все пары параметров, позволяет генерировать все пары автоматически. Самый популярный и используемый в реальной работе.

◆ Пример:

Инструменты сами создают минимальный набор тестов — ты просто запускаешь их

★ Инструменты:

PICT (самый популярный)

AllPairs

[Pairwise Tool](#)

<https://www.pairwise.org/> – инструменты для попарного тестирования

Какие шаги нужно сделать?

1. Создадим таблицу с параметрами и значениями. Всегда начинаем с параметра, у которого больше всего значений. Используем инструмент Pairwise Tool.

2. Для генерации пар необходимо нажать кнопку Generate Pairwisw

3. После нажатия сформируется таблица с необходимыми тестовыми данными. Количество проверок сократится с 96 до 16. Таблицу необходимо использовать для создания чек-листа или тест-кейсов.

— Ортогональные массивы (Orthogonal Arrays)

Математический способ построения комбинаций с равномерным покрытием. Используется редко, требует подготовки и знаний математики.

◆ Пример:

Используется в сложных системах (железо, embedded), где важна строгая структура тестов

◆ Расширенные вариации

◆ t-wise тестирование

Обобщение попарного тестирования:

- Pairwise = 2-wise (пары)
- 3-wise = комбинации из 3 параметров
- 4-wise и т.д.

→ Пример:

Если баги возникают при сочетании **трёх условий**, обычного pairwise уже недостаточно

◆ Ограниченное попарное тестирование (Constraints-based)

Когда не все комбинации допустимы (есть бизнес-ограничения).

→ Пример:

- Оплата "наличными" невозможна при "доставке онлайн"
- такие комбинации исключаются из генерации

◆ Когда использовать (очень важно):

- Много параметров (3+)
- У каждого параметра несколько значений
- Невозможно проверить все комбинации (слишком долго)

☞ Итог за 10 секунд:

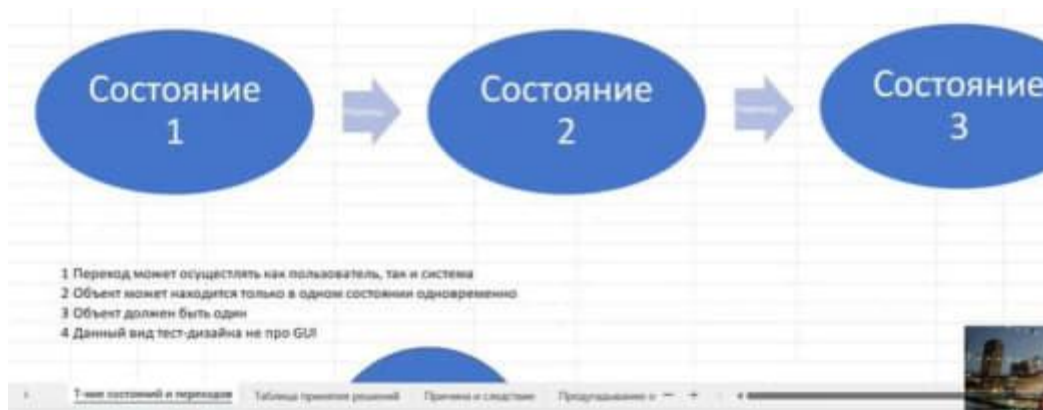
- Суть: проверяем все пары параметров вместо всех комбинаций
- Когда использовать: фильтры, формы, настройки, конфигурац#Попарное тестированиеии
- Частая ошибка: думать, что покрывает ВСЕ баги (покрывает не все)
- Лучшая практика: использовать All Pairs + учитывать ограничения
- Исключение: критичные системы (там нужен 3-wise и выше)
- Предостережение: не делай вручную при сложных данных — используй инструменты

#Попарное тестирование

16:29

Тестирование ПО

16:36



□ Таблица решений и состояний (краткая теория)

◆ Таблица принятия решений (Decision Table)

Техника тест-дизайна, которая систематизирует **все комбинации условий и соответствующие результаты** в виде таблицы для полного покрытия логики.

→ Пример:

Условия: логин/пароль

Комбинации → ожидаемый результат (успех / ошибка)

◆ Таблица состояний (State Table)

Техника, описывающая систему как набор **состояний (state — текущее положение системы)** и проверяющая поведение в каждом состоянии.

→ Пример:

Статусы заказа: новый → оплачен → отправлен

◆ Таблица переходов состояний (State Transition Table)

Техника, которая фиксирует **переходы между состояниями** в зависимости от действий или событий.

→ Пример:

"Оплатить" → переводит заказ из "новый" в "оплачен"

16:37

☞ Итог за 10 секунд:

- Суть: Decision Table — условия → результат, State — состояние → переход
- Когда использовать: сложная логика или системы с этапами
- Частая ошибка: не учитывать все комбинации и переходы
- Лучшая практика: проверять и валидные, и невалидные сценарии
- Исключение: простые сценарии без логики
- Предостережение: пропуск одного условия или состояния = пропущенный баг

16:41

◆ Главное отличие (очень важно):

Decision Table → проверяет ЛОГИКУ условий

State Testing → проверяет ЖИЗНЕННЫЙ ЦИКЛ системы

16:42

◆ Чек-лист (Checklist)

Список проверок без шагов, который фиксирует что нужно проверить, но не описывает как. Набор проверок в виде коротких пунктов без детальных шагов. В 2026 году используется для быстрого регресса (повторного тестирования после изменений), исследовательского тестирования (exploratory testing — свободный поиск багов без сценариев) и как черновик для будущих тест-кейсов.

◆ Тест-кейс (Test Case)

Подробное описание проверки: включает шаги (steps), входные данные (test data) и ожидаемый

результат (expected result). Документированный сценарий с предусловиями, пошаговыми действиями, тестовыми данными и чётким ожидаемым результатом. В 2026 году хранится в TMS (test management system — система управления тестами: Qase, TestRail, Allure TestOps) и содержит метаданные: приоритет, компонент, автотест-статус.

13:58

Тестирование ПО

◆ Чек-лист (Checklist)

Пример:

- форма отправляется
- ошибка при пустом поле
- email валидируется

★ Обязательные части чек-листа

→ Шапка

Информация о тестировании:

- * проект
- * версия (build — сборка приложения)
- * окружение (environment — где тестируем: браузер, ОС)
- * тестировщик, дата
- Тестируемые модули, submodule (регистрация, аутентификация, авторизация)
- Список проверок: они должны отражать основную суть, без лишней детализации
- Статус (passed/failed)

Дополнительные части чек-листа

Ожидаемый результат

Типы тестирования

Отчеты о дефекте

Заметки

◆ Тест-кейс (Test Case)

Пример:

Открыть страницу регистрации

Ввести email "test@mail.com"

Нажать "Отправить"

📖 Ожидаемый результат: регистрация успешна

★ Атрибуты тест-кейса

- *Идентификатор (ID)*: уникальный номер, необходимый для прослеживаемости. В системах по управлению кейсами (TMS – test management system) проставляется автоматически
- *Приоритет (Priority)*: срочность и важность выполнения задачи
- *Требование (Requirement)*: ссылка на требование, для проверки которого служит кейс
- *Модуль (Module)*: название структурной части, в которой находится предмет тестирования
- *Заголовок (Title)*: Отражает суть проверки
- *Тестовые данные и предусловия (input data, test data, preconditions)*: информация о данных, которые необходимы для тестирования (данные для ввода, файлы с определенным расширением и размером и т.д.) + специальное состояние системы до начала тестирования (пользователь зарегистрирован, созданы объекты в базе данных и т.д.)
- *Шаги (Steps)*: последовательность действий для получения ожидаемого результата
- *Ожидаемые результаты (Expected results)*: по каждому шагу тест-кейса описывают реакцию приложения на действия, описанные в поле «шаги тест-кейса». Номер шага соответствует номеру результата.
- *Постусловия (Postconditions)*: возвращение системы в исходное состояние (удаление данных,

пользователей, отключение виртуальной машины и т.д.)

! Обязательными атрибутами для тест-кейса являются: идентификатор, приоритет, заголовок, шаги и ожидаемые результаты.

◆ Ключевая разница

Чек-лист → что проверять

Тест-кейс → как проверять

◆ Когда использовать

Чек-лист:

- регресс (повторные проверки)
- smoke (быстрая проверка)
- когда логика простая

Тест-кейс:

- сложная логика
- новые фичи
- обучение команды

◆ Плюсы и минусы

Чек-лист:

- + быстро писать
- + легко поддерживать
- зависит от опыта QA

Тест-кейс:

- + точность и воспроизводимость
- + подходит новичкам
- долго писать и поддерживать

◆ Частая ошибка

писать чек-лист как тест-кейс

не указывать ожидаемый результат в тест-кейсах

☞ Итог за 10 секунд:

- Суть: чек-лист — кратко, тест-кейс — подробно
- Когда использовать: чек-лист — быстро, тест-кейс — для сложного
- Частая ошибка: путать их между собой
- Лучшая практика: комбинировать оба подхода
- Исключение: простые фичи → только чек-лист
- Предостережение: без тест-кейсов сложную логику легко сломать

#чек-лист #тест-кейс

14:42

Тестирование ПО

Ссылка на регистрацию в QASE <https://qase.io/>

14:44

□ QASE (Test Management System)

◆ QASE

Система управления тестированием (Test Management System — инструмент для хранения, управления и запуска тестов), которая помогает QA организовывать тест-кейсы, чек-листы, тест-раны и отчёты. Актуальна и широко используется в 2026 году.

Основные функции в QASE

◆ Test Cases (Тест-кейсы)

Хранение и создание тест-кейсов

- шаги (steps)
- ожидаемый результат (expected result)
- приоритет (priority — важность)

Пример:

Создаешь тест-кейс "Регистрация пользователя" и описываешь шаги

◆ Test Suites (Наборы тестов)

Группировка тестов по логике

- по модулям
- по фичам

Пример:

Suite "Авторизация" → логин, регистрация, восстановление пароля

◆ Test Runs (Тест-прогоны)

Запуск тестов и фиксация результатов

Статусы:

- passed (пройдено)
- failed (не пройдено)
- blocked (заблокирован)

◆ Bug Tracking (Связка с багами)

Интеграция с баг-трекерами (например: Jira)

→ можно привязать баг к тесту

◆ Checklists

Поддержка чек-листов

→ быстрые проверки без шагов

◆ Shared Steps (Общие шаги)

Повторно используемые шаги

→ не нужно копировать одинаковые действия

◆ Tags (Теги)

Маркировка тестов

- regression
- smoke
- api

◆ Reports (Отчёты)

Аналитика по тестированию

- сколько тестов прошло
- сколько упало

- покрытие

◆ API и интеграции

- CI/CD (автоматические прогоны)
- Jira (баги)
- GitHub

👉 Итог за 10 секунд:

- Суть: QASE — инструмент для управления тестами
- Когда использовать: любой реальный проект с тестированием
- Частая ошибка: превращать в "кладбище тестов"
- Лучшая практика: структура + актуальность + теги
- Исключение: маленькие проекты (можно без него)
- Предостережение: если не поддерживать тесты — инструмент бесполезен

#QASE #qase

17:23

Тестирование ПО

◆ Test IT (Test Management System)

Система управления тестированием, которая помогает QA хранить тесты, управлять прогонами и отслеживать качество продукта в одном месте. ()

Ссылка на регистрацию TestIT <https://testit.software/>

#TestIT #testIT

17:35

◆ Основные функции Test IT (Test Management System)

— Управление тест-кейсами

Создание, хранение и структурирование тестов

- тест-кейсы
- тест-сьюты (группы тестов)

— Test Runs (прогоны)

Запуск тестов и фиксация результатов

- * passed / failed / blocked
- * отслеживание прогресса

— Хранение тестовой документации

Все тесты и результаты хранятся централизованно

→ удобно для команды

— Отчёты и метрики

Показывает:

- * качество релиза
- * прогресс тестирования
- * загрузку команды ()

— Интеграции

Поддержка связи с:

- * CI/CD (автоматизация)
- * баг-трекеры
- * автотесты ()

— Работа с manual + auto тестами

Поддерживает:

- * ручное тестирование
- * автоматизированные тесты в одной системе ()

#TestIT #testIT

14 April 2026

13:33

Тестирование ПО

Атрибуты (поля) отчёта о дефекте			
Общий вид всей структуры отчёта о дефекте представлен			
Идентификатор	Краткое описание	Подробное описание	Шаги по воспроизведению
19	Бесконечный цикл обработки входного файла с атрибутом «только для чтения»	Если у входного файла выставлен атрибут «только для чтения», после обработки приложению не удаётся переместить его в каталог-приёмник: создаётся копия файла, но оригинал не удаляется, и приложение снова и снова обрабатывает этот файл и безуспешно пытается переместить его в каталог-приёмник. Ожидаемый результат: после обработки файл перемещён из каталога-источника в каталог-приёмник. Фактический результат: обработанный файл копируется в каталог-приёмник, но его оригинал остаётся в каталоге-источнике. Требование: ДС-2.1.	1. Поместить в каталог-источник файл допустимого типа и размера. 2. Установить данному файлу атрибут «только для чтения». 3. Запустить приложение. Дефект: обработанный файл появляется в каталоге-приёмнике, но не удаляется из каталога-источника, файл в каталоге-приёмнике непрерывно обновляется (видно по значению времени последнего изменения).

🐛 Отчёт о дефекте (Bug Report)

◆ Дефект (Bug / Defect)

Ошибка в системе, которая приводит к неправильному результату, сбою или невозможности использовать функциональность.

◆ Отчёт о дефекте (Bug Report)

Документ, в котором QA фиксирует баг так, чтобы разработчик смог воспроизвести и исправить его.

#Bug Report #отчёт о дефекте

13:35

◆ Основные атрибуты отчета о дефекте

📄 ID (идентификатор)

Уникальный номер бага в системе

📄 Summary (краткое описание)

Кратко отвечает на 3 вопроса:

- что произошло
- где произошло
- при каких условиях

◆ Пример:

"Ошибка при регистрации с пустым email"

3 Description (подробное описание)

Развернутое описание проблемы

- фактический результат (actual result — что произошло)
- ожидаемый результат (expected result — как должно быть)
- ссылка на требования

4 Steps to Reproduce (STR, шаги воспроизведения)

Пошаговая инструкция, как получить баг

Пример:

Открыть страницу регистрации

Оставить email пустым

Нажать "Отправить"

5 Environment (окружение)

Где найден баг:

- ОС (операционная система)
- браузер
- устройство

6 Severity (серьёзность — влияние на систему)

- Blocker – приводит систему в нерабочее состояние.
- Critical — система не работает / краш/ потеря данных/неверная работа функций (то есть система работает но не так как требуется)
- Major — важная функция сломана
- Medium — есть баг, но есть обход
- Minor — мелкий дефект

7 Priority (приоритет — срочность исправления)

- ASAP — срочно
- High — высокий
- Normal — стандарт
- Low — можно отложить

8 Attachments (вложения)

Доказательства бага:

- скриншоты
- видео
- логи

9 Comments (комментарии)

Дополнительная информация, обсуждение

10 Status (статус)

Этап жизни бага:

- New → Open → In Progress → Fixed → Closed

! Важно понять (ключевое отличие)

Severity → насколько баг серьёзный

Priority → как быстро его чинить

👉 Итог за 10 секунд:

- Суть: баг-репорт = инструкция, как найти и понять баг
- Когда использовать: всегда при обнаружении дефекта
- Частая ошибка: писать "не работает" без деталей
- Лучшая практика: чётко, кратко, воспроизводимо
- Исключение: нет (это база QA)
- Предостережение: плохой баг-репорт = баг не исправят

#Bug Report #отчёт о дефекте

16:50

Тестирование ПО



Рисунок 2.5.с — Жизненный цикл отчёта о дефекте с наиболее типичными переходами между состояниями

Источник: Святослав Куликов "Тестирование ПО. Базовый курс."

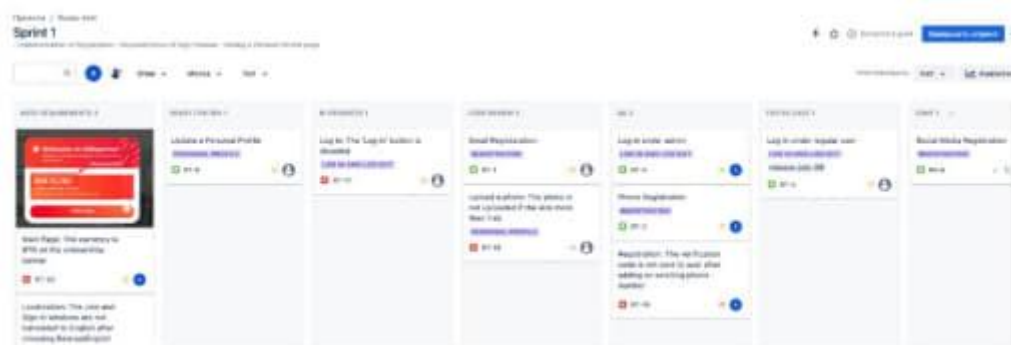
Жизненный цикл дефекта. Спринт



На реальных проектах вы будете работать в багтрекинг-системах по типу Jira на специальных досках. Обычно они представляют собой бэклог текущего спринта, элементы которого находятся в разных статусах.

Жизненный цикл есть не только у дефектов, но и у всех задач и зависит от принятого процесса (workflow) в команде.

16:50



21:06

Тестирование ПО

ShareX — бесплатное программное обеспечение с открытым исходным кодом для создания скриншотов и скринкастов. Разработчик — ShareX Team.

#скрины



Ошибки, сбои, отказы, улучшения

◆ Ошибка (Error)

Действие человека (разработчика, аналитика), которое приводит к неправильной логике или результату.

◆ Дефект (Defect / Bug)

Ошибка, которая попала в систему и проявляется как несоответствие требованиям.

◆ Отказ (Failure)

Ситуация, когда система фактически не выполняет свою функцию во время работы.

◆ Сбой

Самоустраняющийся отказ или однократный отказ, устраняемый незначительным вмешательством оператора

! Связь (очень важно):

Error → Defect → Failure

☞ человек ошибся → баг в системе → пользователь увидел проблему

16:23



Типы улучшений

◆ Improvement

Запрос на улучшение существующего функционала

◆ Feature

Добавление новой функциональности

◆ Change Request

Изменение текущей логики или поведения системы

Пример:

Изменить расчет скидки

◆ Enhancement

Расширение или развитие функциональности (улучшенная версия feature/improvement)

Пример:

Добавить фильтры к уже существующему поиску

!Важно понимать

Bug → это ошибка (чинить обязательно)

Improvement / Feature → это идея (делать по приоритету бизнеса)

16:41

□ Azure DevOps – Платформа от Microsoft для управления разработкой ПО: планирование, разработка, тестирование и доставка продукта в одном месте.

Ссылка на регистрацию в Azure Devops <https://azure.microsoft.com/en-us/pricing/details/devops/azure-devops-services/>

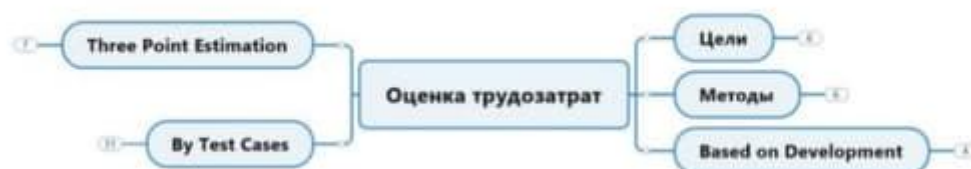
Ссылка на документацию <https://docs.microsoft.com/en-us/azure/devops/test/?view=azure-devops>

17:40

□ YouTrack – Система управления задачами и багами (issue tracker — трекер задач) от JetBrains, которая используется для ведения задач, багов и процессов разработки.

★ Ссылка на регистрацию в Youtrack <https://www.jetbrains.com/youtrack/>

17:52



Оценка трудозатрат (Estimation) –

Процесс определения сколько времени, ресурсов и усилий потребуется на тестирование. Используется для планирования сроков и нагрузки.

◆ Что оценивает QA

- время на написание тестов
- время на выполнение тестов
- время на баги (поиск + ретест)

- подготовка окружения
- регресс (повторное тестирование)

□ Основные методы оценки трудозатрат

1 Экспертная оценка (Expert Judgment)

Оценка на основе опыта

2 Аналогия (Analogous Estimation)

Сравнение с похожими задачами

3 Декомпозиция (Work Breakdown)

Разбивка задачи на части

4 Трёхточечная оценка (Three-point estimation)

$(O + 4M + P) / 6$

#оценка трудозатрат

17 April 2026

13:01

Тестирование ПО

□ Методы оценки трудозатрат (разбор с примерами)

◆ Экспертная оценка (Expert Judgment)

Оценка, основанная на **опыте и знаниях специалиста**, который уже делал похожие задачи.

Пример:

QA смотрит на задачу и говорит:

☞ "Форма регистрации — примерно 3 часа"

Как производится:

1. Анализ задачи
 2. Сравнение с прошлым опытом
 3. Быстрая оценка "на глаз"
- ☞ используется быстро, без расчётов

◆ Аналогия (Analogous Estimation)

Оценка через **сравнение с похожей задачей**, которая уже была выполнена ранее.

Пример:

Была задача "фильтр товаров" — заняла 1 день

Новая похожая задача → тоже ~1 день

Как производится:

1. Найти похожую задачу
2. Взять её оценку
3. Скорректировать (сложнее/проще)

◆ Декомпозиция (Work Breakdown)

Оценка через **разбиение задачи на мелкие части** и оценку каждой отдельно.

Пример:

Тестирование формы:

- анализ — 1 час
- тест-кейсы — 2 часа
- выполнение — 3 часа

— баги — 2 часа

☞ итог: 8 часов

Как производится:

1. Разбить задачу на этапы
2. Оценить каждый этап
3. Сложить всё вместе

☞ самый точный метод

◆ Трёхточечная оценка (Three-point estimation)

Оценка с учётом **рисков и неопределённости** через 3 значения:

- O (optimistic) — если всё идеально
- M (most likely) — реалистично
- P (pessimistic) — если всё плохо

Формула:

$$(O + 4M + P) / 6$$

Также рассчитывается стандартное отклонение: $(P - O)/6$

Финальный вариант:

$((O + 4M + P) / 6) \pm ((P - O)/6)$, ответ записывается как результат первого выражения $\pm$ результат второго.

Пример:

$$O = 4 \text{ часа}$$

$$M = 8 \text{ часов}$$

$$P = 16 \text{ часов}$$

$$\text{☞ } (4 + 4 \times 8 + 16) / 6 = 9.3 \text{ часа}$$

◆ Когда какой метод использовать

- Экспертная → быстро, мало времени
- Аналогия → есть похожий опыт
- Декомпозиция → нужна точность
- Трёхточечная → есть неопределённость

☞ Итог за 10 секунд:

- Суть: 4 метода — опыт, сравнение, разбиение, математика
- Когда использовать: планирование задач
- Частая ошибка: не учитывать риски
- Лучшая практика: комбинировать методы
- Исключение: очень простые задачи
- Предостережение: одна оценка $\neq$ точный результат

#оценка трудозатрат

13:48

Тестирование ПО



◆ **Test Plan** – Документ, который описывает **что тестируем, как тестируем, когда и кто это делает**.
Используется для планирования и контроля всего процесса тестирования.

◆ **Master Test Plan** (главный тест-план)

Общий план для всего проекта — объединяет несколько уровней тестирования (например: UI, API, интеграционное).

◆ **Level Test Plan** (уровневый тест-план)

План для **конкретного уровня тестирования** (например только API или только мобильное приложение).

Пример

Ты работаешь над интернет-магазином:

— Master Test Plan → описывает всё тестирование проекта

— Level Test Plan → отдельный план только для тестирования корзины или API

#тест план

13:55

■ Тест-план (Test Plan) — основа работы тестировщика

◆ **Структура Test Plan (что внутри)**

- 1 Цель (зачем тестируем)
- 2 Что тестируем (features to be tested)
- 3 Что НЕ тестируем (важно!)
- 4 Стратегия (как тестируем: ручное, авто, виды тестов)
- 5 Критерии (когда начать и закончить)
- 6 Ресурсы (люди, техника, окружение)
- 7 Сроки (дедлайны)
- 8 Роли (кто за что отвечает)
- 9 Риски (что может пойти не так)
- 10 Документация
- 11 Метрики (цифры качества, например % багов)

◆ **Entry Criteria** (когда МОЖНО начинать тестирование)

Условия, без которых тестирование не имеет смысла

Примеры:

- есть билд (сборка приложения)
- требования согласованы

◆ **Exit Criteria** (когда МОЖНО закончить тестирование)

Условия, при которых тестирование считается завершённым

Примеры:

- протестировано 80% кейсов
- нет критических багов
- регрессия покрыта автотестами

◆ **Пример из реальной работы**

Ты не начинаешь тестировать, если:

- нет билда
- требования сырые

Ты не заканчиваешь тестирование, если:

- есть критические баги
- не проверен основной функционал

◆ **Виды тест-планов (согласно стандарту и схеме):**

1 Мастер-план (Master Test Plan):

- Для кого: Для всех команд на одном большом проекте.
- Суть: Глобальная координация, стандарты качества, общие инструменты.

2 Детальный план (Detailed Test Plan):

- Для кого: Для каждой команды, конкретной итерации (спринта) или релиза.
- Суть: Конкретные тест-кейсы, расписание на неделю/две, назначенные исполнители.

3 План приёмочных испытаний (Acceptance Test Plan):

- Для кого: Для заказчика или конечного пользователя.
- Суть: Фокус на бизнес-сценариях (UAT — User Acceptance Testing), проверка, что продукт решает задачу бизнеса.

Что обязательно входит в план (Чек-лист 2026):

- Сроки: Когда начинаем, когда заканчиваем, дедлайны.
- Окружение: На каких устройствах/браузерах/ОС проверяем (стенд Staging/Pre-prod).
- Ресурсы: Кто именно тестирует (имена или роли), доступы.
- Критерии входа/выхода: Когда начинаем (билд готов) и когда заканчиваем (0 критических багов).
- Приоритеты: Что тестируем в первую очередь (Critical/High), если мало времени.
- Риски: Что может пойти не так и план «Б».

✂ Важно знать новичку:

- Тест-план — это не догма. Если нашли критический баг, план меняется (остановка тестирования).
- В Agile (2026) детальный план часто заменяют: «Backlog спринта» (список задач) + «Checklist» (чек-лист проверок).
- Master-план пишется редко, обычно только в энтерпрайзе (банки, госсектор).

👉 **Итог за 10 секунд:**

- Суть: Test Plan — это план всей работы тестировщика
- Когда использовать: на любом проекте перед началом тестирования
- Частая ошибка: писать формально и не использовать в работе
- Лучшая практика: делать короткий и реально применимый план

- Исключение: маленькие проекты (может быть упрощённый план)
- Предостережение: нельзя начинать тестирование без критериев входа

#тест план

16:55

Тестирование ПО

III Отчет о тестировании (Test Progress Report) — как принимают решение о релизе

(Test Progress Report) — как принимают решение о релизе

◆ Test Progress Report

Документ, который показывает **текущее состояние тестирования**: что уже проверено, какие есть проблемы и можно ли выпускать продукт.

◆ Цель отчета

Дать **оценку качества продукта** и ответ на главный вопрос:

☞ можно релизить или нет

Пример

Ты протестировал фичу “оплата”:

- найдено 15 багов
- 3 из них критические
- протестировано 70% кейсов

→ В отчете ты пишешь:

✗ релиз сейчас рискованный

✓ нужно исправить критические баги

◆ Что входит в отчет (структура)

1 Summary (краткое описание)

Главное: прогресс, проблемы, вывод

2 Test Team (команда)

Кто тестировал

3 Testing Process

Что именно делали (например: тестировали API, UI)

4 Timetable (расписание)

Когда и сколько тестировали

5 New Defects Statistics

Сколько новых багов нашли

6 New Defects List

Список этих багов

7 Overall Defects Statistics

Общая статистика по всем багам (открытые, закрытые)

8 Recommendations

Рекомендации (релиз / не релиз / доработать)

9 Appendixes

Графики, таблицы, диаграммы

◆ Пример из реальной работы

В конце спринта ты пишешь:

- протестировано 85% функционала
- осталось 2 критических бага
- найдено 40 багов всего

→ *Рекомендация:*

✗ не выпускать в прод

или

⚠ выпускать с риском (если бизнес давит)

👉 Итог за 10 секунд:

- Суть: отчет показывает текущее качество продукта
- Когда использовать: в конце спринта / перед релизом
- Частая ошибка: писать без выводов и рекомендаций
- Лучшая практика: всегда давать четкий вывод “релиз / не релиз”
- Исключение: маленькие команды — отчет может быть устным
- Предостережение: нельзя скрывать критические баги

#отчет о тестировании #test progress report

20 April 2026

16:12

Тестирование ПО

👉 Тестовая стратегия (Test Strategy) — как команда будет тестировать продукт

◆ Test Strategy

Документ или раздел в документации, который описывает **общий подход к тестированию проекта**: что проверяем, какими методами, какими инструментами, кто участвует и какие риски учитываем.

Это **не список тест-кейсов**, а именно правила игры для всей команды QA.

✦ *Простыми словами*

Test Plan отвечает: **что делаем и когда**

Test Strategy отвечает: **как именно будем тестировать**

Пример:

Strategy: “критические сценарии автоматизируем”

Plan: “на этой неделе проверить checkout и доставку”

✦ Что ещё важно знать тестировщику в 2026:

- Тест-стратегия ≠ тест-план: стратегия — это «что и зачем» (глобально), план — «кто, когда и как» (конкретные задачи и сроки)
- Стратегия пишется один раз на проект/продукт, но обновляется при смене технологий, команды или бизнес-целей
- В 2026 году в стратегию обязательно включают: подход к тестированию ИИ-фич (если есть), безопасность (OWASP), доступность (WCAG), работу с данными (GDPR/локализация)
- Стратегия согласуется с разработчиками, продактом и девопсами — это командный документ
- Если стратегии нет — ты тестируешь вслепую: тратишь время не на то, пропускаешь риски, не можешь обосновать качество

◆ Лучшая практика Senior QA

1. Понять, где деньги бизнеса
2. Понять, где чаще ломается
3. Самое важное тестировать первым

4. Автоматизировать повторяемое
5. Регулярно обновлять стратегию

◆ **Компоненты тест-стратегии (разделы документа, определяющие как, чем и в каких границах проводится тестирование)**

В 2026 стратегия — живой документ в Confluence/Notion с ссылками на автотесты и пайплайны. Отвечает на 7 вопросов: Что? Чем? Как? Когда? Кто? Где? Какие риски?

✦ Ядро стратегии (14 разделов):

1. Цели и бизнес-ценность
2. Область (in-scope / out-of-scope)
3. Подходы (ручное/авто, техники тест-дизайна)
4. Уровни (модульное → интеграционное → системное → UAT)
5. Типы (функциональное, безопасность, производительность, доступность, ИИ-валидация)
6. Окружение и тестовые данные
7. Инструменты (фреймворки, CI/CD, баг-трекер)
8. Автоматизация (что, порог покрытия, поддержка)
9. Роли и ответственность
10. Критерии входа/выхода/приостановки
11. Управление рисками (матрица + планы)
12. Метрики и отчётность (DRE, время прогона, покрытие)
13. Регламенты (баг-фикс, ретест, релиз)
14. Глоссарий (термины проекта)

☞ **Итог за 10 секунд:**

- Суть: Тест-стратегия — это единый документ, который задаёт правила игры в тестировании продукта, описывает как команда тестирует продукт. Стратегия = 14 разделов, задающих правила, инструменты и границы тестирования
- Когда использовать: при старте нового проекта, смене архитектуры, переходе на новые релиз-циклы или при онбординге нового тестировщика
- Частая ошибка: скопировать шаблон стратегии из другого проекта и не адаптировать под свои риски и технологии
- Лучшая практика: писать стратегию совместно с командой, хранить в одном источнике правды (Confluence/Notion) и ревьюить её раз в квартал
- Исключение: для мелких хотфиксов или однодневных задач стратегия не нужна — достаточно чек-листа
- Предостережение: никогда не начинай массовое тестирование без утверждённой стратегии — это гарантия хаоса и пропущенных критических багов

▮ Метрики в тестировании + Матрица трассировки (RTM)

◆ **Метрики в тестировании (Testing Metrics)** – это измеримые показатели в цифрах, по которым команда понимает: как идёт тестирование, насколько продукт покрыт проверками, сколько найдено дефектов и можно ли двигаться к релизу.

→ В ISTQB метрики делят на:

- **project metrics** (метрики проекта),
- **product metrics** (метрики продукта)
- **process metrics** (метрики процесса).

◆ Простыми словами

Метрика отвечает на вопрос “что у нас происходит на самом деле?”, а не “как нам кажется”.

Например: не “мы почти закончили”, а “выполнено 82% тестов, из них 9% failed, 2 critical бага ещё открыты”. Это уже факт, а не мнение. На это и опираются при test monitoring, test control и test completion. ()

◆ Зачем нужны метрики

- 1 Понять готовность к релизу
- 2 Видеть риски проекта
- 3 Контролировать прогресс тестирования
- 4 Улучшать процесс QA
- 5 Давать прозрачный статус менеджерам

#метрики

17:37

Тестирование ПО

◆ Виды метрик

— Project Metrics — метрики проекта

Показывают прогресс относительно плана и критериев выхода.

Типичные примеры: сколько тестов выполнено, сколько passed/failed, сколько часов уже потрачено по сравнению с оценкой, сколько автотестов реально сделано по сравнению с планом. ()

— Product Metrics — метрики продукта

Показывают состояние качества самого продукта.

Примеры: покрытие требований, code coverage, availability (доступность), response time (время ответа), defect density (плотность дефектов), residual risk (остаточный риск). ()

— Process Metrics — метрики процесса

Показывают насколько эффективно и предсказуемо работает сам процесс тестирования.

ISTQB выделяет для процесса такие важные свойства, как effectiveness (результативность), efficiency (эффективность по ресурсам) и predictability (предсказуемость). Эти метрики используют для улучшения процесса, а не только для отчётов. ()

◆ Основные метрики, которые реально нужно знать тестировщику

— Test Execution Progress — прогресс выполнения тестов

Показывает, сколько тестов уже выполнено.

→ Как считается

$$\text{Executed} / \text{Total} \times 100\%$$

Где:

Executed = сколько тестов уже выполнили

Total = сколько тестов всего запланировано

Пример

Запланировано 120 тестов. Выполнили 90.

$$90 / 120 \times 100\% = 75\%$$

Значит, выполнено 75% тестов.

Зачем нужна

Чтобы видеть, насколько команда близка к завершению проверки. Это одна из базовых project metrics. ()

— Fail Rate — процент упавших тестов

Показывает, какая доля выполненных тестов завершилась с ошибкой.

→ Как считается

$\text{Failed} / \text{Executed} \times 100\%$

Пример

18 failed из 90 executed.

$18 / 90 \times 100\% = 20\%$

Зачем нужна

Помогает увидеть, насколько билд нестабилен. Часто смотрится вместе с pass rate, а не отдельно.

— Blocked Rate — процент заблокированных тестов

Показывает, сколько тестов нельзя выполнить из-за внешней причины: нет окружения, нет данных, баг блокирует поток, сервис недоступен.

→ Как считается

$\text{Blocked} / \text{Planned} \times 100\%$

или

$\text{Blocked} / (\text{Executed} + \text{Blocked}) \times 100\%$

Главное — внутри команды использовать одну и ту же формулу всегда, чтобы метрика была сопоставимой.

— Пример

Из 120 тестов 12 заблокированы.

$12 / 120 \times 100\% = 10\%$

— Зачем нужна

Высокий blocked rate часто говорит не о качестве продукта, а о проблемах процесса, среды или зависимостей.

— Requirements Coverage — покрытие требований

Показывает, какая часть требований вообще покрыта тестами.

ISTQB отдельно подчёркивает, что traceability test cases ↔ requirements позволяет проверить, покрыты ли требования тест-кейсами. ()

→ Как считается

$\text{Covered Requirements} / \text{Total Requirements} \times 100\%$

Пример

Всего 40 требований. Для 34 есть тесты.

$34 / 40 \times 100\% = 85\%$

Зачем нужна

Чтобы не получилось так, что тест-кейсов много, а важное требование вообще никто не проверил.

— Product Risk Coverage — покрытие продуктовых рисков

Показывает, какая часть важных рисков уже закрыта тестами. ISTQB приводит эту метрику как одну из базовых для monitoring/control и completion. ()

→ Как считается

Обычно так:

$\text{Covered Risks} / \text{Total Identified Risks} \times 100\%$

Иногда риски ещё взвешивают по приоритету.

Пример

Выделили 10 ключевых рисков. Проверили 7.

$7 / 10 \times 100\% = 70\%$

Зачем нужна

Она полезнее обычного процента тестов, потому что показывает, закрыли ли вы именно опасные места, а не просто много мелких проверок.

— Defect Count — количество дефектов

Показывает, сколько багов найдено за период, релиз или цикл тестирования.

→ Как считается

Обычно просто считают число дефектов:

— new defects

— open defects

— fixed defects

— closed defects

— reopened defects

Пример

За спринт найдено 26 багов:

2 critical, 5 high, 11 medium, 8 low.

Зачем нужна

Чтобы видеть тенденцию: багов становится меньше или больше, и какого они уровня.

— Defect Severity Distribution — распределение багов по серьёзности

Показывает, насколько опасны найденные дефекты.

→ Пример

30 багов:

1 critical

4 high

15 medium

10 low

Зачем нужна

20 low багов и 1 critical — это не одно и то же. Для решения о релизе важен не только объём, но и структура дефектов.

17:37

— Defect Density — плотность дефектов

Показывает, сколько дефектов приходится на единицу объёма продукта.

ISTQB приводит defect density как пример defect metric. ()

→ Как считается

Обычно:

$\text{Number of Defects} / \text{Size of Component}$

———Где:

—————Number of Defects (количество дефектов) — это число подтверждённых багов, найденных в компоненте за определённый период (спринт/релиз/этап тестирования).

→ Учитываем: только баги со статусом Confirmed/Resolved (не дубликаты, не «отклонённые»)

→ Не учитываем: улучшения (enhancements), вопросы, задачи на рефакторинг

————— Size of Component (размер компонента) — это выбранная единица измерения «объёма» тестируемой части.

Варианты единицы размера (выбираете ОДИН и используете его постоянно):

- Модуль/сервис = 1 (например, «модуль оплаты»)
- Экран/страница = 1 (например, «страница корзины»)
- 1000 строк кода (KLOC — Thousand Lines of Code)
- User Story / задача = 1 (например, «история US-45»)
- API-эндпоинт = 1 (например, «метод POST /api/order»)

Пример:

Ты тестируешь модуль «Корзина» в интернет-магазине.

Вариант 1: Считаем на модуль

- Найдено: 15 подтверждённых багов
- Размер: 1 модуль («Корзина»)
- Defect Density = $15 / 1 = 15$ багов на модуль

Вариант 2: Считаем на 1000 строк кода (KLOC)

- Найдено: 15 багов
- Размер: модуль содержит 3000 строк кода = 3 KLOC
- Defect Density = $15 / 3 = 5$ багов на KLOC

Зачем нужна

Помогает найти самые проблемные зоны продукта.

— Defect Reopen Rate — процент переоткрытых дефектов

Показывает, сколько багов после «исправления» пришлось открыть снова.

→ Как считается

$\text{Reopened Defects} / \text{Fixed Defects} \times 100\%$

Пример

Исправили 20 багов. Из них 4 пришлось переоткрыть.

$4 / 20 \times 100\% = 20\%$

Зачем нужна

Если reopen rate высокий, значит:

- плохое качество фиксов,
- некачественная проверка фикса,
- плохая коммуникация по bug report.

— Automation Coverage — покрытие автоматизацией
Показывает, какая часть проверок автоматизирована.

→ Как считается

$\text{Automated Tests} / \text{Total Relevant Tests} \times 100\%$

Пример

Есть 200 регрессионных проверок. 120 автоматизированы.
 $120 / 200 \times 100\% = 60\%$

Зачем нужна

Чтобы понимать, насколько регрессия зависит от ручного труда. Но эту метрику нельзя оценивать без контекста: автоматизировать всё подряд — плохая идея.

— Actual vs Planned Effort — факт против плана по времени
ISTQB приводит actual vs planned estimation in hours как пример тестовой метрики. ()

→ Как считается

$\text{Actual Hours} / \text{Planned Hours} \times 100\%$

или просто сравнивают факт и план рядом.

Пример

Планировали 40 часов на регрессию, ушло 52 часа.
 $52 / 40 \times 100\% = 130\%$ от плана

Зачем нужна

Показывает ошибки в оценке, неожиданные риски и узкие места процесса.

— Actual vs Planned Cost — факт против плана по стоимости
Тоже приведена ISTQB как пример test management metric. ()

→ Как считается

$\text{Actual Cost} / \text{Planned Cost} \times 100\%$

Пример

Планировали 1000 у.е. на тестирование релиза, потратили 1200.
 $1200 / 1000 \times 100\% = 120\%$

Зачем нужна

Чаще используется менеджерами, но тестировщику важно понимать её смысл.

— Code Coverage — покрытие кода

Показывает, какая часть кода была выполнена тестами. ISTQB приводит code coverage как пример coverage metric. ()

→ Как считается

Зависит от типа покрытия:

- statement coverage (покрытие инструкций),
- branch coverage (покрытие ветвей),
- decision coverage и др.

100% code coverage не равно 100% quality. Это только показатель того, что код был затронут тестами.

ISTQB указывает, что traceability test results ↔ risks помогает оценить уровень residual risk, а risk metrics входят в отчётность. ()

Это риск, который остаётся после выполненного тестирования.

Значит, residual risk по оплате всё ещё есть.

Это одна из самых “взрослых” метрик. Она помогает честно сказать бизнесу: “да, релиз возможен, но вот какие риски мы не закрыли”.

Тестирование ПО

[illegible]

◆ **Матрица трассировки требований (Requirements Traceability Matrix / RTM)** – Это таблица связей, которая показывает, как один элемент проекта связан с другим. Trace (англ. «след, путь») → Traceability («возможность проследить путь»)

→ Самый частый пример — требования и тест-кейсы. ISTQB прямо определяет traceability matrix именно так и подчёркивает, что она позволяет прослеживать связи в обе стороны, измерять покрытие и оценивать влияние изменений. ()

◆ Traceability — трассируемость

Это возможность установить связь между связанными артефактами: требованиями, test conditions, test cases, test results, defects и рисками. В Foundation Level ISTQB отдельно говорит о необходимости поддерживать traceability между test basis, testware, test results и defects на всём тестовом процессе.

#матрица трассировки #traceability matrix #RTM

11:55

Матрица трассировки требований (Requirements Traceability Matrix / RTM/ Матрица покрытия требований

Чаще всего связывают:

Требования ↔ Тест-кейсы

Требования ↔ Баги

Требования ↔ Результаты тестов

User Story ↔ Acceptance Criteria ↔ Тесты

RTM помогает понять:

☞ каждое ли требование проверено

☞ есть ли тесты на важный функционал

☞ какие баги относятся к какому требованию

☞ что затронет изменение требования

◆ Виды трассировки

— *Forward Traceability (прямая)* –

Требование → тесты

Вопрос: покрыто ли требование?

Движение идёт **от требования к тестам**: для каждого требования проверяют, есть ли тест-кейсы, которые его покрывают. Используется, чтобы убедиться, что ни одно требование не забыто и весь нужный функционал будет протестирован.

— *Backward Traceability (обратная)*

Тест → требование

Вопрос: зачем существует этот тест?

Движение идёт **от теста к требованию**: для каждого тест-кейса ищут, на каком требовании он основан. Используется, чтобы убрать лишние, дублирующие или устаревшие тесты, которые больше не несут пользы.

— *Bidirectional Traceability (двусторонняя)*

Можно идти в обе стороны.

Это лучший вариант.

Позволяет двигаться **в обе стороны одновременно**: от требования к тестам и от тестов к требованиям. Это лучший вариант для реальных проектов, потому что он одновременно контролирует полноту покрытия и актуальность тестовой документации.

◆ Как пользоваться в работе

Ситуация: изменили правило пароля.

Было: минимум 8 символов

Стало: минимум 12 + спецсимвол

◆ Ты открываешь RTM и сразу видишь:

какие тесты переписать

какие автотесты обновить

что заново прогнать

какие баги проверить повторно

◆ Где используется чаще всего

Банки

Медицинские системы

Enterprise проекты

Гос системы

Большие продукты с множеством требований

★ Что ещё важно знать тестировщику в 2026:

— Во многих Agile командах RTM встроена в Jira / Xray / TestRail.

— Excel используют реже, но принцип трассировки обязателен.

— На собеседовании могут спросить:

“Как проверить coverage требований?” — ответ: через RTM.

— Чем сложнее проект — тем важнее трассируемость.

— Хороший QA думает не тестами, а связями: требование → риск → тест → результат.

☞ Итог за 10 секунд:

— Суть: RTM связывает требования с тестами и багами

— Когда использовать: если есть требования и нужен контроль покрытия

— Частая ошибка: создать таблицу и забыть обновлять

— Лучшая практика: двусторонняя трассировка

— Исключение: маленький MVP может жить без формальной RTM

— Предостережение: без RTM легко пропустить важный функционал

[#матрица](#) трассировки [#traceability](#) matrix [#RTM](#)

12:29

Тестирование ПО

◆ Какие метрики любят на собеседовании

1 Pass / Fail Rate

2 Coverage

3 Defect Leakage

4 Severity bugs

5 Reopen Rate

6 Velocity QA команды

7 Automation %

✗ Частые ошибки с метриками

— Считать много цифр без пользы

— Использовать метрики для давления на людей

— Оценивать QA только по количеству багов

— Игнорировать контекст проекта

★ Что ещё важно знать тестировщику в 2026:

- Метрики нужны для решений, а не ради таблиц.
- Лучше 5 полезных метрик, чем 30 бесполезных.
- Сейчас важны product metrics + QA metrics вместе.
- Dashboards в Jira / Power BI / Grafana стали нормой.
- AI помогает анализировать тренды дефектов.

∞ Матрица трассировки (Traceability Matrix / RTM)

RTM – Документ, который связывает требования ↔ тест-кейсы ↔ результаты тестирования ↔ баги.

Нужен, чтобы убедиться:

- ☞ каждое требование проверено
- ☞ ничего не забыто

Простыми словами

Есть требование:

“Пользователь может войти по email”.

RTM показывает:

Требование → какие тесты его проверяют → прошли ли они → есть ли баги.

♦ Зачем нужна RTM

- 1 Проверка покрытия требований
- 2 Контроль изменений
- 3 Быстрый impact analysis
- 4 Удобство аудита
- 5 Понимание готовности релиза

♦ Что входит в RTM

Req ID	Требование	Test Case ID	Статус	Bug ID
R-01	Логин email	TC-11	Passed	
R-02	Сброс пароля	TC-15	Failed	BUG-24

♦ Пример из работы

Изменили требование оплаты в рассрочку.

По RTM быстро находят:

- какие тест-кейсы менять
- какие автотесты упадут
- какие модули затронуты

✗ Частые ошибки RTM

- Заполняют один раз и забывают
- Нет ID требований
- Не обновляют после изменений
- Слишком сложная таблица

☞ Итог за 10 секунд:

- Суть: метрики = цифры качества, RTM = связь требований и тестов
- Когда использовать: на всех проектах, особенно средних и больших
- Частая ошибка: считать цифры без выводов

- Лучшая практика: coverage + risks + trends
- Исключение: маленький MVP может жить без формальной RTM

12:57

13:08

✦ Чем RTM отличается от отчёта о дефекте (Bug Report)

◆ RTM (Requirements Traceability Matrix)

Это **таблица связей**, которая показывает, как требования связаны с тест-кейсами, результатами тестирования и багами. Она нужна для контроля покрытия требований, понимания прогресса тестирования и анализа изменений в проекте.

◆ Bug Report (отчёт о дефекте)

Это **отдельный документ / запись в системе**, в котором подробно описана конкретная ошибка в продукте. Он нужен, чтобы разработчик понял проблему, воспроизвёл её и исправил.

◆ Главное отличие по цели

RTM отвечает на вопрос:

☞ Всё ли важное протестировано и что с этим связано?

Bug Report отвечает на вопрос:

☞ Какая именно ошибка найдена и как её исправить?

◆ Главное отличие по содержанию

RTM содержит:

- ✓ ID требований
- ✓ ID тест-кейсов
- ✓ статусы тестов
- ✓ связанные баги

Bug Report содержит:

- ✓ название бага
- ✓ шаги воспроизведения
- ✓ фактический результат
- ✓ ожидаемый результат
- ✓ severity / priority
- ✓ окружение
- ✓ вложения (скриншоты, видео, логи)

◆ Пример

RTM:

R-15 → TC-21 → Failed → BUG-455

Это показывает, что требование №15 проверялось тестом №21 и найден баг.

Bug Report BUG-455 уже подробно объясняет:

- как открыть страницу
- что нажать
- что сломалось
- что ожидалось

◆ Простыми словами

RTM = карта проекта

Bug Report = карточка конкретной проблемы

◆ Где используются вместе

Ты запускаешь тест-кейс, находишь ошибку, создаёшь Bug Report, а в RTM фиксируется связь:

Требование → Тест → Баг

✦ Что ещё важно знать тестировщику в 2026:

- В Jira / Azure DevOps связь RTM и багов часто строится автоматически через links.
- На собеседовании любят вопрос: “Чем баг отличается от тест-кейса / RTM?”
- RTM показывает системную картину, баг-репорт — детальную проблему.
- Хороший QA умеет работать и на уровне деталей, и на уровне системы.

🕒 Итог за 10 секунд:

- Суть: RTM управляет покрытием, Bug Report описывает ошибку
- Когда использовать: RTM — на протяжении проекта, Bug Report — при нахождении бага
- Частая ошибка: путать таблицу связей с описанием ошибки
- Лучшая практика: связывать баги с требованиями через RTM
- Исключение: маленький проект может жить без RTM
- Предостережение: без качественного Bug Report баг могут “не воспроизвести”

#матрица трассировки #traceability matrix #RTM #bug report

13:57

Тестирование ПО



🌐 Тестирование веб-приложений (Web Application Testing)

◆ Веб-приложение (Web Application)

Это программа, которая работает *через браузер и использует интернет*. Пользователь открывает сайт, взаимодействует с интерфейсом, а данные обрабатываются на сервере.

Примеры:

- ✓ интернет-магазин
- ✓ онлайн-банк
- ✓ соцсеть
- ✓ CRM система
- ✓ личный кабинет

◆ Простыми словами

Обычный сайт показывает информацию.

Веб-приложение позволяет выполнять действия: зарегистрироваться, оформить заказ, загрузить файл

◆ Тестирование веб-приложений

Это проверка того, что веб-приложение работает корректно:

- ✓ функции работают
- ✓ данные сохраняются
- ✓ страницы открываются
- ✓ ошибки обрабатываются
- ✓ приложение удобно пользователю
- ✓ безопасно
- ✓ быстро работает

◆ Что обязан знать тестировщик

- 1 Как работает браузер
- 2 Что такое клиент и сервер
- 3 HTTP / HTTPS
- 4 Cookies / Session
- 5 DevTools
- 6 API запросы
- 7 HTML / CSS / JS базово
- 8 SQL базово
- 9 Авторизация
- 10 Кэш

14:03

★ 6 главных направлений тестирования веб-приложений (2026):

1 Функциональное тестирование Functional Testing

- Что: работает ли логика приложения по требованиям
- Как: чек-листы, тест-кейсы, исследовательское тестирование
- Инструменты: Jira, TestRail, Postman (для API)

2 UI/UX и кросс-браузерное тестирование UI and Cross-Browser Testing

- Что: корректность отображения, удобство, адаптивность
- Как: проверка на разных разрешениях (1920×1080, 360×800), браузерах, ОС
- Инструменты: BrowserStack, LambdaTest, DevTools (режим эмуляции)

3 Тестирование производительности Performance Testing

- Что: скорость загрузки, стабильность под нагрузкой
- Как: нагрузочные тесты (100/1000/10000 пользователей), замер LCP/FID/CLS
- Инструменты: k6, JMeter, Lighthouse, WebPageTest

4 Тестирование безопасности Security Testing

- Что: защита данных, уязвимости, аутентификация
- Как: проверка на OWASP Top 10 (XSS, CSRF, инъекции, небезопасные зависимости)
- Инструменты: OWASP ZAP, Burp Suite Community, Snyk

5 Тестирование доступности (Accessibility)

- Что: могут ли люди с ограничениями пользоваться сайтом
- Как: проверка по WCAG 2.2 AA, тестирование скринридерами, навигация с клавиатуры
- Инструменты: axe DevTools, WAVE, Lighthouse Accessibility

6 API-тестирование (бэкенд веб-приложения)

- Что: корректность работы серверной части, которую не видит пользователь
- Как: проверка эндпоинтов: статус-коды, структура ответа, валидация данных

- Инструменты: Postman, Insomnia, Playwright API

✂ Особенности веб-тестирования в 2026:

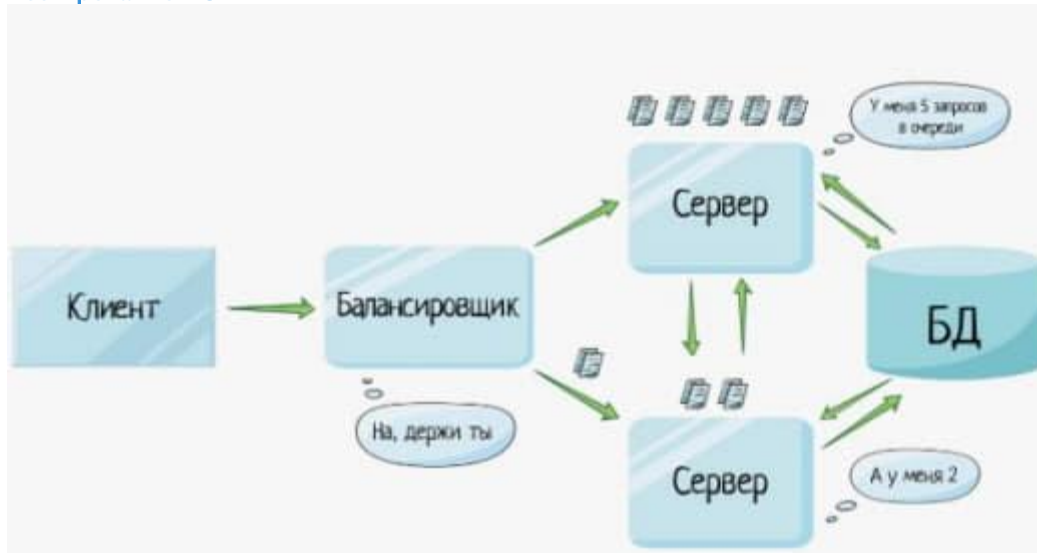
- SPA (Single Page Application): тестировать динамику без перезагрузки страницы (React, Vue, Angular)
- PWA (Progressive Web Apps): проверка офлайн-режима, push-уведомлений, установки на устройство
- Скорость как фича: пользователи уходят, если загрузка > 3 сек — тестируй производительность с первого спринта
- Мобайл-фёрст: 60–80% трафика с телефонов — тестируй на реальных девайсах, не только в эмуляторе
- Безопасность по умолчанию: даже в учебном проекте проверяй базовые уязвимости
- Автоматизация рутины: регресс, кросс-браузерные проверки, API — отдавай автотестам

👉 Итог за 10 секунд:

- Суть: Тестирование веб-приложений — это проверка, что сайт работает правильно, быстро, безопасно и удобно на любом устройстве и в любом браузере
- Когда использовать: всегда, когда создаёшь или обновляешь веб-продукт — от лендинга до сложного сервиса
- Частая ошибка: тестировать только в одном браузере (например, Chrome) и на десктопе — пользователи придут с других устройств и найдут баги
- Лучшая практика: начать с чек-листа критических сценариев (регистрация, покупка, вход), добавить автотесты на регресс, проверять производительность и доступность на ранних этапах
- Исключение: для внутреннего инструмента на 5 человек можно не тестировать кросс-браузерность и доступность — достаточно функционала и базовой безопасности
- Предостережение: никогда не выпускай в продакшн без проверки безопасности (хотя бы базовой) и без тестирования на мобильном устройстве — это главные источники критических инцидентов

15:28

Тестирование ПО



🌐 Клиент-серверная архитектура в тестировании (Client-Server Architecture)

🔹 Клиент-серверная архитектура

Это модель работы приложения, где система разделена на клиент и сервер.

🔹 Клиент (Client)

Часть, с которой работает пользователь. Обычно это:

- ✓ браузер
- ✓ мобильное приложение

✓ desktop программа

Клиент отправляет запросы и показывает результат пользователю.

◆ Сервер (Server)

Это удалённая часть системы, которая:

- ✓ принимает запросы
- ✓ обрабатывает данные
- ✓ работает с базой данных
- ✓ выполняет бизнес-логику
- ✓ отправляет ответы клиенту

#клиент-серверная архитектура #Client #Server

16:39

Тестирование ПО

◆ Зачем тестировщику знать клиент-серверную архитектуру? Баг может быть:

- на клиенте
- на сервере
- в запросе
- в ответе
- в базе данных
- в интеграции между сервисами

Если не понимать архитектуру — сложно найти причину ошибки.

◆ Основные компоненты

1 Client (Frontend)

То, что видит пользователь.

- кнопки
- формы
- страницы
- интерфейс

Технологии: HTML, CSS, JavaScript, React, Vue.

2 Backend / Server

Серверная логика.

- авторизация
- расчёт скидки
- создание заказа
- отправка email

Технологии: Java, Python, Node.js, .NET, PHP и др.

3 Database (База данных)

Хранит информацию.

- пользователи
- товары
- заказы
- платежи

4 API

Мост между клиентом и сервером.

Чаще всего:

- REST API
- GraphQL
- WebSocket

◆ Как идёт запрос

Пример: открыть профиль пользователя

- 1 Клиент отправляет GET запрос
- 2 Сервер получает запрос
- 3 Сервер идёт в базу данных
- 4 Получает данные
- 5 Возвращает JSON ответ
- 6 Клиент рисует экран профиля

◆ Что тестирует QA

На клиенте

- кнопки работают
- поля валидируются
- UI корректный
- данные отображаются правильно

На сервере

- бизнес-логика верная
- правильные статусы ответов
- ошибки обрабатываются
- данные сохраняются

Между клиентом и сервером

- правильный запрос
- корректный response body
- токены работают
- нет лишних данных

◆ Частые баги

✗ Клиентские баги

- — кнопка не нажимается
- — неверный текст
- — поле невалидно работает
- — мобильная верстка ломается

✗ Серверные баги

- — 500 Internal Server Error
- — неверный расчёт цены
- — не создаётся заказ
- — не сохраняются данные

✗ Интеграционные баги

- — UI ждёт одно поле, API отдаёт другое
- — сервер вернул null
- — токен просрочен
- — frontend не обрабатывает ошибку 401

◆ Что должен уметь тестировщик

Базово понимать HTTP

- — GET — получить данные
- — POST — создать
- — PUT/PATCH — изменить
- — DELETE — удалить

Знать коды ответа

- — 200 OK
- — 201 Created
- — 400 Bad Request
- — 401 Unauthorized
- — 403 Forbidden
- — 404 Not Found
- — 500 Server Error

◆ Виды архитектуры

1 Two-tier (2 уровня)

Клиент ↔ Сервер

Простая схема.

2 Three-tier (3 уровня)

Клиент ↔ Backend (Сервер) ↔ Database

Самый популярный вариант.

3 Microservices

Много маленьких сервисов.

Например:

- ◆ auth service
- ◆ payment service
- ◆ order service

#клиент-серверная архитектура #Client #Server

18:37

Тестирование ПО

🖥️ Толстый и тонкий клиент в тестировании (Thick Client / Thin Client)

◆ Клиент (Client)

Это часть системы, с которой работает пользователь: интерфейс приложения, браузер, мобильное приложение или desktop программа. Клиент получает действия пользователя, отправляет запросы и показывает результат.

◆ Тонкий клиент (Thin Client) – Это клиент, у которого **минимум логики**, а основная обработка происходит на сервере.

Примеры:

- обычный сайт в браузере
- веб-приложение, где почти всё считает backend
- удалённый рабочий стол
- терминалы в компаниях

Простыми словами

Клиент только показывает интерфейс, а “думает” сервер.

→ **Пример**

Ты нажал кнопку “Рассчитать стоимость”.

Браузер отправил запрос → сервер всё посчитал → вернул результат.

◆ **Толстый клиент (Thick Client / Fat Client)**

Это клиент, у которого **много логики находится на устройстве пользователя.**

Примеры:

- desktop программа 1С
- онлайн гиры (но там скорее всего смешанный вариант)
- Photoshop
- Excel
- мобильное приложение с офлайн-режимом
- SPA приложение с большим количеством frontend логики

◆ Простыми словами

Приложение само многое умеет делать без сервера или с минимальной зависимостью от него.

→ **Пример**

Калькулятор скидки считает цену прямо в приложении без запроса на сервер.

◆ **Главное отличие**

- Thin Client – Логика на сервере.
- Thick Client – Логика на клиенте.

◆ **Что важно тестировщику**

Если Thin Client:

Проверять:

- — запросы к серверу
- — ответы API
- — сетевые ошибки
- — скорость загрузки
- — работу при слабом интернете
- — серверную валидацию

Если Thick Client:

Проверять:

- — локальную логику
- — сохранение данных на устройстве
- — офлайн режим
- — обновления приложения
- — совместимость с ОС
- — использование памяти / CPU

◆ **Примеры багов**

Тонкий клиент

- кнопка нажата, но запрос не ушёл
- сервер вернул 500
- страница долго грузится
- при потере сети всё ломается

Толстый клиент

- данные не сохраняются локально
- приложение зависает
- после обновления сломался модуль
- офлайн режим работает неверно

♦ Веб-приложения сегодня

Многие современные SPA (React, Vue, Angular) — это **смешанный вариант**.

Почему:

- ♦ интерфейс работает как толстый клиент
 - ♦ данные берутся с сервера как у тонкого клиента
- То есть сейчас часто используется гибридная модель.

♦ Что важно знать тестировщику в 2026

- Термины “толстый/тонкий клиент” старые, но до сих пор используются на собеседованиях.
- Сейчас часто спрашивают не слова, а архитектурное понимание: где бизнес-логика? где валидация? где хранение данных?
- Electron, desktop apps, mobile apps часто ближе к thick client.
- Простые web admin panels часто ближе к thin client.
- Современный frontend всё чаще становится “толще”.

♦ Частые ошибки новичков

- Думать, что толстый клиент = просто большое приложение
- Думать, что тонкий клиент = любой сайт
- Не понимать, где реально выполняется логика
- Не тестировать офлайн сценарии thick client систем

☞ Итог за 10 секунд:

- Суть: тонкий клиент почти ничего не считает сам, толстый клиент содержит много логики
- Когда использовать: для понимания архитектуры и где искать баг
- Частая ошибка: смотреть только на UI, а не на место выполнения логики
- Лучшая практика: определить где находится логика и тестировать именно этот слой
- Исключение: современные SPA часто гибридные
- Предостережение: одинаковый интерфейс может скрывать совершенно разную архитектуру

[#клиент](#)-серверная архитектура [#толстый](#) клиент [#тонкий](#) клиент

18:48

📦 **Дополнительное хранение данных на клиенте** — Cookies, LocalStorage, SessionStorage, Cache

♦ Что значит “на клиенте”

Это данные, которые сохраняются **на устройстве пользователя**: в браузере или приложении. Они помогают сайту помнить пользователя, ускорять работу и сохранять временную информацию.

△ Это не основная база данных продукта, а локальное хранение рядом с пользователем.

♦ **1. Cookies (куки)** – Это маленькие данные, которые сайт сохраняет в браузере и отправляет на сервер при следующих запросах.

Чаще всего используются для:

- сессии пользователя
- авторизации
- remember me

- язык сайта
- настройки региона
- аналитики

Простыми словами

Ты вошёл в личный кабинет.
Сайт сохранил cookie сессии.

При следующем открытии страницы сервер понимает:

☞ это тот же пользователь.

Частый баг

✗ Пользователь вышел, но cookie остался и сессия не закрылась.

♦ **2. LocalStorage** – Хранилище данных в браузере **без срока действия**, пока пользователь сам не очистит браузер или код не удалит данные.

Используют для:

- тема сайта (dark/light)
- настройки интерфейса
- черновики
- корзина
- токены (иногда)
- недавно просмотренное

Простыми словами

Закрыл браузер → открыл завтра → данные всё ещё есть.

Частый баг

✗ Пользователь вышел, а старый токен остался в localStorage.

♦ **3. SessionStorage** – Хранилище данных **только на время текущей вкладки браузера**.

Когда вкладка закрывается — данные исчезают.

Используют для:

- данные многошаговой формы
- временные фильтры
- временные таблицы
- промежуточное состояние страницы

Простыми словами

Открыл вкладку → работаешь.
Закрыл вкладку → всё удалилось.

Частый баг

✗ Данные формы теряются после случайного refresh, хотя должны были жить в сессии.

♦ **4. Cache (кэш)** – Это временное сохранение файлов и данных для ускорения загрузки.

Что может кэшироваться:

- картинки

- CSS
- JavaScript
- API ответы
- страницы

Простыми словами

Если сайт уже открывали, часть файлов не качается заново, а берётся из памяти/диска.

Частый баг

✗ После релиза пользователь видит старый интерфейс из-за кэша.

♦ Где смотреть тестировщику

- Chrome DevTools → Application tab

Там можно открыть:

- Cookies
- LocalStorage
- SessionStorage
- Cache Storage

♦ Что важно знать тестировщику в 2026

Cookies с auth часто безопаснее при httpOnly.

LocalStorage удобен, но чувствительные токены там рискованнее хранить.

После logout важно чистить данные клиента.

Кэш часто вызывает “не воспроизводится у меня”.

Проверка storage — обязательный навык web QA.

👉 Итог за 10 секунд:

- Cookies — для сессий и авторизации
- LocalStorage — долгие локальные данные
- SessionStorage — временные данные вкладки
- Cache — ускорение загрузки
- Лучшая практика: QA всегда проверяет что хранится у клиента
- Предостережение: локальное хранение часто источник багов и проблем безопасности

[#хранение](#) [данных](#) [#Cookies](#) [#LocalStorage](#) [#SessionStorage](#) [#Cache](#)

24 April 2026

12:37

Тестирование ПО

🌐 Веб-страница, веб-сайт, веб-приложение, веб-сервис, веб-сервер — разница простыми словами
! часто путают эти термины. На собеседовании и в работе важно говорить правильно – это разные сущности.

♦ Веб-страница (Web Page) – Это одна отдельная страница, которую открывает браузер по URL-адресу.

Примеры:

- главная страница сайта
- страница товара
- страница контактов
- статья в блоге

Пример URL:

<https://site.com/about>

◆ Простыми словами

Веб-страница = один лист внутри сайта.

◆ Что проверяет QA

открывается ли страница

корректный контент

верстка

ссылки

ошибки 404 / 500

◆ **Веб-сайт (Website)** – Это *набор связанных веб-страниц*, размещённых под одним доменом.

Примеры:

интернет-магазин

новостной портал

сайт компании

блог

Например:

site.com

└ Главная

└ О нас

└ Каталог

└ Контакты

◆ Простыми словами

Сайт = дом + страницы внутри.

◆ Что проверяет QA

навигация между страницами

меню

SEO основы

корректные ссылки

единый дизайн

◆ **Веб-приложение (Web Application)** – Это программа, работающая в браузере, где пользователь выполняет действия и используется бизнес-логика.

Примеры:

онлайн-банк

Gmail

CRM система

Trello

маркетплейс с личным кабинетом

◆ Простыми словами

Если пользователь не просто читает сайт, а работает в системе — это веб-приложение.

◆ Что проверяет QA

регистрация
логин
формы
роли доступа
сохранение данных
API
безопасность

◆ **Веб-сервер (Web Server)** – Это сервер (железо + ПО), который принимает HTTP/HTTPS запросы и отдаёт страницы, файлы или ответы API.

Популярные web servers:

Nginx
Apache
IIS

◆ Простыми словами

Браузер просит страницу → веб-сервер отдаёт страницу.

◆ Что проверяет QA косвенно

— сайт доступен
— корректные статусы 200/404/500
— SSL сертификат
— скорость ответа
— ошибки деплоя

◆ **5. Веб-сервис (Web Service)** – Это сервис для обмена данными между системами через интерфейс (API).

Примеры:

— сервис оплаты
— сервис доставки
— сервис авторизации
— погодный API
— банковский API

◆ Простыми словами

Одно приложение “разговаривает” с другим через веб-сервис.

◆ Что проверяет QA

✓ API запросы
✓ JSON/XML ответы
✓ авторизацию
✓ ошибки 400/401/500
✓ интеграции

◆ Сравнение

Термин-----	Что это -----
-----	-----
Веб-страница-----	одна страница-----
Веб-сайт-----	набор страниц-----
Веб-приложение	программа в браузере-----
Веб-сервер-----	сервер, который отдаёт данные -
Веб-сервис-----	API для общения систем-----

♦ Пример интернет-магазина

Веб-сайт: весь shop.com

Веб-страница: shop.com/product/15

Веб-приложение: личный кабинет, корзина, оформление заказа

Веб-сервер: Nginx + backend сервер

Веб-сервис: API оплаты банка

♦ Что важно знать тестировщику в 2026

- Многие современные сайты одновременно и website, и web application.
- Frontend + Backend + API часто разделены микросервисами.
- В вакансиях “web QA” почти всегда значит тестирование web applications.
- Нужно понимать network, browser, server responses.
- Web service сегодня часто = REST API / GraphQL API.

♦ Частые ошибки новичков

- Называть любой сайт приложением
- Путать сервер и сервис
- Думать, что страница = весь сайт
- Не различать UI и API часть системы

☞ Итог за 10 секунд:

- Веб-страница = один экран по URL
- Веб-сайт = набор страниц
- Веб-приложение = система для действий пользователя
- Веб-сервер = отдаёт данные браузеру
- Веб-сервис = API для обмена между системами
- Предостережение: в реальных проектах эти сущности часто работают вместе

[#веб-страница](#) [#веб-сайт](#) [#веб-приложение](#) [#веб-сервис](#) [#веб-сервер](#) [#web page](#) [#website](#) [#web server](#) [#web application](#) [#web service](#)

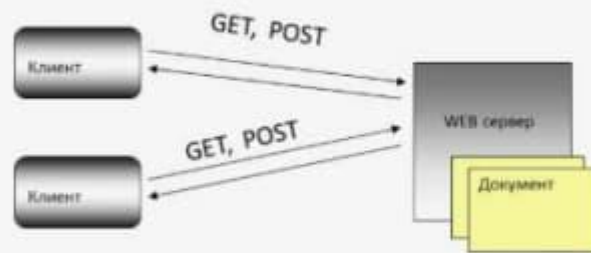
17:02

Тестирование ПО

Протокол HTTP



Тим Бернерс-Ли, изобретатель
URI, URL, HTTP, HTML и Web



🌐 HTTP-протокол

◆ HTTP (HyperText Transfer Protocol)

Это протокол (набор правил), по которому клиент (браузер) и сервер обмениваются данными.

#http протокол

17:12

◆ HTTPS

Это HTTP + шифрование (SSL/TLS). Используется почти везде.

SSL (Secure Sockets Layer) и TLS (Transport Layer Security) — криптографические протоколы, которые обеспечивают защищённую передачу данных между клиентом и сервером через интернет. Сегодня используется TLS, но исторически прижилось название «SSL-сертификат».

TLS — это преемник SSL, более безопасная и современная версия.

Практический вывод

Когда вы видите `https://` и замок в браузере — почти наверняка используется TLS 1.2 или 1.3. Настоящий SSL уже не встречается в безопасных соединениях.

◆ Как работает HTTP

1. Клиент отправляет запрос (request)
 2. Сервер обрабатывает
 3. Сервер отправляет ответ (response)
- ☞ Это называется модель **request-response**

◆ Структура общения: Запрос и Ответ

Любое общение в HTTP состоит из двух частей.

1. HTTP Request (Запрос от клиента)

Состоит из 4 частей:

- 1. Метод (Method): Что мы хотим сделать? (Получить, отправить, удалить).
- 2. URL (Адрес): Куда стучимся? (<https://api.site.com/users>).
- 3. Заголовки (Headers): Служебная информация (какой у нас браузер, какой тип данных отправляем, токен авторизации).
- 4. Тело (Body): Сами данные (например, логин и пароль в формате JSON). Есть не во всех запросах.

◆ Пример запроса

POST [/login](#) HTTP/1.1

Host: [site.com](#)

```
Content-Type: application/json
{
  "email": "test@mail.com",
  "password": "123456"
}
```

📄 HTTP Response (Ответ от сервера)

Состоит из 3 частей:

- 1.Статус-код (Status Code): Цифра, говорящая о результате (200 — OK, 404 — не найдено).
- 2.Заголовки (Headers): Информация об ответе (тип данных, дата, размер).
- 3.Тело (Body): То, что сервер вернул (HTML-страница, JSON с данными, картинка).

💎 Пример ответа

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "id": 1,
  "name": "User"
}
```

17:17

Метод	Что делает	Пример использования	Идемпотентность* 
GET	Получает данные	Открыть список товаров	Да (безопасно повторять)
POST	Создает новое	Зарегистрировать пользователя	Нет (создаст дубликат)
PUT	Обновляет целиком	Изменить профиль (перезаписать)	Да
PATCH	Обновляет частично	Поменять только аватарку	Да
DELETE	Удаляет	Удалить комментарий	Да

**Идемпотентность: если отправить запрос 10 раз, результат будет таким же, как если бы отправили 1 раз (кроме POST).*

💎 HTTP методы

— **GET** – Получить данные

Пример: открыть страницу

! не должен менять данные

— **POST** – Создать данные

Пример: регистрация, создание заказа

— **PUT** – Полностью обновить ресурс

— **PATCH** – Частично обновить

— **DELETE** – Удалить данные

— **HEAD** – Как GET, но без body (редко используют)

— **OPTIONS** – Проверка доступных методов (часто в CORS)

★ Что важно тестировщику

— GET не должен изменять данные

— POST/PUT/PATCH — изменяют

— DELETE — удаляет

#http методы #get #post #put #patch #delete

18:01

Тестирование ПО

◆ HTTP статус-коды

✓ 1xx — информационные

Редко используются

✓ 2xx — Success (успех)

200 OK — всё хорошо, Стандартный успех. Данные пришли.

201 Created — Успешно создано (часто после POST). QA должен проверить, что вернулся именно 201, а не 200, если так требуется по документации.

204 No Content — Успешно, но тела ответа нет (часто после DELETE).

🔄 3xx —Redirection редиректы

300 Multiple Choices – Сервер возвращает несколько вариантов ответа на один запрос. Клиент должен самостоятельно выбрать один из них. Сейчас встречается крайне редко

301 Moved Permanently: Адрес сменился навсегда.

302 Found / 307 Temporary Redirect: Временный редирект.

⚠ Для QA важно: Проверять, куда именно перекидывает пользователя и не теряются ли при этом данные (сессия, корзина).

304 Not Modified – Кэширование Не является редиректом. Означает, что ресурс не изменился с момента последнего запроса. Тело ответа не отправляется, клиент использует локальную кэшированную копию.

⚠ Важно 304: Хотя формально он находится в диапазоне 3xx, он не перенаправляет клиента на другой URL. Это ответ для механизма условного кэширования (If-Modified-Since, ETag).

✖ 4xx — Client Error ошибка клиента

400 Bad Request: Кривой запрос (ошибка в синтаксисе, не хватает поля).

401 Unauthorized: Нет токена или он протух. Нужно залогиниться.

403 Forbidden: Ты залогинен, но у тебя нет прав (ты юзер, а лезешь в админку).

404 Not Found: Такого адреса не существует.

405 Method Not Allowed: Этот метод тут запрещен (пытаешься удалить через GET).

429 Too Many Requests: Ты спамишь запросами.

!Q Что спрашивают на собеседовании:

«В чем разница между 401 и 403?»

Ответ: 401 — «Я не знаю, кто ты» (нет логина). 403 — «Я знаю, кто ты, но тебе сюда нельзя» (нет прав).

🚨 5xx — Server Error ошибка сервера

500 Internal Server Error: Общая ошибка на сервере («упало всё»).

502 Bad Gateway: Сервер-посредник получил плохой ответ от другого сервера.

503 Service Unavailable: Сервер перегружен или на техработках.

◆ Headers (заголовки) – Это дополнительная информация о запросе/ответе

Не читай все подряд. Смотри на эти:

→ **В Запросе (Request Headers):**

Authorization: Токен доступа (Bearer token). Если его нет или он неверный — будет 401.

Content-Type: Какой формат данных ты шлешь (обычно application/json).

Cookie: Данные сессии.

→ **В Ответе (Response Headers):**

Content-Type: В каком формате пришел ответ (должен совпадать с тем, что ждет фронтенд).

Set-Cookie: Сервер просит сохранить куки.

Cache-Control: Как долго браузеру хранить эту страницу в кэше.

♦ **Body (тело запроса/ответа) и форматы данных**

Содержит данные

Чаще всего:

JSON (самый популярный)

XML

Form-data

🔍 Что проверять:

1. Типы данных: age должно быть числом, а не строкой "22".
2. Обязательные поля: Если убрать email, сервер должен вернуть понятную ошибку 400, а не 500.
3. Граничные значения: Что будет, если отправить пустую строку "" или null?

✕ Частые баги

GET изменяет данные

неправильный статус (200 вместо 400)

сервер возвращает 500

отсутствует авторизация

неверный JSON

CORS ошибка

timeout

♦ **Что важно знать тестировщику в 2026**

- HTTP — база API тестирования
- Без понимания network ты не найдёшь половину багов
- DevTools — основной инструмент
- REST API — стандарт
- JSON — основной формат
- Нужно понимать auth, headers, tokens

🏆 **Итог за 10 секунд:**

- HTTP — это правила общения клиента и сервера
- Request → Response — основа
- Методы показывают действие
- Статусы показывают результат
- Лучшая практика: всегда проверять network
- Предостережение: UI может работать, а API — нет

18:21

Тестирование ПО

Модель				
Уровень (layer)	Тип данных (PDU ⁽¹⁾)	Функции	Примеры	Оборудование
Host layers	7. Прикладной (application)	Доступ к сетевым службам	HTTP, FTP, POP3, SMTP, WebSocket	Хосты (клиенты сети), Межсетевой экран
	6. Представление (presentation)	Представление и шифрование данных	ASCII, EBCDIC, JPEG, MIDI	
	5. Сетевой (session)	Управление сеансом связи	RPC, PAP, L2TP, gRPC	
	4. Транспортный (transport)	Принимает данные между конечными пунктами и надёжность	TCP, UDP, SCTP, Порты	
Media layers	3. Сетевой (network)	Определение маршрута и логическая адресация	IPv4, IPv6, IPsec, AppleTalk, ICMP	Маршрутизатор, Сетевой шлюз, Межсетевой экран
	2. Канальный (data link)	Биты (bit)/ Кадры (frame)	PPP, IEEE 802.22, Ethernet, DSL, ARP, сетевая карта	Сетевой мост, Коммутатор, точка доступа
	1. Физический (physical)	Биты (bit)	USB, RJ («витая пара»), коаксиальный, оптоволоконный, радиоканал	Концентратор, Повторитель (сетевое оборудование)

Модель OSI

♦ **Сетевая модель OSI (The Open Systems Interconnection model)** — сетевая модель стека (магазина) сетевых протоколов OSI/ISO. Посредством данной модели различные сетевые устройства могут взаимодействовать друг с другом. Модель определяет различные уровни взаимодействия систем. Каждый уровень выполняет определённые функции при таком взаимодействии.

Подробнее [здесь](#)

18:54

Тестирование ПО

Модель OSI

♦ **Модель OSI (Open Systems Interconnection)** — это теоретическая «пирамида» из 7 уровней, которая объясняет, как данные проходят от твоего браузера до сервера и обратно.

→ TCP / UDP — два главных способа доставки данных по сети:

— TCP (Transmission Control Protocol) — надёжный протокол передачи данных с установлением соединения, как заказное письмо с уведомлением о вручении.

— UDP (User Datagram Protocol) — быстрый протокол без гарантии доставки и без установки соединения, как бросок мяча другу — если не поймал, значит не судьба.

💡 **Лайфхак для QA:**

Если ты видишь, что в логах есть потерянные пакеты, но приложение продолжает работать — скорее всего, используется UDP.

Если же приложение зависает или выдаёт ошибку при потере пакета — вероятно, TCP, и это баг (должен быть retry).

♦ Пример из работы тестировщика

Ты тестируешь видеозвонок в приложении (например, Zoom или Telegram):

Если используется TCP → звонок может лагать, но картинка не рассыпется.

Если используется UDP → картинка может «пикселировать», но звук идёт без задержек.

🔧 **Задача QA:** Понять, какой протокол используется, и проверить, соответствует ли поведение

приложения ожиданиям (например, в играх важна скорость — там UDP; в банковских транзакциях — точность — там TCP).

◆ Разбираем модель OSI по уровням

□ Уровни Host Layers (где живёт твой код и браузер)

7 Прикладной (Application) — самый важный для QA

Что делает. Предоставляет интерфейс пользователю (браузер, мессенджер, API-клиент).

Примеры протоколов: HTTP, HTTPS, FTP, SMTP, WebSocket, GraphQL.

Что проверяет QA:

Корректность запросов/ответов (статусы, заголовки, тело).

Работа с куками, токенами, кэшем.

Обработка ошибок (4xx, 5xx).

Поддержка разных версий протоколов (HTTP/1.1, HTTP/2, HTTP/3).

6 Представления (Presentation)

Что делает. Шифрование, сжатие, преобразование форматов (JSON ↔ XML, ASCII ↔ UTF-8).

Примеры: SSL/TLS (шифрование), JPEG, MP3, gzip.

Что проверяет QA:

Данные приходят в нужном формате (например, JSON, а не HTML).

Шифрование работает (HTTPS, сертификаты).

Сжатие не ломает данные (проверить Content-Encoding: gzip).

5 Сеансовый (Session)

Что делает. Управление сеансом связи (начало, пауза, завершение).

Примеры: RPC, NetBIOS, PPTP.

Что проверяет QA:

Сохраняется ли сессия после перезагрузки страницы?

Не теряются ли данные при разрыве соединения?

Работает ли механизм повторного подключения (reconnect)?

4 Транспортный (Transport) — критичен для понимания TCP/UDP

Что делает. Доставляет данные между конечными точками (порт → порт).

Протоколы: TCP (надёжный), UDP (быстрый), SCTP (редко).

Что проверяет QA:

Потеря пакетов? (для UDP — нормально, для TCP — баг).

Задержки? (latency).

Порядок доставки? (TCP гарантирует, UDP — нет).

Повторная отправка? (TCP сам повторяет, UDP — нет).

□ Уровни Media Layers (сетевая инфраструктура — реже трогаем, но полезно знать)

3 Сетевой (Network)

Что делает. Маршрутизация, IP-адресация.

Протоколы: IPv4, IPv6, ICMP (ping), IPSec.

Что проверяет QA:

Доступен ли сервер по IP?

Работает ли ping/traceroute?

Нет ли проблем с DNS (домен не резолвится)?

Блокируется ли трафик фаерволом?

🔗 Канальный (Data Link)

Что делает: Передача кадров внутри одной сети (MAC-адреса).

Протоколы: Ethernet, Wi-Fi (802.11), PPP.

Что проверяет QA:

Работает ли приложение в локальной сети?

Нет ли конфликтов MAC-адресов (редко, но бывает в IoT).

Стабильность Wi-Fi соединения (потери пакетов на этом уровне).

📶 Физический (Physical)

Что делает: Передача битов по проводам/радиоканалу.

Оборудование: Кабели, роутеры, модемы.

Что проверяет QA:

Есть ли физическое подключение? (иногда банально — кабель выдернут).

Работает ли мобильный интернет при плохом сигнале?

Не перегревается ли оборудование? (в embedded/IoT тестировании).

18:54

👉 Итог за 10 секунд:

— Суть: Модель OSI — это 7 уровней, по которым идут данные. QA чаще всего работает на уровнях 5–7 (HTTP, сессии, шифрование).

— Когда использовать: При диагностике сетевых багов, тестировании API, мобильных приложений, IoT.

— Частая ошибка: Путать TCP и UDP, игнорировать заголовки, считать 200 = успех.

— Лучшая практика: Всегда смотри в Network Tab, используй Wireshark для сложных случаев, тестируй при плохой сети.

— Исключение: WebSocket, gRPC, GraphQL — они работают поверх HTTP, но имеют свои особенности.

— Предостережение: В 2026 году активно внедряется HTTP/3 (на базе UDP) — учись тестировать его поддержку!

18:59

? Q Что спрашивают на собеседовании:

«Чем отличается TCP от UDP?» → см. таблицу выше. Главное: надёжность vs скорость.

«На каком уровне модели OSI работает HTTP?» → Уровень 7 (прикладной).

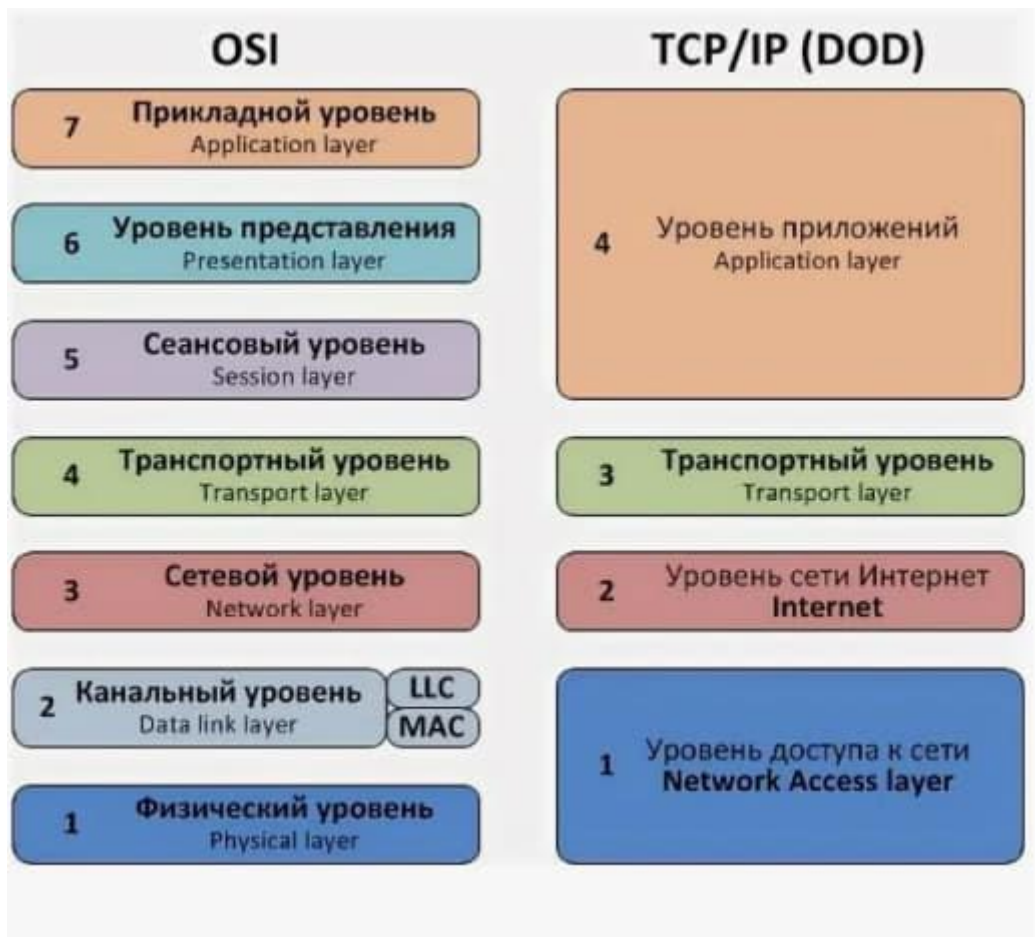
«Почему в играх используют UDP, а не TCP?» → Потому что важна скорость, а не идеальная доставка.

«Как проверить, работает ли HTTPS?» → Посмотреть замок в браузере, проверить сертификат, использовать openssl s_client.

«Что такое QUIC?» → Протокол поверх UDP, лежащий в основе HTTP/3. Уменьшает время установки соединения.

19:18

Тестирование ПО



Сетевые модели OSI и TCP/IP

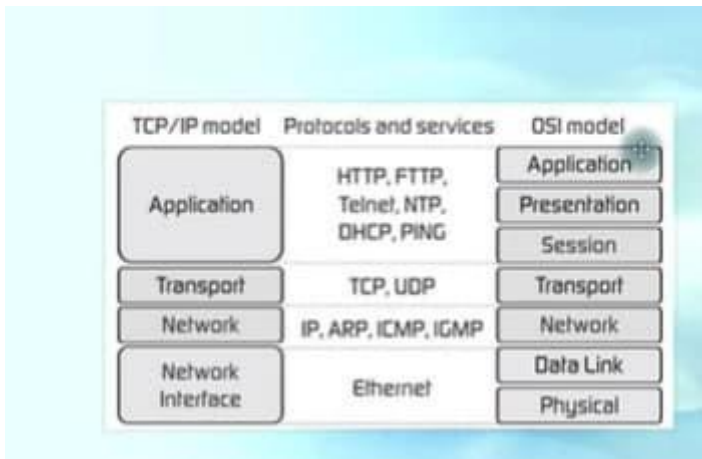
◆ **Сетевая модель** — это «этажерка» правил, которая показывает, как информация проходит от твоего устройства до сервера и обратно.

Простыми словами: Это инструкция для курьера. Чтобы письмо дошло, его нужно написать (верхний уровень), упаковать в конверт (средний), указать адрес и город (нижний), а потом физически донести на машине (самый низ).

◆ **TCP/IP** — практическая модель из 4 уровней (так интернет работает в реальности).

◆ **OSI** — теоретическая модель из 7 уровней (эталон для обучения и точной диагностики).

◆ **HTTP** — протокол самого верхнего уровня. Это язык, на котором браузеры и серверы обмениваются страницами и данными.



19:19

★ Что ещё важно знать тестировщику в 2026:

- Реальные практики: В работе все пользуются TCP/IP. OSI вспоминают только на собеседованиях или когда нужно точно локализовать баг (например, «проблема не в коде, а в DNS-резолвинге»).
- Инструменты: ping (проверка IP/ICMP), traceroute (поиск пути), Wireshark (глубокий разбор пакетов), Chrome DevTools → Network (твоя главная консоль для HTTP).

? Q Что спрашивают на собеседовании:

- «На каком уровне работает HTTP?» → Прикладной (7-й в OSI, 4-й в TCP/IP).
- «Зачем нужны две модели?» → OSI для теории и диагностики, TCP/IP для реальной работы интернета. В реальности работает TCP/IP, но на собеседовании и в документации часто ссылаются на OSI, потому что там подробнее расписаны ошибки.
- «Что такое ICMP?» → Служебный протокол для диагностики (ping, traceroute).

☞ Итог за 10 секунд:

— Суть: TCP/IP (4 уровня) — практика, OSI (7 уровней) — теория. Они просто по-разному группируют одни и те же шаги.

Когда использовать: При диагностике «почему не грузится», настройке тестового окружения, на собеседованиях.

— Частая ошибка: Думать, что HTTP работает на транспортном уровне. Нет, он на самом верху, поверх TCP.

— Лучшая практика: Дебажить снизу вверх: Физика → IP → TCP порт → HTTP запрос.

— Исключение: В современных облаках и микросервисах границы размываются, но принцип «запрос идёт по слоям» остаётся законом.

— Предостережение: Не учи наизусть все протоколы из центра схемы. Запомни только цепочку: Ethernet → IP → TCP → HTTP. Остальное гугли по мере необходимости.

☞ Итог за 10 секунд:

— Суть: TCP/IP (4 уровня) — практика, OSI (7 уровней) — теория. Они просто по-разному группируют одни и те же шаги.

Когда использовать: При диагностике «почему не грузится», настройке тестового окружения, на собеседованиях.

— Частая ошибка: Думать, что HTTP работает на транспортном уровне. Нет, он на самом верху, поверх TCP.

— Лучшая практика: Дебажить снизу вверх: Физика → IP → TCP порт → HTTP запрос.

— Исключение: В современных облаках и микросервисах границы размываются, но принцип «запрос идёт по слоям» остаётся законом.

— Предостережение: Не учи наизусть все протоколы из центра схемы. Запомни только цепочку: Ethernet → IP → TCP → HTTP. Остальное гугли по мере необходимости.

🌐 URL, IP, DNS, кэш и куки: как интернет находит и запоминает всё

💡 Термин(ы)

- IP-адрес — числовой адрес устройства в сети ([192.168.1.1](#) или 2001:0db8::1)
- Домен — человеко-читаемое имя для IP ([example.com](#))
- DNS — сервис, который переводит домен → IP («телефонная книга интернета»)
- URL (Uniform Resource Locator) — полный адрес ресурса: протокол + домен + путь + параметры + якорь
- Кэш — временное хранилище файлов на устройстве для ускорения повторной загрузки
- Куки — маленькие данные от сервера браузеру для запоминания пользователя/сессии

27 April 2026

12:41

Тестирование ПО

∞ URL — адрес страницы в интернете

- 💡 URL (Uniform Resource Locator) — строка, указывающая точное местоположение ресурса в сети.

💡 Пример:

<https://www.example.com:443/path/to/page?search=qa#section>

- https — протокол (как «курьер»)
- [www.example.com](#) — домен (имя сайта)
- :443 — порт (дверь, по умолчанию 443 для HTTPS)
- /path/to/page — путь к файлу
- ?search=qa — параметры (доп. данные)
- [#section](#) — якорь (конкретное место на странице)

∞ URI, URL, URN: В чём разница?

- 💡 URI (Uniform Resource Identifier) — это общий термин для идентификации любого ресурса в интернете.
- 💡 URL (Uniform Resource Locator) — это адрес, где ресурс находится (как адрес дома).
- 💡 URN (Uniform Resource Name) — это имя ресурса (как ISBN у книги или название документа).
- Пример:

<http://www.webdev.com/accueil/index.html>

- Весь адрес целиком → это URI (идентифицирует ресурс)
- <http://www.webdev.com/> → это URL (протокол + домен = где искать)
- [/accueil/index.html](#) → это URN (путь к конкретному файлу = что именно ищем)

🔍 Основная часть:

Что проверяет QA:

- Работает ли ссылка (код ответа 200, а не 404/500)
- Корректно ли кодируются спецсимволы в параметрах (?q=тест → ?q=%D1%82%D0%B5%D1%81%D1%82)
- Не «утекают» ли чувствительные данные в URL (пароли, токены)
- Работают ли редиректы (301/302) при смене адреса

🐞 Где чаще всего баги:

- Опечатки в параметрах → страница грузится, но данные не передаются
- Отсутствие кодировки кириллицы → битая ссылка
- Смешивание HTTP/HTTPS → предупреждения браузера, потеря куки

✓ Как проверять:

- DevTools → Network → смотреть статус-коды и заголовки

Прогнать URL через валидатор (например, <https://validator.w3.org/checklink>)

- Проверить в инкогнито (без кэша и куки)

☛ Итог за 10 секунд:

- URI — это ВСЁ, что идентифицирует ресурс (общий термин)
- URL — это АДРЕС (где находится: протокол + домен)
- URN — это ИМЯ (что именно: путь к файлу)
- Важно: URL и URN — это подмножества URI
- QA-совет: всегда проверять полный URI в запросах и кодировать спецсимвол

#URL

12:45

Типы IP:

Тип	Пример	Зачем нужен	⬇
Публичный	8.8.8.8	Доступ из интернета (сервер, роутер)	
Приватный	192.168.0.1	Внутри локальной сети (дом, офис)	
Статический	Не меняется	Для серверов, чтобы адрес был постоянным	
Динамический	Меняется при переподключении	Для обычных пользователей (дешевле)	

📍 IP-адрес — цифровой паспорт устройства

- ◆ IP (Internet Protocol) — уникальный числовой идентификатор устройства в сети.

Простыми словами: Как номер телефона: чтобы дозвониться, нужно знать номер. Так и сервер «звонит» вашему устройству по IP.

- ◆ Пример:

192.168.1.1 (IPv4, локальный)

2001:0db8:85a3::8a2e:0370:7334 (IPv6, глобальный)

Что проверяет QA:

- Корректно ли определяется геолокация по IP (для локализации контента)
- Работает ли сайт с разных подсетей (например, корпоративный доступ)
- Не блокируется ли доступ по диапазону IP (антифрод, геоблокировка)

🐞 Где баги:

- Кэширование контента по неверному IP → пользователь видит чужие данные
- Ошибки в CORS-политиках при запросах с разных доменов/IP

✓ Как проверять:

Использовать сервисы типа <https://api.ipify.org?format=json>

Менять IP через VPN/прокси и проверять поведение сайта

В DevTools → Network → смотреть Remote Address

#IP

12:52

📠 DNS — телефонная книга интернета

◆ **DNS (Domain Name System)** — система, преобразующая доменные имена (example.com) в IP-адреса (93.184.216.34).

Простыми словами: вы вводите google.com, а DNS находит его IP.

→ **Пример:**

Вы вводите <https://habr.com> → браузер спрашивает у DNS: «Какой у habr.com IP?» → DNS отвечает: 188.114.96.3 → браузер подключается к этому адресу.

🔍 **Как работает (упрощённо):**

Браузер проверяет локальный кэш (на вашем ПК)

Если нет — спрашивает у роутера / провайдера

Если нет — идёт к корневым серверам → TLD (.com) → авторитетный сервер домена

Получает IP и подключается

🔍 **Что проверяет QA:**

- Корректно ли резолвится домен (нет ли «битых» ссылок)
- Как быстро работает DNS (влияет на Time to First Byte)
- Работает ли сайт при смене DNS-провайдера (например, с Google DNS на Cloudflare)

🐛 **Где баги:**

- Кэш DNS устарел → пользователь попадает на старый сервер
- Ошибки в записях DNS (A, CNAME, MX) → сайт не грузится / почта не работает
- Долгий ответ DNS → сайт «тупит» при загрузке

✓ **Как проверять:**

- Команда в терминале: `nslookup example.com` или `dig example.com`
- Онлайн-инструменты: <https://dnschecker.org>
- В DevTools → Network → смотреть DNS Lookup во вкладке Timing

#DNS

13:15

Тестирование ПО

Типы кэша:

Тип	Где хранится	Срок жизни	Пример	📄
Memory Cache	В оперативной памяти	До закрытия вкладки	Картинки в текущей сессии	
Disk Cache	На жёстком диске	До истечения <code>max-age</code>	CSS/JS файлы	
Service Worker	В фоне браузера	Управляется кодом	PWA-приложения	

Кэш браузера — временная память для скорости

◆ **Кэш** — локальное хранилище временных данных (картинки, CSS, JS), чтобы не загружать их каждый раз заново.

Простыми словами: Как сохранить рецепт в закладки: в следующий раз не нужно искать его снова.

→ **Пример:**

Вы зашли на сайт → браузер сохранил логотип (logo.png) → при повторном визите берёт его из кэша, а не качает с сервера.

🔍 **Что проверяет QA:**

- Загружается ли новая версия файла после деплоя (не «залип» ли старый кэш)
- Корректно ли работают заголовки Cache-Control, ETag, Last-Modified
- Не ломается ли функционал при отключённом кэше (режим инкогнито)

🐞 Где баги:

- Агрессивный кэш → пользователи видят старую версию сайта
- Отсутствие кэша → сайт грузится медленно, растёт нагрузка на сервер
- Кэш + авторизация → один пользователь видит данные другого

✓ Как проверять:

DevTools → Network → галочка «Disable cache»

Проверять заголовки ответа: Cache-Control: max-age=3600

Очищать кэш вручную и проверять поведение

#кэш

13:23

Типы кук:

Тип	Срок жизни	Где хранится	Пример использования
Session	До закрытия браузера	В памяти	Корзина покупок без регистрации
Persistent	До указанной даты	На диске	«Запомнить меня» при входе
Secure	Только по HTTPS	—	Токены авторизации
HttpOnly	Недоступны из JS	—	Защита от XSS-атак
SameSite	Ограничивают отправку на другие домены	—	Защита от CSRF

🍪 Куки (Cookies) — данные о пользователе

◆ **Куки** — небольшие текстовые файлы, которые сервер отправляет браузеру для хранения данных о сессии, предпочтениях, авторизации.

→ Пример:

Вы залогинились → сервер отправляет куку session_id=abc123 → браузер отправляет её обратно с каждым запросом → сервер понимает: «А, это тот же пользователь».

🔍 Что проверяет QA:

- Создаётся ли кука при логине и удаляется при логауте
- Не передаются ли чувствительные данные в открытом виде
- Работает ли сайт при отключённых куках (деградация функционала)
- Корректно ли обрабатываются атрибуты Secure, HttpOnly, SameSite

🐞 Где баги:

- Куки без Secure → перехват в публичных сетях
- Отсутствие SameSite → уязвимость к CSRF-атакам
- Конфликт кук на поддоменах (app.site.com vs site.com)

✓ Как проверять:

DevTools → Application → Cookies

Использовать curl: curl -v -c cookies.txt <https://example.com>

Проверять заголовки Set-Cookie в Network-панели

13:28

? Что спрашивают на собеседовании (2026):

- «Чем отличается Cache-Control: no-store от no-cache?»
→ no-store — вообще не кэшировать, no-cache — кэшировать, но всегда проверять актуальность на сервере.
- «Как проверить, что сайт корректно работает при смене DNS?»
→ Использовать публичные DNS (8.8.8.8, 1.1.1.1), проверить TTL записей, протестировать в разных регионах.
- «Почему после деплоя пользователи видят старую версию сайта?»
→ Скорее всего, проблема в кэше браузера или CDN. Решение: версионировать файлы (style.v2.css), использовать Cache-Control: must-revalidate.
- «Как защитить куки от кражи?»
→ Атрибуты Secure (только HTTPS), HttpOnly (недоступны из JS), SameSite=Strict (ограничение по домену).

🔗 Итог за 10 секунд:

- URL — адрес ресурса; проверяй кодировку, параметры, редиректы
- IP — цифровой адрес устройства; тестируй геолокацию и доступ с разных сетей
- DNS — преобразует домен в IP; баги: устаревший кэш, ошибки в записях
- Кэш — ускоряет загрузку; главная проблема — пользователи видят старую версию
- Куки — хранят сессию и настройки; всегда ставь Secure + HttpOnly + SameSite
- Используй: в 2026 — автоматизируй проверку заголовков, тестируй с отключённым кэшем, проверяй куки на уязвимости
- Ошибка новичка: думать, что «у меня работает» = «у всех работает» (забывая про кэш, куки, гео, сети)

18:49

Тестирование ПО

📍 MAC-адрес — физический «паспорт» сетевого устройства

◆ **MAC-адрес (Media Access Control)** — уникальный 48-битный идентификатор сетевой карты, записанный в шестнадцатеричном формате.

Аналогия:

MAC = номер паспорта (не меняется, выдаётся один раз)

IP = адрес регистрации (можно переехать и сменить)

◆ Как MAC и IP работают вместе? (ARP-протокол)

- ◆ ARP (Address Resolution Protocol) — протокол, который «связывает» IP и MAC внутри локальной сети.
- ◆ ARP-таблица — локальная «записная книжка», где хранятся пары IP ↔ MAC.

→ Пример работы:

1. Твой ПК (192.168.1.10) хочет отправить данные принтеру (192.168.1.50)

🔍 Что проверяет QA:

- Корректно ли определяется устройство по MAC при авторизации в корпоративной сети
- Работает ли фильтрация по «белому списку» MAC-адресов (родительский контроль, гостевой доступ)
- Не «утекает» ли MAC-адрес во внешние логи/аналитику (приватность)
- Корректно ли обрабатывается смена IP при том же MAC (переподключение к Wi-Fi)
- Работает ли приложение при подмене MAC (тестирование устойчивости)

🐞 Где баги:

- Клонирование MAC → два устройства с одинаковым MAC в одной сети = конфликты, потеря пакетов
- Устаревшая ARP-таблица → устройство сменило IP, но кэш «помнит» старый MAC → данные уходят не туда
- Фильтрация по MAC без резерва → пользователь сменил устройство → доступ заблокирован, поддержка в шоке
- Утечка MAC в аналитику → нарушение GDPR/политик приватности (MAC = персональный идентификатор)
- Игнорирование IPv6 → в IPv6 MAC используется для генерации адреса (SLAAC), могут быть коллизии

✓ Как проверять:

- Просмотр ARP-таблицы: `arp -a` → сверять, что пары IP ↔ MAC соответствуют реальным устройствам
 - Очистка ARP-кэша:
2. ПК знает IP принтера, но не знает его MAC
 3. ПК рассылает в локалку: «Кто имеет 192.168.1.50? Сообщите свой MAC!»
 4. Принтер отвечает: «Это я! Мой MAC: AA:BB:CC:DD:EE:FF»
 5. ПК сохраняет связь в ARP-таблице и отправляет данные напрямую

? Что спрашивают на собеседовании (2026):

«Можно ли изменить MAC-адрес?»

→ Технически — да (спуфинг), но это временно и может нарушать политики безопасности.

«Почему в одной сети не может быть двух устройств с одинаковым MAC?»

→ ARP не сможет однозначно доставить пакеты → начнутся потери данных и конфликты.

«Как узнать производителя устройства по MAC?»

→ Первые 3 байта (OUI) можно проверить в базе: <https://wireshark.org/tools/oui-lookup.html>

«В чём разница между `arp -a` и `ip neigh`?»

→ `arp -a` — устаревшая команда (работает везде), `ip neigh` — современная (Linux), показывает больше деталей.

📌 Итог за 10 секунд:

- MAC = физический адрес устройства (не меняется, 48 бит, шестнадцатеричный)
- IP = логический адрес в сети (меняется, для маршрутизации между сетями)
- ARP = протокол, который «связывает» IP и MAC внутри локальной сети
- QA-фокус: валидация формата, проверка фильтрации по MAC, тестирование приватности
- Частая ошибка: думать, что MAC = гарантия безопасности (его легко подделать)
- Лучшая практика: не хранить и не передавать MAC без шифрования — это персональные данные
- Проверка: `arp -a`, Wireshark, regex-валидация, тесты на клонирование и смену устройства

19:09

Тестирование ПО

Chrome DevTools (Инструменты разработчика Chrome)

◆ **DevTools** — набор инструментов для отладки, инспекции и профилирования веб-приложений прямо в браузере.

📄 Основные вкладки DevTools для QA:

📄 Elements (Элементы)

Что это: Инспектор HTML/CSS в реальном времени.

Зачем QA:

Сравнивать мокап (дизайн) с реализацией

Проверять стили: размеры, цвета, шрифты, отступы

Искать элементы для автотестов (XPath, CSS-селекторы)

Видеть структуру DOM-дерева

Console (Консоль)

Что это: Панель для выполнения JavaScript и просмотра логов.

Зачем QA:

Видеть ошибки JS (красный текст)

Отлаживать автотесты

Выполнять команды для дебага

Проверять console.log() от разработчиков

Network (Сеть) * ГЛАВНАЯ ДЛЯ QA

Что это: Мониторинг всех сетевых запросов.

Зачем QA:

Анализировать запросы/ответы API

Проверять статус-коды (200, 404, 500)

Смотреть время загрузки

Находить дублирующие запросы

Сохранять логи в формате HAR для баг-репортов

Application (Приложение)

Что это: Управление хранилищами данных.

Зачем QA:

Проверять куки (создание, удаление, атрибуты)

Тестировать localStorage/sessionStorage

Очищать кэш и данные сессии

Проверять аутентификацию

Sources (Источники)

Что это: Отладчик JavaScript с точками остановки.

Зачем QA:

Пошаговая отладка автотестов

Просмотр переменных в реальном времени

Анализ стека вызовов при ошибках

Performance (Производительность)

Что это: Профилировщик скорости работы сайта.

Зачем QA:

Находить «тормозящие» элементы

Анализировать время загрузки

Делать «скриншот» производительности для баг-репортов

Security (Безопасность)

Что это: Информация о сертификатах и шифровании.

Зачем QA:

Проверять HTTPS-соединение

Искать смешанный контент (HTTP + HTTPS)
Контролировать актуальность SSL-сертификатов

Lighthouse

Что это: Автоматический аудит качества сайта.

Зачем QA:

Быстрая проверка производительности

Анализ доступности (accessibility)

SEO-аудит

Проверка best practices

✦ Что спрашивают на собеседовании в 2026:

- 1. «Как сохранить сетевой запрос для баг-репорта?»
→ Правой кнопкой на запрос в Network → «Save all as HAR with content» → приложить файл к багу.
- 2. «Как найти CSS-селектор элемента для автотеста?»
→ Elements → правая кнопка на элемент → Copy → Copy selector (или Copy XPath).
- 3. «Почему нужно очищать кэш перед тестированием?»
→ Старые данные из кэша могут маскировать баги или создавать ложные срабатывания.
- 4. «Как посмотреть, какие куки создал сайт?»
→ Application → Cookies → выбрать домен → увидеть все куки с атрибутами.
- 5. «Что делать, если в Console красные ошибки?»
→ Зафиксировать текст ошибки, стек вызовов, шаги воспроизведения → создать баг-репорт.

🔗 Итог за 10 секунд:

- Суть: DevTools — главный инструмент QA для отладки фронтенда и API
- Когда использовать: Всегда при тестировании веб-приложений
- Частая ошибка: Тестировать без открытого Network → пропуск багов
- Лучшая практика: Очищать кэш перед каждым тестом или использовать Incognito (Ctrl+Shift+N)
- Исключение: Performance-тесты требуют «чистого» кэша для реалистичных метрик
- Предостережение: Изменения в Elements временные — после перезагрузки всё вернётся

🖱️ Тестирование веб-форм: Клиентская vs Серверная валидация

◆ Валидация формы (Form Validation)

Client-side validation / Server-side validation – Процесс проверки данных, введенных пользователем, на соответствие правилам перед сохранением или обработкой.

Простыми словами: Как фильтр на входе: клиентский проверяет «на глаз» (быстро, но можно обмануть), серверный проверяет «по документам» (надёжно, но медленнее).

📦 Что это такое и какие виды есть

◆ Клиентская валидация (Frontend)

Выполняется в браузере до отправки данных. Реализуется через HTML5-атрибуты (required, pattern, minlength) или JavaScript.

→ **Зачем:** Мгновенная обратная связь, экономия трафика, улучшение UX. Пользователь сразу видит, где ошибся.

◆ Серверная валидация (Backend)

Выполняется на сервере после получения HTTP-запроса. Реализуется на языке бэкенда (Java, Python,

Node.js и т.д.).

→ **Зачем:** Безопасность, целостность БД, защита от подделок запросов. Единственный гарант, что «мусор» не попадёт в систему.

Как работают вместе

Клиентская — первый рубеж (подсветка полей, подсказки). Серверная — финальный контроль (бизнес-правила, уникальность, безопасность). Если клиентскую отключить — серверная всё равно должна работать.

★ Что ещё важно знать тестировщику в 2026: что спрашивают на собеседовании

— 1. «Почему нельзя полагаться только на клиентскую валидацию?»

→ Её легко обойти: отключить JS, перехватить запрос через DevTools/Proху, отправить curl/Postman. Серверная — единственный источник истины.

— 2. «Что такое гибридная (асинхронная) валидация?»

→ Проверка на клиенте + мгновенный запрос на сервер без перезагрузки (например, «логин уже занят» или «промокод недействителен»). В 2026 это стандарт для UX.

— 3. «Как проверять валидацию в автотестах?»

→ Playwright/Cypress заполняют поля, проверяют появление ошибок, перехватывают API-запросы (page.route) и валидируют статус-коды (400/422) и тело ответа бэкенда.

— 4. «Как учитывать доступность (a11y) в валидации?»

→ Ошибки должны озвучиваться скринридерами (aria-live="polite"), фокус автоматически перемещаться на первое ошибочное поле, сообщения быть понятными без опоры на цвет.

🕒 Итог за 10 секунд:

— Суть: Клиентская = скорость и UX, Серверная = безопасность и данные. Обе обязательны.

— Когда использовать: Клиентскую — для мгновенной подсказки, Серверную — всегда, как финальный фильтр.

— Частая ошибка: Тестировать только фронтенд и забывать проверять API напрямую.

— Лучшая практика: Отключать JS и пробовать отправить форму — сервер должен отклонить невалидные данные.

— Исключение: Формы без JS (чистый HTML) должны работать через серверную валидацию с перезагрузкой страницы.

— Предостережение: Никогда не доверяй данным только потому, что «браузер их проверил».

4 May 2026

12:22

Тестирование ПО

<https://testbase.ru> – что должен знать начинающий тестировщик.

12:25

Тестирование веб-приложений (Web Application Testing)

📌 РАЗДЕЛ 1: ФУНДАМЕНТ — Что нужно знать перед стартом

📦 Клиент-серверная архитектура

Client-Server — модель, где браузер (клиент) отправляет запросы, а сервер обрабатывает их и возвращает ответ.

→ Что проверяет QA:

Корректность запросов (методы, заголовки, тело)

Статус-коды ответов (200, 400, 500)

Время ответа сервера

Обработку ошибок на клиенте

2 HTTP/HTTPS и методы запросов

HTTP (HyperText Transfer Protocol) — протокол передачи данных в вебе. *HTTPS* — его защищённая версия с шифрованием.

◆ Основные методы: GET (получить), POST (создать), PUT/PATCH (обновить), DELETE (удалить)

.

→ Что проверяет QA:

Соответствие метода действию (не GET для удаления!)

Наличие Content-Type, Authorization заголовков

Шифрование данных (только HTTPS для продакшена)

Корректные статус-коды для каждого сценария

■ РАЗДЕЛ 2: УРОВНИ ТЕСТИРОВАНИЯ — Пирамида тестов

Виды тестирования по уровню доступа

— *Unit* – тестируем: Отдельные функции/методы, тестирует:

Разработчик, Инструменты: Jest, Mocha, Pytest

— *Integration* – тестируем: Взаимодействие модулей/сервисов, тестирует: Разработчики + QA,

Инструменты: Postman, Supertest

— *E2E* – тестируем: Полный пользовательский сценарий, тестирует: QA-автоматизаторы,

Инструменты: Playwright, Cypress

— *UI/UX* – тестируем: Внешний вид и удобство, тестирует: Мануальные QA, Инструменты: Figma,

BrowserStack

■ РАЗДЕЛ 3: ФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ — «Что делает система?»

1 Тестирование форм и ввода данных

◆ Валидация — проверка данных на соответствие правилам до сохранения.

→ Чек-лист проверки формы:

- ☐ Обязательные поля (required)
- ☐ Формат: email, телефон, дата, пароль
- ☐ Граничные значения: мин/макс длина, числа
- ☐ Спецсимволы, эмодзи, пробелы, инъекции
- ☐ Сообщения об ошибках: понятные, на нужном языке
- ☐ Успешная отправка: редирект, сообщение, сохранение в БД
- ☐ Повторная отправка (double submit)
- ☐ Работа без JS (серверная валидация)

2 Тестирование навигации и ссылок

→ Что проверять:

Все ссылки ведут куда надо (нет 404)

Кнопки «Назад»/«Вперёд» браузера работают корректно

Хлебные крошки (breadcrumbs) актуальны

Пагинация: первая/последняя страница, переход по номерам

Глубокие ссылки (deep links) открывают нужный контент

3 Тестирование бизнес-логики

→ Пример: Корзина покупок

1. Добавить товар → количество в иконке обновилось
2. Изменить количество в корзине → пересчиталась сумма
3. Удалить товар → он исчез, сумма обновилась
4. Оформить заказ → создан заказ в БД, пришло письмо
5. Вернуться в корзину после заказа → она пуста или показывает последние товары

🔍 Что проверяет QA: Состояние системы после каждого действия + синхронизация между фронтендом и бэкендом.

13:26

Тестирование ПО

📁 РАЗДЕЛ 4: НЕФУНКЦИОНАЛЬНОЕ ТЕСТИРОВАНИЕ — «Как хорошо система работает?»

📁 Производительность (Performance)

Цель: Убедиться, что приложение быстро работает под нагрузкой.

Метрики для контроля:

Метрика	Норма	Как измерить
Time to First Byte (TTFB)	< 200 мс	DevTools → Network
First Contentful Paint (FCP)	< 1.8 с	Lighthouse
Time to Interactive (TTI)	< 3.9 с	Lighthouse
Время ответа API	< 500 мс	Postman / k6

→ Виды нагрузочного тестирования:

Load: обычная ожидаемая нагрузка

Stress: пиковая нагрузка, поиск предела

Spike: резкий скачок пользователей

Endurance: длительная работа (поиск утечек памяти)

📁 Безопасность (Security)

Цель: Найти уязвимости до злоумышленников.

→ Чек-лист базовой проверки:

- ☐ Все формы защищены от XSS (экранирование ввода)
- ☐ Параметры в URL не содержат чувствительных данных
- ☐ Авторизация: токены в HttpOnly cookies, не в localStorage
- ☐ Роли: пользователь не может получить доступ к админке через URL
- ☐ HTTPS: нет смешанного контента (HTTP-ресурсы на HTTPS-странице)
- ☐ Заголовки безопасности: Content-Security-Policy, X-Frame-Options
- ☐ Ограничение попыток входа (brute-force защита)

🐛 Частые баги:

Прямой доступ к чужим данным через /api/user/123 (IDOR)

Возможность подобрать пароль без капчи/лимитов

Утечка токенов в логах или через referer

📁 Доступность (Accessibility / a11y)

Цель: Сделать приложение удобным для людей с ограничениями.

→ Базовые проверки:

- Навигация с клавиатуры (Tab/Shift+Tab/Enter)
- Скринридеры читают кнопки, формы, ошибки (aria-label, alt)
- Контрастность текста $\geq 4.5:1$ (проверка через Lighthouse)
- Ошибки форм озвучиваются (aria-live="polite")
- Масштабирование до 200% не ломает верстку

→ Стандарт: Руководство WCAG 2.2 уровня AA — минимальный порог для продакшена.

4 Кроссбраузерность и адаптивность

Что проверять:

- Браузеры – Chrome, Firefox, Safari, Edge (последние 2 версии)
- Мобильные – Эмуляция в DevTools + реальные устройства (BrowserStack)
- Разрешения – 1920×1080, 1366×768, 360×640 (мобильный)
- Ориентация – Портретная / альбомная на мобильных
- ОС – Windows, macOS, iOS, Android

✓ Лайфхак: Использовать @media запросы в CSS — проверять, что стили применяются корректно на разных ширинах.

13:39

■ РАЗДЕЛ 5: ТЕХНИКИ ТЕСТ-ДИЗАЙНА ДЛЯ ВЕБ-ПРИЛОЖЕНИЙ

1 Эквивалентное разделение (Equivalence Partitioning)

Суть: Разбить входные данные на группы, где поведение системы одинаково. Тестировать по одному значению из каждой группы.

Пример: Поле «Возраст» (18–100):

Группы:

- <18 → невалидно (тестируем: 17)
- 18–100 → валидно (тестируем: 25, 50, 100)
- >100 → невалидно (тестируем: 101)

2 Анализ граничных значений (Boundary Value Analysis)

Суть: Баги чаще на границах диапазонов. Тестировать: мин, мин+1, макс–1, макс.

Пример: Пароль 8–20 символов:

Тестировать длину: 7, 8, 9, 19, 20, 21

3 Таблица решений (Decision Table)

Суть: Для сложной логики с несколькими условиями — выписать все комбинации и ожидаемые результаты.

Пример: Скидка в корзине:

Сумма заказа — | Есть промокод — | Ожидаемая скидка

< 1000 Р — | Нет — | 0% — — — |

< 1000 Р — | Да — — | 5% — — |

≥ 1000 Р — | Нет — — | 10% — — — |

≥ 1000 Р — | Да — — | 15% — — |

■ РАЗДЕЛ 6: ИНСТРУМЕНТЫ — Арсенал веб-тестировщика

📄 Мануальное тестирование

Инструмент

- Chrome DevTools – Инспекция элементов, сетевые запросы, консоль, кэш
- Postman – Тестирование API, коллекции, окружения, автоматизация
- Figma – Сверка с макетом, экспорт ресурсов, комментарии
- BrowserStack / LambdaTest – Тестирование на реальных устройствах и браузерах
- Lighthouse – Аудит производительности, a11y, SEO, best practices

📄 Автоматизация (2026)

Инструмент — Язык — Плюсы

- Playwright — JS/TS, Python, Java, C# — Быстрый, надёжный, авто-ожидания, мультибраузер
- Cypress — JS/TS — Простой синтаксис, время-тревел дебаг, отличная документация
- WebDriverIO — JS/TS — Гибкость, поддержка мобильных, интеграция с Appium
- k6 — JS — Нагрузочное тестирование, интеграция в CI/CD

→ Тренд 2026:

- AI-помощники для генерации тест-кейсов и локаторов (GitHub Copilot, Testim)
- Low-code автоматизация для мануальных тестировщиков (Reflect, Mabl)
- Визуальное регрессионное тестирование (Percy, Chromatic)

14:11

Тестирование ПО

📌 РАЗДЕЛ 7: ТЕСТОВАЯ ДОКУМЕНТАЦИЯ ДЛЯ ВЕБ-ПРИЛОЖЕНИЙ

◆ Что обязательно вести:

- Чек-лист смоук-тестов — 10-20 критических сценариев (логин, главная, ключевая функция), обновлять при каждом релизе
- Тест-кейсы на регресс — Пошаговые сценарии с ожидаемым результатом, обновлять при изменении функционала
- Баг-репорты — Шаги, окружение, логи, скриншоты/видео, HAR-файлы, обновлять при каждом найденном баге
- Матрица тестирования — Какие браузеры/устройства/роли покрываем, обновлять в начале спринта

→ Формат баг-репорта (минимум):

- Заголовок: [Кратко, что сломалось]
- Окружение: Браузер, ОС, версия приложения
- Шаги воспроизведения: 1. ... 2. ... 3. ...
- Фактический результат: ...
- Ожидаемый результат: ...
- Вложения: Скриншот/видео, HAR-файл, консольные логи
- Приоритет: Critical / High / Medium / Low

📌 РАЗДЕЛ 8: CI/CD И АВТОМАТИЗАЦИЯ — Как встроить тесты в процесс

Пайплайн тестирования веб-приложения

1. Разработчик пушит код → запускается CI (GitHub Actions / GitLab CI)
2. Линтеры + Unit-тесты (быстро, < 2 мин)
3. Сборка приложения → деплой на staging
4. Запуск E2E-тестов (Playwright/Cypress) на staging

5. Визуальные регрессионные тесты (Percy)
6. Нагрузочный смоук (k6, 5 мин)
7. Если всё зелёное → мерж в main / деплой на prod
8. Пост-деплой смоук на prod (только критические сценарии)

→ Что проверяет QA в пайплайне:

Стабильность тестов (нет флейков)

Время прохождения (оптимизировать параллелизацией)

Читаемость отчётов (Allure, HTML-отчёты)

Уведомления в чат при падении

📌 РАЗДЕЛ 9: СОВРЕМЕННЫЕ ТРЕНДЫ 2026

Что изменилось и на что обратить внимание:

◆ Тестирование веб-компонентов:

Многие приложения на React/Vue → тестировать изолированные компоненты (Storybook + Testing Library)

◆ PWA и оффлайн-режим:

Проверять кэширование, работу без сети, синхронизацию после восстановления соединения

◆ Голосовые интерфейсы и AI-фичи:

Тестировать не только UI, но и NLP-обработку, fallback-сценарии при неверном распознавании

◆ Тестирование данных и приватности:

Проверять, что персональные данные не утекают в аналитику, логи, сторонние сервисы

Соответствие GDPR/локальным законам

✓ Shift-Right мониторинг:

После релиза: отслеживать реальные ошибки (Sentry), метрики производительности (RUM), фидбэк пользователей

✦ Что спрашивают на собеседовании в 2026:

1. «Как протестировать форму, если разработчик сказал, что валидация только на фронтенде?»

→ Это красный флаг. Нужно настоять на серверной валидации + протестировать обход фронтенда через Postman.

2. «Как отличить баг фронтенда от баг бэкенда?»

→ Открыть DevTools → Network: если запрос не ушёл / неверные данные в теле → фронтенд; если запрос верный, но ответ 500 / неверные данные → бэкенд.

3. «Что делать, если автотесты стали медленными?»

→ Проанализировать: параллелить тесты, убрать лишние wait, использовать API-тесты вместо UI там, где можно, кэшировать данные.

4. «Как тестировать приложение, которое использует WebSockets?»

→ В DevTools → Network → WS: смотреть сообщения в реальном времени; в автотестах: Playwright поддерживает page.waitForEvent('websocket').

5. «Что проверять в первую очередь при смоук-тесте нового релиза?»

→ Критический путь пользователя (например: логин → главная → ключевое действие → подтверждение), интеграции с внешними сервисами, миграции БД.

☎ Итог за 10 секунд:

— Суть: Веб-тестирование = функционал + производительность + безопасность + удобство + совместимость

— Когда использовать: На всех этапах: от требований до пост-релизного мониторинга

— Частая ошибка: Тестировать только «счастливый путь» и игнорировать ошибки, сеть, разные

браузеры

- Лучшая практика: Всегда проверять серверную валидацию, даже если фронтенд «всё контролирует»
- Исключение: Внутренние инструменты могут иметь упрощённую валидацию, но безопасность — всегда на сервере
- Предостережение: Не доверяй автотестам на 100% — регулярно проводи исследовательское тестирование

15:11

Тестирование ПО

📖 Логи — дневник событий приложения

Логи (Logs / System Logs) — хронологическая запись событий, действий и ошибок, которые происходят в приложении, на сервере или в системе.

◆Виды логов

→ иерархия по убыванию детализации (лог-уровни):

- *DEBUG* — для отладки разработчиками, максимальная детализация

Пример: User object: {id: 123, name: "Anna"}

- *INFO* — нормальные рабочие события, стандарт для продакшена

Пример: User 123 logged in successfully

- *WARNING* — что-то подозрительное, но не критично

Пример: Password attempt 3/5 for user 123

- *ERROR* — ошибка, которая ломает сценарий

Пример: Database connection timeout

- *CRITICAL* — аварийная остановка, система не может работать

Пример: Out of memory, service stopped

→ по месту хранения:

- Клиентские логи — консоль браузера, localStorage

Зачем: отладка фронтенда, ошибки JavaScript

- Серверные логи — файлы на сервере (/var/log/, logs/app.log)

Зачем: анализ бэкенд-ошибок, аудит, расследование инцидентов

- БД-логи — логи запросов к базе данных

Зачем: поиск медленных запросов, отладка данных

- Аудит-логи — отдельное защищённое хранилище

Зачем: фиксация кто, когда и что изменил (для безопасности и комплаенса)

→ по формату:

- Текстовые: 2026-01-15 14:30:22 [INFO] User 123 logged in

- JSON-структурированные:

```
{"timestamp":"2026-01-15T14:30:22Z","level":"INFO","user_id":123,"action":"login"}
```

- Системные: syslog, Windows Event Log

★ Что ещё важно знать тестировщику в 2026: что спрашивают на собеседовании

? «Какие уровни логов ты знаешь и когда какой использовать?»

→ *DEBUG* — для разработки, *INFO* — обычные события, *WARN* — предупреждения, *ERROR* — ошибки, *CRITICAL* — аварийные остановки. В продакшене обычно *INFO+*, в деве — *DEBUG*.

? «Как найти нужную запись в логах, если их тысячи?»

→ Использовать фильтры по времени, уровню (*ERROR*), ключевым словам (*user_id:123*), инструменты: *grep*, *jq* (для JSON), *Kibana/Loki* для визуального поиска.

? «Что делать, если в логах нет чувствительных данных, но баг воспроизводится?»

→ Запросить у разработчиков включить DEBUG-уровень на staging, воспроизвести баг, собрать логи, приложить к репорту, затем вернуть уровень обратно.

? «Как отличить баг фронтенда от бэкенда по логам?»

→ Если ошибка в браузере (Console) и нет запроса на сервер → фронтенд. Если запрос ушёл, но сервер вернул 500/ошибку в своих логах → бэкенд.

? «Что такое структурированные логи и зачем они?»

→ Логи в формате JSON, которые легко парсить автоматически. Позволяют строить дашборды, алерты, быстро фильтровать по полям (user_id, error_code). Стандарт в 2026.

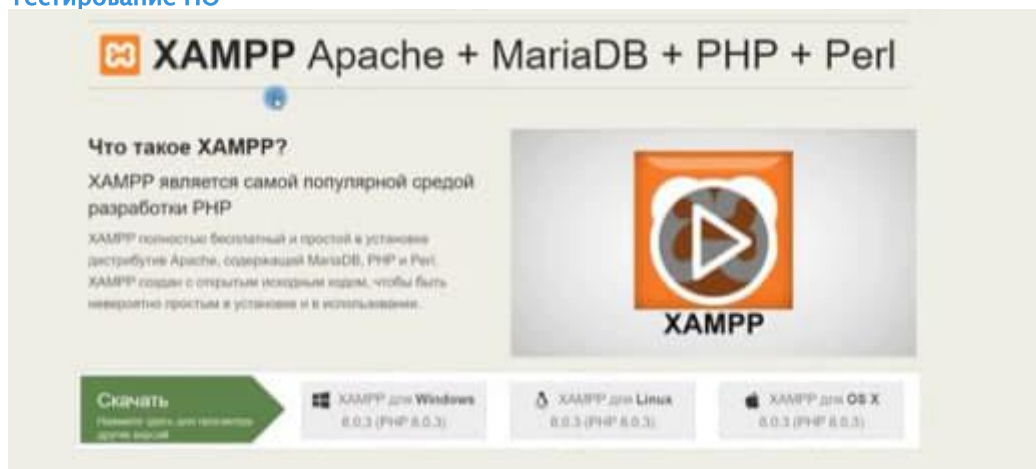
👉 Итог за 10 секунд:

- Суть: Логи — это запись всех событий в приложении для отладки, аудита и анализа
- Когда использовать: При поиске причин багов, анализе инцидентов, проверке безопасности, расследовании жалоб пользователей
- Частая ошибка: Искать баг только по UI, игнорируя логи → трата времени на догадки
- Лучшая практика: Всегда прикладывать релевантные фрагменты логов к баг-репорту (с маскированием чувствительных данных)
- Исключение: В продакшене не логировать пароли, токены, персональные данные в открытом виде (требование безопасности)
- Предостережение: Логи могут быть отключены, переполнены или не синхронизированы по времени — всегда проверяй актуальность и настройки логирования

#логи

15:31

Тестирование ПО





🔗 API Testing — проверка «внутренностей» системы

API (Application Programming Interface) — набор правил и протоколов, позволяющий разным программам обмениваться данными.

◆ Основные виды API

— *REST (Representational State Transfer)*

Самый популярный стандарт. Использует обычные HTTP-запросы. Данные обычно в формате JSON.

→ Пример: GET /users/123 (получить юзера №123).

— *SOAP (Simple Object Access Protocol)*

Старый, строгий стандарт. Данные только в XML. Сложнее в использовании, но очень надёжен для банков.

→ Пример: Сложные банковские транзакции.

— *GraphQL* – Современный и гибкий. Клиент сам пишет запрос, указывая, какие именно поля ему нужны (чтобы не качать лишнее).

→ Пример: Мобильное приложение запрашивает только name и avatar, игнорируя email и address.

#API

16:46

Тестирование ПО

◆ HTTP-методы (Команды API)

— GET — Получить данные (чтение). Не меняет ничего на сервере.

Пример: Открыть профиль, получить список товаров.

— POST — Создать новые данные.

□ Пример: Регистрация, добавление товара в корзину, отправка комментария.

— PUT — Полностью обновить данные (заменить старые на новые).

Пример: Редактирование профиля (нужно отправить все поля заново).

— PATCH — Частично обновить данные (изменить только то, что поменялось).

Пример: Смена только пароля или только аватарки.

— DELETE — Удалить данные.

Пример: Удаление аккаунта или комментария.

◆ Структура запроса и ответа

— Запрос (Request) — что мы отправляем:

Endpoint (URL): Адрес, куда стучим (<https://api.site.com/v1/login>).

Method: Действие (GET, POST...).

Headers (Заголовки): Мета-данные. Самое важное — Authorization (токен доступа) и Content-Type

(формат данных, обычно application/json).

Body (Тело): Сами данные (для POST/PUT). Обычно JSON-объект.

— Ответ (Response) — что нам присылают:

Status Code: Код состояния (200 — ОК, 404 — Не найдено, 500 — Ошибка сервера).

Body: Данные в ответе (список юзеров, сообщение об успехе).

Headers: Информация о сервере, кэшировании и т.д.

✦ Что ещё важно знать тестировщику в 2026: что спрашивают на собеседовании

? «Чем отличается PUT от PATCH?»

→ PUT заменяет объект целиком (если не передать поле, оно сотрётся/станет null). PATCH обновляет только переданные поля, остальные не трогает.

? «Что такое идемпотентность?»

→ Свойство запроса: сколько раз его ни отправь, результат будет одинаковым.

→ Идемпотентные: GET, PUT, DELETE (удалил один раз — удалилось, второй раз — всё равно удалено).

→ НЕ идемпотентный: POST (отправил платёж два раза — списали деньги дважды).

? «Какие статус-коды ты знаешь?»

→ 2xx (Успех): 200 OK, 201 Created (успешно создано).

→ 4xx (Ошибка клиента): 400 Bad Request (кривые данные), 401 Unauthorized (не залогинен), 403 Forbidden (нет прав), 404 Not Found.

→ 5xx (Ошибка сервера): 500 Internal Server Error (упало на бэкенде), 502 Bad Gateway.

? «Как тестировать API без документации?»

→ Использовать снифферы трафика (DevTools Network, Charles Proxy), смотреть, какие запросы шлёт фронтенд, и пробовать повторять их в Postman.

👉 Итог за 10 секунд:

— Суть: API-тестирование — это проверка логики и данных «под капотом», без интерфейса

— Когда использовать: Всегда. Это база. Сначала API, потом UI

— Частая ошибка: Проверять только позитивные сценарии (200 OK) и забывать про 400/500 ошибки

— Лучшая практика: Всегда проверять структуру JSON-ответа (есть ли нужные поля, правильные ли типы данных)

— Исключение: Иногда API возвращает 200 OK, но в теле пишет «Error: ...» — это плохой дизайн, но встречается

— Предостережение: Никогда не тестируй на Production (боевой базе) через POST/DELETE — создашь мусор или удалишь реальные данные

#API

18:34

Тестирование ПО

🔪 SOAP vs REST — два подхода к общению программ

◆ SOAP (Simple Object Access Protocol) — строгий протокол обмена сообщениями в формате XML, использующий WSDL-контракт для описания интерфейса.

◆ REST (Representational State Transfer) — гибкий архитектурный стиль построения веб-сервисов, обычно использующий JSON и стандартные HTTP-методы.

Простыми словами:

→ SOAP — как официальное письмо по почте: строгий бланк, печать, регистрация, доставка с уведомлением. Надёжно, но долго.

→ REST — как сообщение в мессенджере: быстро, просто, гибко. Отправил — получил ответ.

📄 Что такое XML

XML (eXtensible Markup Language) — язык разметки для хранения и передачи структурированных

данных.

```
<user>
  <id>123</id>
  <name>Anna</name>
  <email>anna@example.com</email>
</user>
```

◆ Особенности:

Теги придумываешь сам (в отличие от HTML, где теги фиксированы)

Строгая структура: каждый открывающий тег должен закрываться

Поддержка атрибутов: `<book category="cooking">`

VERBOSE (многословный): много «служебного» текста → большой размер файла

☐ Что такое WSDL — контракт для SOAP

WSDL (Web Services Description Language) — XML-файл, который описывает: какие методы есть у SOAP-сервиса, какие данные принимать и что возвращать.

Простыми словами: Как технический паспорт устройства или меню в ресторане с точным составом блюд. Без WSDL ты не узнаешь, что «заказать» у SOAP-сервиса.

→ Структура WSDL:

```
<definitions> — начало документа
  <types> — какие типы данных используются (string, int, custom)
  <message> — как выглядит запрос и ответ (структура)
  <portType> — какие операции доступны (методы: getUser, createUser)
  <binding> — как передавать данные (SOAP over HTTP)
  <service> — адрес сервиса (URL)
```

◆ Зачем это тестировщику:

WSDL — это документация + автоматическая валидация

Инструменты (SoapUI, Postman) могут импортировать WSDL и сами создать запросы

Если запрос не соответствует WSDL → сервер сразу отклонит его

★ Что ещё важно знать тестировщику в 2026: что спрашивают на собеседовании

? «Почему SOAP всё ещё используют, если REST проще?»

→ Потому что SOAP даёт гарантии: строгая валидация по контракту (WSDL), встроенная безопасность, поддержка транзакций. Для банков и медицины «надёжнее» важнее, чем «проще».

? «Как протестировать SOAP-сервис, если нет документации?»

→ Запросить WSDL-файл (обычно по адресу <https://api.service.com?wsdl>) → импортировать в SoapUI/Postman → инструмент сам сгенерирует шаблоны запросов.

? «В чём главная разница между XML и JSON для тестировщика?»

→ XML: строгий, с тегами и атрибутами, сложнее парсить, тяжелее по весу.

→ JSON: лёгкий, читаемый, нативный для JavaScript, проще в автотестах.

→ Проверка: в XML важно проверять пространства имён (namespaces), в JSON — типы полей (string/number/null).

? «Что делать, если WSDL устарел?»

→ Это частый баг! Сверять WSDL с реальными запросами через сниффер (DevTools/Charles). Если расхождения — создавать баг: «WSDL не актуален, автотесты падают».

☞ Итог за 10 секунд:

— Суть: SOAP — строгий протокол с контрактом (WSDL), REST — гибкий стиль на базе HTTP

- Когда использовать: SOAP — для банков/энтерпрайза, REST — для веба/мобилок/микросервисов
- Частая ошибка: Пытаться тестировать SOAP как REST (игнорировать XML-структуру и WSDL)
- Лучшая практика: Всегда импортировать WSDL в Postman/SoapUI — это сэкономит часы ручной работы
- Исключение: Некоторые «REST»-сервисы используют XML — смотреть не на название, а на Content-Type и структуру
- Предостережение: Не тестируй SOAP-сервисы наугад: без соблюдения контракта ты получишь 500-ю ошибку, а не полезный ответ

#API

19:08

Тестирование ПО

📄 XSD (XML Schema Definition) — паспорт структуры XML

XSD (XML Schema Definition) — язык описания структуры XML-документа: какие элементы могут быть, их типы данных, обязательность, ограничения.

□ Иерархия в SOAP (Simple Object Access Protocol):

1. **WSDL** описывает: какие методы есть у сервиса и какие сообщения он принимает
2. **XSD** описывает: точную структуру этих сообщений (какие поля, какого типа, обязательные или нет)
3. **XML** — это сами данные, которые должны соответствовать XSD

★ Зачем это тестировщику в 2026

1. Валидация запросов/ответов

Ты можешь проверить, что SOAP-сервис возвращает данные строго по схеме. Если в ответе появилось новое поле или исчезло старое → это баг.

2. Автоматическая генерация тестов

Инструменты (SoapUI, Postman) используют XSD для:

Автозаполнения шаблонов запросов

Проверки корректности ответов

Генерации тестовых данных

3. Поиск несоответствий

Разработчик изменил код, но забыл обновить XSD → тесты падают. Или наоборот: XSD обновили, а код нет.

4. Документация

XSD — это формальная документация. Если её нет, можно восстановить структуру данных из XSD.

★ Что спрашивают на собеседовании в 2026

? «В чём разница между WSDL и XSD?»

→ WSDL описывает что делает сервис (методы, операции).

→ XSD описывает как выглядят данные (структура, типы полей).

WSDL может ссылаться на XSD для описания типов сообщений.

? «Как проверить, что XML соответствует XSD?»

→ Использовать XSD-валидатор: в SoapUI (включить валидацию в настройках), онлайн-инструменты (freeformatter.com), или библиотеки в коде (lxml для Python).

? «Что такое namespace в XSD?»

→ Пространство имён (например, xmlns:xs="http://www.w3.org/2001/XMLSchema") нужно, чтобы избежать конфликтов имён элементов. Как префикс у телефона: +7 для России, +1 для США.

? «Может ли SOAP работать без XSD?»

→ Технически да, но это плохая практика. Без XSD нет гарантии, что клиент и сервер «говорят на одном языке». WSDL без XSD = меню без описания блюд.

👉 Итог за 10 секунд:

- Суть: XSD — это схема, которая описывает правила для XML (какие поля, типы, обязательность)
- Когда использовать: При тестировании SOAP-сервисов, валидации XML-запросов/ответов, проверке контрактов
- Частая ошибка: Игнорировать XSD и проверять XML «на глаз» → пропуск багов в типах данных
- Лучшая практика: Включить автоматическую валидацию по XSD в SoapUI/Postman
- Исключение: В простых случаях можно использовать DTD (старый формат), но XSD — стандарт
- Предостережение: XSD может быть сложным и многослойным (один XSD импортирует другой) → проверяй всю цепочку

#XSD #SOAP

6 May 2026

13:00

Тестирование ПО

📄 Основные правила и ошибки в синтаксисе XML

△ Именно эти пункты часто дают на собеседовании как тест «найди 3 ошибки».

◆ Обязательное наличие корневого элемента

В документе может быть только один главный тег, внутри которого лежит всё остальное.

❑ — *Ошибка*: `<tag1></tag1><tag2></tag2>`

— *Правильно*: `<root><tag1></tag1><tag2></tag2></root>`

◆ Наличие закрывающих тегов

Каждый открывающий тег обязан иметь пару (или быть самозакрывающимся `<tag />`).

❑ — *Ошибка*: `<tag>This is a paragraph.<tag>`

❑ — *Правильно*: `<tag>This is a paragraph.</tag>`

◆ Чувствительность к регистру

`<Tag>` и `<tag>` — это совершенно разные элементы.

❑ — *Ошибка*: `<Tag>example</tag>`

❑ — *Правильно*: `<tag>example</tag>`

◆ Правильное вложение

Теги нельзя перекрещивать. Внутренний тег должен закрыться раньше внешнего.

❑ — *Ошибка*: `<b><i>Text</b></i>`

❑ — *Правильно*: `<b><i>Text</i></b>`

◆ Кавычки в значениях атрибутов

Атрибуты всегда должны быть в двойных или одинарных кавычках.

❑ — *Ошибка*: `<tag attribute=value></tag>`

❑ — *Правильно*: `<tag attribute="value"></tag>`

◆ Экранирование спецсимволов

Знаки разметки нельзя использовать как обычный текст. Их нужно заменять на HTML-сущности.

❑ — *Ошибка*: `<tag> 5 < 10 </tag>`

❑ — *Правильно*: `<tag> 5 < 10 </tag>`

(Также: `>` для `>`, `&` для `&`, `"` для `"`, `'` для `'`)

★ Что спрашивают на собеседовании в 2026:

? «Как быстро найти синтаксическую ошибку в XML из 1000 строк?»

→ Не искать глазами. Скопировать в валидатор (xmllint, онлайн-парсер, SoapUI). Он укажет точную строку и символ, где парсер споткнулся.

? «Почему XML строже JSON?»

→ JSON — это формат данных (парсеры часто «прощают» мелочи), а XML — язык разметки со стандартом W3C. Любая синтаксическая ошибка = невалидный документ = мгновенный отказ

сервера.

? «Что будет, если в SOAP-запросе забыть экранировать &?»

→ Парсер подумает, что начинается новая сущность, не найдёт её и вернёт ошибку валидации (обычно HTTP 400 или SOAP Fault), даже не начнёт обработку.

? «Можно ли в XML использовать комментарии?»

→ Да, синтаксис: `<!-- Это комментарий -->`. Но внутри комментария нельзя использовать `--`.

☞ Итог за 10 секунд:

- Суть: XML требует строгой структуры: один корень, парные теги, правильный регистр, кавычки в атрибутах
- Когда использовать: При написании SOAP-запросов, настройке конфигов, валидации ответов API
- Частая ошибка: Забывать экранировать `<`, `>`, `&` внутри текста или перепутать регистр тегов
- Лучшая практика: Всегда прогонять XML через валидатор или нажимать `Ctrl+Shift+P` → `Format Document` перед отправкой
- Исключение: Самозакрывающиеся теги `<tag />` допустимы для пустых элементов
- Предостережение: Не путай XML с HTML — HTML может «прощать» ошибки, XML никогда не прощает

#XML

13:34

Тестирование ПО

🌐 REST и JSON — гибкий стиль общения программ

◆ **REST (Representational State Transfer)** — архитектурный стиль построения веб-сервисов, использующий стандартные HTTP-методы и форматы данных (чаще JSON).

◆ **JSON (JavaScript Object Notation)** — легковесный текстовый формат обмена данными, основанный на парах «ключ: значение».

📦 Что такое REST — основные принципы

- Клиент-сервер: Разделение интерфейса (клиент) и хранения данных (сервер) → независимое развитие.
- Без состояния (Stateless): Каждый запрос содержит всю информацию для обработки. Сервер не «помнит» клиента между запросами.
- Кэшируемость: Ответы можно помечать как кэшируемые → ускорение повторных запросов.
- Единообразие интерфейса:
Ресурсы идентифицируются через URL (`/users/123`)
Действия — через HTTP-методы (GET, POST, PUT, DELETE)
Сообщения — самодостаточны (все данные в запросе/ответе)
- Слоистая система: Клиент не знает, общается ли он напрямую с сервером или через прокси/балансировщик.
- HTTP-методы в REST: GET — получить ресурс, POST — создать новый ресурс, PUT — полностью заменить ресурс, PATCH — частично обновить ресурс, DELETE — удалить ресурс

📦 Что такое JSON — синтаксис и элементы

JSON — текстовый формат данных, который легко читает человек и парсит машина.

— **Объекты** — пары «ключ: значение» в фигурных скобках `{}`

Пример: `{ "name": "John", "age": 30 }`

— **Массивы** — упорядоченный список значений в квадратных скобках `[]`

Пример: `["apple", "banana", "cherry"]`

Массив объектов: `[ { "id": 1 }, { "id": 2 } ]`

Значения внутри — могут быть:

Строка в двойных кавычках: `"Hello"`

Число: 123, 20.5

Булево: true, false

null — пустое значение

Вложенный объект или массив

— Строки — только двойные кавычки, одинарные не допускаются

Правильно: "example"

Ошибка: 'example'

Пример:

```
{  
  "name": "John Doe",  
  "age": 30,  
  "isEmployed": true,  
  "address": {  
    "street": "123 Main St",  
    "city": "Anytown"  
  },  
  "phoneNumbers": ["123-456-7890", "456-789-0123"],  
  "metadata": null  
}
```

★ Что спрашивают на собеседовании в 2026:

? «В чём разница между REST и RESTful?»

→ REST — теория, архитектурные принципы. RESTful — практика, когда сервис реально следует этим принципам (правильные методы, статус-коды, ресурсные URL).

? «Почему в JSON только двойные кавычки?»

→ Потому что стандарт ECMA-404 (JSON) жёстко это фиксирует. Одинарные кавычки — это JavaScript, но не JSON. Парсер выдаст ошибку.

? «Можно ли в JSON использовать комментарии?»

→ Нет, стандарт JSON не поддерживает комментарии. Если нужно — используй отдельное поле "comment": "..." или документацию.

? «Что такое HATEOAS в REST?»

→ Hypermedia as the Engine of Application State: в ответе сервера есть ссылки на связанные ресурсы.

Пример: после создания заказа приходит ссылка /orders/123/pay для оплаты. Редко используется, но спрашивают.

☞ Итог за 10 секунд:

— Суть: REST — гибкий стиль на базе HTTP, JSON — удобный формат данных с парами «ключ: значение»

— Когда использовать: Почти всегда в вебе и мобилках — это современный стандарт для новых сервисов

— Частая ошибка: Писать ключи в JSON без кавычек или использовать одинарные кавычки → парсер упадёт

— Лучшая практика: Всегда валидировать JSON через онлайн-инструменты или Postman перед отправкой

— Исключение: Иногда в ответе может прийти не JSON, а текст/HTML — смотреть заголовок Content-Type

— Предостережение: Не доверяй типу данных «на глаз» — 30 (число) и "30" (строка) в JSON это разные значения

15:25

📌 API

◆ **API (Application Programming Interface)** — интерфейс прикладного программирования, механизм, позволяющий разным программам обмениваться данными и функциями.

◆ Типы API

— **Локальные API** — взаимодействие внутри одной системы/приложения

Пример: модули одного приложения общаются через внутренние функции

— **Удалённые API (Web API)** — взаимодействие через сеть по HTTP/HTTPS

Пример: мобильное приложение → сервер через REST/SOAP

#API

14:03

Тестирование ПО

◆ Способы вызова API

— *Вызов функции системой* — автоматизированное взаимодействие компонентов внутри одной системы

Пример: модуль авторизации вызывает модуль логирования

— *Вызов метода другой системой* — одна система использует функционал другой через сеть

Пример: микросервис заказов обращается к микросервису пользователей

Основа микросервисной архитектуры

— *Вызов метода человеком через GUI* — пользователь взаимодействует с интерфейсом, который «под капотом» шлёт запросы к API

Пример: нажал кнопку «Войти» → фронтенд отправил POST /api/login

◆ Как работает удалённый API (HTTP)

1. Клиент формирует HTTP Request:

- Метод: GET/POST/PUT/DELETE
- URL: `https://api.example.com/users`
- Headers: Content-Type, Authorization
- Body: данные (для POST/PUT)

2. Сервер обрабатывает запрос

3. Сервер возвращает HTTP Response:

- Status Code: 200, 404, 500...
- Headers: метаданные
- Body: результат (JSON/XML)

◆ Дополнительно: что важно знать тестировщику

Статус-коды (часть ответа API):

- 2xx — успех: 200 OK, 201 Created
- 4xx — ошибка клиента: 400 Bad Request, 401 Unauthorized, 404 Not Found
- 5xx — ошибка сервера: 500 Internal Server Error

Форматы данных:

- JSON — стандарт для REST: `{ "key": "value" }`
- XML — используется в SOAP: `<key>value</key>`

Аутентификация в API:

- API Key — простой ключ в заголовке
- Bearer Token (JWT) — токен в Authorization: Bearer <token>
- OAuth 2.0 — делегированный доступ через сторонний сервис

Инструменты для тестирования:

- Postman — коллекции запросов, тесты, окружения
- curl — быстрые запросы в терминале
- Swagger/OpenAPI — интерактивная документация
- DevTools → Network — перехват запросов фронтенда

✦ Что спрашивают на собеседовании в 2026:

? «В чём разница между локальным и удалённым API?»

→ Локальный работает внутри одного приложения (быстро, без сети), удалённый — через сеть по HTTP (медленнее, но позволяет соединять разные системы).

? «Почему микросервисы общаются через API, а не напрямую к БД?»

→ Чтобы сохранить независимость сервисов: каждый отвечает за свои данные, а другие получают их только через чёткий контракт (API). Это упрощает поддержку и масштабирование.

? «Как протестировать API, если нет документации?»

→ Перехватить реальные запросы через DevTools → Network или Charles Proxy, изучить структуру и повторить в Postman.

☞ Итог за 10 секунд:

- Суть: API — это «посредник» между программами для обмена данными и функциями
- Когда использовать: При тестировании интеграций, микросервисов, мобильного бэкенда
- Частая ошибка: Тестировать только через UI, игнорируя прямые вызовы API
- Лучшая практика: Всегда проверять статус-коды, структуру ответа и валидацию входных данных
- Исключение: Некоторые API возвращают 200 OK с ошибкой в теле — читать документацию
- Предостережение: Не тестируй методы POST/DELETE на боевом сервере без изоляции данных

15:38

Тестирование ПО



✦ Postman и Swagger — инструменты тестировщика для работы с API

◆ Postman — платформа для разработки, тестирования и документирования API через удобный графический интерфейс.

◆ Swagger (OpenAPI) — спецификация и набор инструментов для описания, визуализации и тестирования REST API.

#Postman #Swagger

15:38



16:01

Тестирование ПО

❏ Что такое Postman и зачем он нужен

◆ **Postman** — приложение для отправки HTTP-запросов, проверки ответов, автоматизации тестов и совместной работы над коллекциями.

— Отправка запросов всех методов (GET, POST, PUT, DELETE)

Пример: проверить, что GET /api/users возвращает список пользователей

— Работа с окружениями (переменные для dev/stage/prod)

Пример: одна коллекция работает на всех серверах, меняется только {{base_url}}

— Написание автотестов на JavaScript

Пример: проверить статус-код и наличие полей в ответе за 2 клика

— Коллекции и папки для организации запросов

Пример: папка «Авторизация», папка «Заказы», папка «Негативные тесты»

— Генерация кода запроса для разных языков

Пример: скопировать готовый curl или fetch для вставки в автотест

— Мониторинг и запуск тестов по расписанию

Пример: каждый час проверять, что главный эндпоинт отвечает 200 OK

❏ Что такое Swagger (OpenAPI) и зачем он нужен

◆ **Swagger** — фреймворк для описания REST API в формате YAML/JSON + инструменты для визуализации и тестирования.

— Интерактивная документация (Swagger UI)

Пример: разработчик обновил код → документация обновилась автоматически

— Описание эндпоинтов, параметров, типов данных и ответов

Пример: видно, что POST /users требует name: string, email: string (required)

— Кнопка «Try it out» для тестирования прямо в браузере

Пример: заполнить форму → отправить запрос → увидеть ответ без установки постмана

— Генерация клиентских библиотек и серверных заглушек

Пример: скачать готовый SDK для Python или TypeScript

— Валидация соответствия кода спецификации

Пример: линтер ругается, если код возвращает поле, которого нет в Swagger

💡 Как Postman и Swagger работают вместе

— Swagger описывает, как должен работать API

— Postman использует это описание для создания тестов

— Экспорт из Swagger → импорт в Postman = коллекция готова за 1 минуту

Пример:

Разработчик выложил Swagger по ссылке <https://api.example.com/swagger.json>

В Postman: Import → Paste URL → коллекция с 50 запросами создана

Тестировщик добавляет тесты, переменные окружения и запускает регресс

★ Что спрашивают на собеседовании в 2026:

? «Как быстро начать тестировать новый API?»

→ Запросить Swagger-документацию → импортировать в Postman → проверить базовые эндпоинты (здоровье, авторизация, основной ресурс) → добавить негативные тесты.

? «В чём разница между Swagger и Postman?»

→ Swagger — это описание/документация API (контракт). Postman — инструмент для отправки запросов и тестирования (исполнение). Swagger говорит «что должно быть», Postman проверяет «что есть на самом деле».

? «Как протестировать авторизацию в Postman?»

→ В вкладке Authorization выбрать тип (Bearer Token / API Key) → вставить токен → отправить запрос. Для автоматизации: получить токен через запрос POST /login в Pre-request Script и сохранить в переменную.

? «Что делать, если Swagger устарел?»

→ Это частый баг. Сверять описание с реальными ответами через Postman. Если расхождения → фиксировать: «Документация не актуальна, тесты ломаются». Требовать обновления Swagger как часть Definition of Done.

🔗 Итог за 10 секунд:

- Суть: Postman — инструмент для тестирования API, Swagger — стандарт описания и документирования
- Когда использовать: Swagger — на старте работы с новым API, Postman — на всём протяжении тестирования
- Частая ошибка: Тестировать «на глаз» без сохранения запросов в коллекцию → невозможно повторить или автоматизировать
- Лучшая практика: Импортировать Swagger в Postman, добавить тесты, использовать переменные окружений для разных стадий
- Исключение: Некоторые старые API не имеют Swagger → придётся описывать запросы вручную или перехватывать через DevTools
- Предостережение: Не храните реальные токены и пароли в коллекциях — используйте переменные и .env файлы

[#postman](#) [#swagger](#)

17:23

🗄 БД и СУБД: Хранилище данных, связи и нормальные формы

◆ **БД (База данных / Database)** — структурированное хранилище данных, организованное для удобного сохранения, поиска и управления информацией.

◆ **СУБД (Система управления базами данных / DBMS)** — программное обеспечение для создания, чтения, обновления и удаления данных в БД через язык запросов (обычно SQL).

☐ Отличия БД от СУБД

- База данных (БД) — само хранилище, файл или набор файлов с данными
Пример: файл shop.db с таблицами пользователей и заказов
- СУБД — программа, которая управляет этим хранилищем
Пример: MySQL, PostgreSQL — они понимают SQL-запросы и возвращают результаты

[#БД](#) [#СУБД](#)

17:58

Тестирование ПО

☐ Типы баз данных

- *Иерархические БД* — данные организованы как дерево, связи «родитель–потомок»
Пример: файловая система компьютера (папки → файлы)
- *Сетевые БД* — сложная структура с множественными связями между записями
Пример: старые корпоративные системы 70–80-х годов
- *Реляционные БД (SQL)* — данные в таблицах, связанных через общие ключи

Пример: PostgreSQL, MySQL — для заказов, пользователей, финансов (с ними работаем на курсе)

— *Нереляционные БД (NoSQL)* — данные не в таблицах, а в форматах: ключ–значение, документы, колонки, графы

Пример: MongoDB (документы), Redis (кэш), Cassandra (большие данные)

□ Типы СУБД

◆ Реляционные (SQL):

- MySQL — открытая, популярна в веб-разработке (на курсе работаем с ней)
- PostgreSQL — открытая, поддерживает сложные запросы и большие объёмы
- Oracle Database — коммерческая, для корпоративных решений
- Microsoft SQL Server — интеграция с продуктами Microsoft

◆ Нереляционные (NoSQL):

- MongoDB — документо-ориентированная, хранит данные в формате, похожем на JSON
- Redis — ключ–значение, используется для кэширования и очередей сообщений
- Cassandra — для работы с большими объёмами данных, высокая скорость записи

□ Типы отношений между таблицами

— *1 к 1* (Один к одному): каждая запись в одной таблице соответствует одной записи в другой

Пример: таблица «Сотрудники» ↔ таблица «Паспорта»: у одного сотрудника один паспорт

— *1 ко многим* (Один ко многим): одна запись в первой таблице связана со многими во второй

Пример: таблица «Отделы» ↔ таблица «Сотрудники»: в одном отделе много сотрудников

— *Многие к 1* (Многие к одному): множество записей в одной таблице связаны с одной записью в другой

Пример: таблица «Заказы» ↔ таблица «Пользователи»: много заказов у одного пользователя

— *Многие ко многим*: записи в обеих таблицах могут быть связаны с множеством записей в другой

Пример: таблица «Студенты» ↔ таблица «Курсы»: один студент ходит на много курсов, один курс посещают много студентов

→ Реализуется через третью таблицу-связку (например, enrollments)

□ Нормализация и денормализация

— Нормализация — процесс организации данных для снижения избыточности и улучшения целостности

— Избыточность — повторение данных в разных местах → риск несоответствия при обновлении

— Целостность — точность и надёжность данных, обеспечение корректных связей между таблицами

— Денормализация — намеренное добавление избыточности для ускорения чтения данных

— Используется в аналитике, кэшировании, высоконагруженных системах

— Минус: медленнее запись, больше места, риск рассинхрона

□ Нормальные формы (кратко)

— 0NF (Нулевая): нет структуры, данные могут дублироваться и быть неструктурированными

— 1NF (Первая): каждая запись уникальна, все атрибуты содержат атомарные (неделимые) значения

Пример: вместо phones: "123,456" → отдельная таблица user_phones с одной записью на номер

— 2NF (Вторая): таблица в 1NF + все неключевые поля зависят от всего первичного ключа (актуально для составных ключей)

Пример: если ключ (order_id, product_id), то product_name должен зависеть только от product_id

— 3NF (Третья): таблица в 2NF + нет транзитивных зависимостей (неключевые поля не зависят друг от друга)

Пример: если city определяет zip, вынести их в отдельный справочник

— 4NF, 5NF — существуют, но на практике используются редко, достаточно знать первые две формы

#БД #СУБД

17:59

🔗 Как это связано с тестированием

— Тестовые данные: зная структуру таблиц и связи, можно готовить валидные наборы для автотестов

Пример: создать пользователя → получить его id → использовать в заказе

— Проверка целостности: после действий в приложении сверять, что данные в связанных таблицах синхронизированы

Пример: удалили категорию → проверить, что товары либо удалены, либо перенесены в «без категории»

— Поиск багов в логике: дублирование, потерянные связи, «висячие» записи — часто следствие нарушения нормальных форм

Пример: один товар в двух категориях с разной ценой → баг в бизнес-логике

— Написание проверочных запросов: простые SQL-проверки в автотестах экономят время на отладку

Пример: `SELECT COUNT(*) FROM orders WHERE user_id = 123` → убедиться, что заказ создан

✦ Что спрашивают на собеседовании в 2026:

? «В чём разница между БД и СУБД?»

→ БД — это само хранилище данных (файлы, таблицы). СУБД — программа, которая управляет этим хранилищем (принимает запросы, возвращает результаты). Как файл .txt (БД) и текстовый редактор (СУБД).

? «Зачем нужны нормальные формы?»

→ Чтобы избежать дублирования данных и аномалий при обновлении. Если цена товара хранится в 10 местах, при изменении придётся обновлять везде — легко ошибиться. Нормализация = один источник истины.

? «Как реализовать связь многие-ко-многим в реляционной БД?»

→ Через третью таблицу-связку (junction table) с двумя внешними ключами. Пример: students, courses, enrollments(student_id, course_id).

? «Когда стоит денормализовать базу?»

→ Когда скорость чтения критична, а данные редко меняются: аналитика, отчёты, кэш. Но с контролем актуальности: например, пересчитывать денормализованные поля по расписанию.

? «Как проверить, что данные не дублируются?»

→ Написать запрос с `GROUP BY` и `HAVING COUNT(*) > 1` по уникальным полям. В автотестах: после создания записи проверить, что в БД ровно одна такая запись.

🏆 Итог за 10 секунд:

— Суть: БД — хранилище, СУБД — инструмент управления, нормальные формы — правила против дублирования

— Когда использовать: При проектировании тестовых данных, проверке целостности связей, анализе багов с данными

— Частая ошибка: Игнорировать связи между таблицами → тестировать «в вакууме», пропуская интеграционные баги

— Лучшая практика: Знать схему БД проекта, писать простые проверочные SQL-запросы в автотестах, хранить тестовые данные в миграциях

— Исключение: В аналитических БД (Data Warehouse) сознательно денормализуют для скорости отчётов

— Предостережение: Не меняй данные в боевой БД напрямую — только через приложение или на staging-окружении

#БД #СУБД

8 May 2026

12:46

Тестирование ПО

III Типы данных в БД и базовые SQL-команды

◆ **Типы данных (Data Types)** — правила, определяющие, какой вид информации можно хранить в столбце таблицы (числа, текст, даты и т.д.).

◆ **SQL-команды** — инструкции для управления базами данных и таблицами (создание, чтение, обновление, удаление).

□ Числовые типы данных

— **INTEGER (INT)** — целые числа без дробной части Пример: `age INT` → хранение возраста: 25, 100, -5

— **FLOAT** — числа с плавающей точкой (одинарная точность) Пример: `price FLOAT` → хранение цены: 19.99, 0.5

— **DOUBLE** — числа с плавающей точкой (двойная точность, больше знаков после запятой)

Пример: `average_score DOUBLE` → средняя оценка: 4.87654321

— **DECIMAL / NUMERIC** — точные числа с фиксированной точностью (для финансов) Пример: `balance DECIMAL(10,2)` → сумма: 12345678.90 (всего 10 цифр, 2 после запятой)

□ Строковые типы данных

— **VARCHAR(n)** — строки переменной длины до `n` символов Пример: `first_name VARCHAR(50)` → «Anna» займёт место под 4 символа, а не под все 50

— **CHAR(n)** — строки фиксированной длины `n` символов (дополняется пробелами) Пример: `gender CHAR(1)` → всегда 1 символ: «М», «F», «О»

— **TEXT** — длинные текстовые данные (статьи, комментарии, описания) Пример: `comments TEXT` → хранение отзыва на 1000+ символов

— **ENUM('val1', 'val2')** — поле принимает только одно из заданных значений Пример: `status ENUM('active', 'inactive', 'pending')` → нельзя записать «deleted», только из списка

□ Дата и время

— **DATE** — только дата (год-месяц-день) Пример: `birth_date DATE` → 1990-05-15

— **TIME** — только время (часы:минуты:секунды) Пример: `appointment_time TIME` → 14:30:00

— **DATETIME** — дата и время вместе, фиксированное значение Пример: `created_at DATETIME` → 2026-01-15 14:30:22

— **TIMESTAMP** — дата и время, автоматически обновляется при изменении записи, зависит от часового пояса Пример: `updated_at TIMESTAMP` → меняется само при каждом `UPDATE`

□ Специальные типы

— **BOOLEAN** — логический тип: `TRUE / FALSE` (или `1 / 0`) Пример: `is_active BOOLEAN` → пользователь активен или нет

— **BLOB (Binary Large Object)** — хранение бинарных данных (изображения, файлы)

Пример: `profile_picture BLOB` → фото профиля в виде байтов

— **NULL** — специальное значение, означающее «нет данных», «неизвестно» Пример: `middle_name VARCHAR(50) NULL` → у пользователя может не быть отчества

— **Пустая строка ('')** — не то же самое, что `NULL`! Это строка длиной 0 символов Пример: `comment VARCHAR(255) = ''` → поле заполнено, но пусто; `NULL` → поле вообще не заполнялось

□ Работа с базами данных: базовые команды

- Показать все БД - `SHOW DATABASES;`
- Создать новую БД - `CREATE DATABASE mydb;`
- Удалить БД - `DROP DATABASE mydb;`
- Выбрать БД для работы - `USE mydb;`

❑ Работа с таблицами: создание и структура

- Создать таблицу – `CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50));`
- Посмотреть структуру таблицы – `DESCRIBE users;`
- Удалить таблицу – `DROP TABLE users;`

❑ Работа с данными: CRUD-операции

- Добавить запись – `INSERT INTO users (name) VALUES ('Anna');`
- Прочитать данные – `SELECT * FROM users WHERE id = 1;` — Обновить запись – `UPDATE users SET name = 'Liza' WHERE id = 1;`
- Удалить запись – `DELETE FROM users WHERE id = 1;`

❑ Изменение структуры таблицы

- Добавить столбец – `ALTER TABLE users ADD COLUMN phone VARCHAR(20);`
- Изменить тип столбца – `ALTER TABLE users MODIFY COLUMN phone INT;`
- Удалить столбец – `ALTER TABLE users DROP COLUMN phone;`

⚠ Важно: В командах `UPDATE` и `DELETE` всегда добавляй условие `WHERE`, иначе изменятся или удалятся ВСЕ строки в таблице!

13:56

Тестирование ПО

✦ Что спрашивают на собеседовании в 2026:

? «В чём разница между VARCHAR и CHAR?»

→ VARCHAR — переменная длина, экономит место (хранит только фактические символы). CHAR — фиксированная длина, быстрее при частом доступе, но тратит место на пробелы. Используй CHAR для кодов, статусов из 1–2 символов.

? «NULL и пустая строка — это одно и то же?»

→ Нет! NULL = «данных нет вообще», пустая строка "" = «данные есть, но они пустые». В запросах: IS NULL для проверки на NULL, = "" для пустой строки.

? «Зачем нужен TIMESTAMP, если есть DATETIME?»

→ TIMESTAMP автоматически обновляется при изменении записи и учитывает часовые пояса. DATETIME — фиксированное значение, не зависит от таймзоны. Для `created_at` лучше DATETIME, для `updated_at` — TIMESTAMP.

? «Что будет, если выполнить UPDATE без WHERE?»

→ Обновятся ВСЕ строки в таблице! Это частая причина продакшен-инцидентов. Всегда проверяй условие перед запуском.

? «Как выбрать правильный тип для цены?»

→ Не FLOAT/DOUBLE (погрешности округления)! Используй DECIMAL(10,2) — точное хранение копеек.

👉 Итог за 10 секунд:

- Суть: Типы данных определяют, что можно хранить в столбце; SQL-команды управляют БД и таблицами
- Когда использовать: При проектировании схемы БД, написании тестовых данных, проверке целостности через запросы
- Частая ошибка: Использовать FLOAT для денег, забывать WHERE в UPDATE/DELETE, путать NULL и ""
- Лучшая практика: Для цен — DECIMAL, для строк — VARCHAR, для статусов — ENUM, всегда тестировать запросы на staging
- Исключение: CHAR быстрее VARCHAR для фиксированных коротких значений (коды, флаги)
- Предостережение: DROP DATABASE и DELETE без WHERE удаляют данные безвозвратно — проверяй условие дважды!

14:14

Тестирование ПО

III SELECT, JOIN, HAVING, UNION и импорт/экспорт в MySQL

SELECT, JOIN, HAVING, UNION — основные SQL-операторы для выборки, объединения, фильтрации итогов и склейки результатов, а также инструменты для переноса данных в/из файлов.

◆ Простыми словами:

- **SELECT** — «покажи мне данные»
- **JOIN** — «склеить две таблицы по общему полю»
- **HAVING** — «отфильтруй уже посчитанные итоги»
- **UNION** — «объедини результаты двух запросов в один список»
- Импорт/экспорт — «выгрузить таблицу в CSV или загрузить файл в БД»

□ SELECT — выборка данных

- Показать все колонки и строки — `SELECT * FROM users;`
- Выбрать конкретные колонки — `SELECT id, username, email FROM users;`
- Отфильтровать строки по условию — `SELECT * FROM users WHERE age > 18;` Отсортировать результат — `SELECT * FROM users ORDER BY created_at DESC;`
- Ограничить количество записей — `SELECT * FROM users LIMIT 10;`
- Дать псевдоним колонке — `SELECT username AS login FROM users;`

□ JOIN — объединение таблиц

- Внутреннее соединение (только совпадающие записи) — `SELECT u.name, o.total FROM users u INNER JOIN orders o ON u.id = o.user_id;`
- Левое соединение (все из левой + совпадения из правой, иначе NULL) — `SELECT u.name, o.total FROM users u LEFT JOIN orders o ON u.id = o.user_id;`
- Правое соединение (все из правой + совпадения из левой) — `SELECT u.name, o.total FROM users u RIGHT JOIN orders o ON u.id = o.user_id;`

□ HAVING и UNION — фильтрация итогов и склейка

- Фильтр после группировки (WHERE не работает с агрегатами) — `SELECT user_id, COUNT(order_id) as cnt FROM orders GROUP BY user_id HAVING cnt > 5;`
- Объединить результаты двух запросов (убирает дубли) — `SELECT city FROM customers UNION SELECT city FROM suppliers;`
- Объединить с сохранением дублей (быстрее) — `SELECT city FROM customers UNION ALL SELECT city FROM suppliers;`

□ Импорт и экспорт данных

- Выгрузить таблицу в CSV (через SQL) — `SELECT * INTO OUTFILE '/tmp/users.csv' FIELDS TERMINATED BY ',' FROM users;`
- Загрузить данные из CSV в таблицу — `LOAD DATA INFILE '/tmp/users.csv' INTO TABLE users FIELDS TERMINATED BY ',';` — Экспорт через консоль (удобнее для QA) — `mysql -u root -p mydb -e "SELECT * FROM users;" > users.csv` Импорт SQL-дампа базы — `mysql -u root -p mydb < backup.sql`

★ Что спрашивают на собеседовании в 2026:

- ? «В чём разница между WHERE и HAVING?»
→ **WHERE** фильтрует строки до группировки, **HAVING** фильтрует результаты после **GROUP BY**. Пример: `WHERE price > 100` (отбирает товары), `HAVING COUNT(*) > 5` (отбирает категории, где больше 5 товаров).
- ? «Когда использовать UNION, а когда UNION ALL?»
→ **UNION** убирает дубликаты (тратит ресурсы на сортировку), **UNION ALL** возвращает всё как есть (быстрее). В тестах чаще нужен **UNION ALL**, чтобы не потерять данные и ускорить выполнение.
- ? «Почему LEFT JOIN возвращает NULL в правых колонках?»
→ Потому что для записи из левой таблицы нет совпадений в правой. Это норма. Если ожидаешь

данные — проверь целостность внешних ключей.

? «Как безопасно выгрузить данные для анализа, не нагрузив прод?»

→ Использовать реплику (read-only сервер), добавлять фильтры по дате и **LIMIT**, экспортировать через DBeaver/консоль, избегать **SELECT *** на больших таблицах.

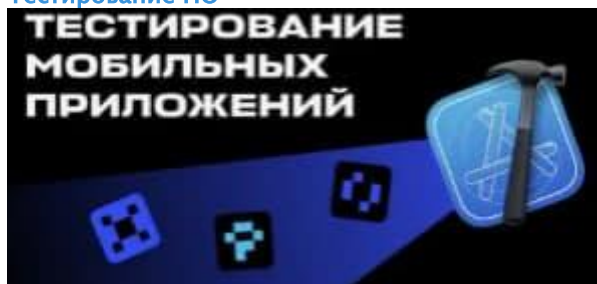
👉 Итог за 10 секунд:

- **Суть:** SELECT выбирает, JOIN склеивает, HAVING фильтрует итоги, UNION объединяет запросы, импорт/экспорт переносит данные в файлы
- **Когда использовать:** Для проверки данных в БД, создания тестовых выборок, отчётов, миграции между средами
- **Частая ошибка:** Путать WHERE и HAVING, забывать ON в JOIN, использовать UNION вместо UNION ALL без необходимости
- **Лучшая практика:** Всегда указывать конкретные колонки вместо *, проверять условия JOIN, использовать **UNION ALL** для скорости
- **Исключение:** FULL JOIN в MySQL напрямую не поддерживается, эмулируется через LEFT JOIN UNION RIGHT JOIN
- **Предостережение:** Экспорт на проде может заблокировать таблицу — всегда тестируй выгрузки на staging или реплике

12 May 2026

15:52

Тестирование ПО



📱 Тестирование мобильных приложений

Мобильное тестирование (Mobile Testing) — процесс проверки качества, функциональности и удобства мобильных приложений на различных устройствах, ОС и версиях.

📱 Основные отличия Android и iOS

- **Компания:** iOS → Apple, Android → Google
- **Ядро:** iOS → Облегченная версия macOS, Android → Linux
- **Система:** iOS → Закрытая (контролируется Apple), Android → Открытая (open source)
- **Язык разработки:** iOS → Objective-C / Swift, Android → Java / Kotlin
- **Магазин приложений:** iOS → App Store, Android → Google Play
- **Расширение приложений:** iOS → .ipa, Android → .apk
- **Установка без магазина:** iOS → Нет (только через TestFlight или джейлбрейк), Android → Да (можно установить APK напрямую)

15:57

📱 Типы мобильных приложений

— **Нативные приложения** — разрабатываются специально для определенной ОС с использованием рекомендованных языков и инструментов (Swift для iOS, Java/Kotlin для Android)

✓ Плюсы:

- Оптимизированы под платформу → лучшая производительность и плавность
- Полный доступ к функциям устройства (камера, микрофон, геолокация, акселерометр)
- Могут работать оффлайн → важно для приложений без сети

☹ Минусы:

- Отдельная разработка для каждой платформы → выше стоимость и время
- Обслуживание и обновление требуют двойных затрат → каждая версия тестируется отдельно

— **Веб-приложения** — мобильные сайты, адаптированные под экраны смартфонов, работают через браузер

✓ Плюсы:

- Одна версия работает на всех платформах и устройствах
- Обновления на сервере → пользователям не нужно скачивать из магазина
- Проще в разработке → стандартные веб-технологии (HTML, CSS, JS)

⊖ Минусы:

- Требуют постоянного интернета → без сети не работают
- Ограниченный доступ к функциям устройства
- Ниже производительность по сравнению с нативными

— **Гибридные приложения** — сочетание нативных и веб-приложений: разрабатываются на веб-технологиях (HTML, CSS, JS) и упаковываются в нативную оболочку

✓ Плюсы:

- Единая база кода для всех платформ → быстрее разработка
- Снижение затрат → общие веб-технологии
- Использование фреймворков (React Native, Ionic, Flutter)

⊖ Минусы:

- Ниже производительность → особенно в сложных анимациях
- Ограниченный доступ к функциям устройства через веб-оболочку
- Проблемы с консистентностью интерфейса на разных платформах

☐ Что необходимо для тестирования

- Изучение требований и статистики перед началом тестирования

Выбор ОС/платформ и версий:

- Учитывать популярные ОС и версии → покрыть максимум пользователей
- Пример: iOS 15–17, Android 10–14 (старые версии тоже важны!)

Выбор устройств:

- Разнообразие устройств на рынке → разные размеры экранов, характеристики
- Пример: iPhone SE (маленький), iPhone 15 Pro Max (большой), Samsung Galaxy (разные линейки)

Выбор форм-фактора:

- Телефоны, планшеты, носимые устройства (умные часы)
- Влияет на процесс тестирования → разные жесты, ориентация экрана

— **Формирование тестовой фермы**

Покупка или аренда устройств:

- Физические устройства → самое точное тестирование
- Минус: дорого, требует места

Эмуляторы и симуляторы:

- Эмулятор (Android) → эмулирует железо и ОС
- Симулятор (iOS) → эмулирует только ОС (быстрее, но менее точно)

Облачные сервисы:

- BrowserStack, Sauce Labs, LambdaTest
- Доступ к множеству виртуальных устройств
- Экономия ресурсов и ускорение разработки

☐ Особенности мобильного тестирования

- **Прерывания (Interrupts)** — как приложение реагирует на внешние события Пример: входящий

звонок, SMS, push-уведомление, низкий заряд батареи, потеря сети

— **Жесты** — проверка различных способов взаимодействия Пример: тап, свайп, пинч (масштабирование), долгое нажатие, встряхивание

— **Ориентация экрана** — портретная и альбомная Пример: поворот устройства → интерфейс адаптируется, данные не теряются

— **Разные типы сетей** — 2G, 3G, 4G, 5G, Wi-Fi Пример: переключение между сетями → приложение не падает, данные синхронизируются

— **Разрешения** — доступ к камере, микрофону, геолокации Пример: пользователь запретил доступ → приложение корректно обрабатывает, показывает инструкцию как включить

15:57

★ Что спрашивают на собеседовании в 2026:

? «В чём разница между эмулятором и симулятором?»

→ **Эмулятор** (Android) — эмулирует и железо, и ОС, медленнее, но точнее. **Симулятор** (iOS) — эмулирует только ОС, быстрее, но менее точно. Для тестирования производительности нужны реальные устройства.

? «Как тестировать приложение, если у тебя нет физического устройства?»

→ Использовать эмуляторы (Android Studio), симуляторы (Xcode) или облачные сервисы (BrowserStack). Но перед релизом обязательно протестировать на реальных устройствах — эмуляторы не покажут проблемы с батареей, нагревом или реальными жестами.

? «Что проверять при повороте экрана?»

→ Интерфейс адаптируется под новую ориентацию, данные не теряются, текущее состояние сохраняется, нет наложений элементов, клавиатура не перекрывает важные поля.

? «Как тестировать работу приложения без интернета?»

→ Включить авиарежим или отключить сеть в настройках эмулятора. Проверить: приложение показывает понятное сообщение, кэшированные данные отображаются, действия пользователя сохраняются для синхронизации при восстановлении связи.

? «Почему нельзя тестировать только на эмуляторах?»

→ Эмуляторы не показывают: реальное потребление батареи, нагрев устройства, производительность на старом железе, точную работу жестов, проблемы с памятью, реальное время отклика сети.

🕒 Итог за 10 секунд:

— Мобильное тестирование = функционал + совместимость + UX + производительность на разных устройствах

— **Когда использовать:** На всех этапах разработки, начиная с ранних сборок и заканчивая пост-релизным мониторингом

— **Частая ошибка:** Тестировать только на одном устройстве или эмуляторе → пропуск багов на других платформах

— **Лучшая практика:** Комбинировать реальные устройства (критические сценарии) + эмуляторы (быстрая проверка) + облачные фермы (покрытие редких устройств)

— **Исключение:** Внутренние корпоративные приложения можно тестировать на ограниченном парке устройств

— **Предостережение:** Никогда не выпускай приложение без проверки на реальных устройствах — эмуляторы скрывают 30–40% проблем

📱 Особенности тестирования мобильных приложений

Мобильное тестирование (Mobile App Testing) — комплексная проверка функциональности, производительности, совместимости и удобства мобильного приложения в реальных условиях использования.

◆ **Простыми словами:** Как тест-драйв автомобиля: проверяешь не только «едет ли», но и как ведёт себя на разной дороге, при разной погоде, с разным грузом и пассажирами.

☐ Работа с функциями телефона

— **GPS / Геолокация** — проверка корректности работы карт, маршрутов, гео-тегов

Пример: приложение показывает «вы здесь» → сверить с реальными координатами; протестировать перемещение (пешком, в машине)

— **Камера (видео/фото)** — интеграция с камерой: съёмка, обработка, загрузка

Пример: сделать фото → приложение сжимает → загружает на сервер; проверить разрешения, ошибки при отсутствии камеры

— **Экран: размер, разрешение, ориентация** — адаптивность интерфейса

Пример: поворот устройства → интерфейс перестраивается, данные не теряются; проверить на маленьких (iPhone SE) и больших (Pro Max) экранах

— **Жесты** — управление через тапы, свайпы, пинч, долгое нажатие

Пример: зум карты двумя пальцами → масштаб меняется плавно; свайп для удаления → элемент исчезает с анимацией

— **Другие функции** — Bluetooth, NFC, звонки, SMS, контакты

Пример: оплата через NFC → транзакция проходит; приложение запрашивает доступ к контактам → пользователь разрешает/запрещает → корректная обработка

☐ Работа с производительностью

— **Скорость / отзывчивость** — время реакции на действия пользователя

Пример: нажатие кнопки → отклик < 100 мс; загрузка экрана < 2 сек; анимации 60 FPS

— **Загрузка оперативной памяти (RAM)** — мониторинг потребления памяти

Пример: приложение использует > 500 MB на старом устройстве → система закрывает его; проверить утечки через Android Profiler / Xcode Instruments

— **Потребление батареи** — влияние приложения на время работы устройства

Пример: приложение «съедает» 20% заряда за час → пользователь удаляет; оптимизировать фоновые процессы

— **Установка: внутренняя память / внешняя** — работа при разных местах установки

Пример: установить на SD-карту → приложение запускается, данные сохраняются; проверить права доступа

☐ Тестирование установки

— **Установка** — процесс загрузки и первоначальной настройки

Пример: скачать из App Store/Google Play → установить → первый запуск → онбординг → всё работает

— **Удаление** — полное удаление без «хвостов»

Пример: удалить приложение → проверить, что кэш, логи, настройки тоже удалены (если не задумано иное)

— **Переустановка** — повторная установка после удаления

Пример: удалить → установить заново → данные не восстановились (если не было бэкапа) → это норма; если восстановились без входа — баг безопасности

— **Обновление** — переход с прошлой версии на новую

Пример: версия 1.0 → 2.0; проверить: настройки сохранились, база данных мигрировала, старый кэш не ломает новый функционал

☐ Тестирование прерываний (Interrupts)

— **Входящие/исходящие вызовы** — поведение при звонке

Пример: во время оплаты пришёл звонок → после разговора приложение восстановилось, заказ не потерялся

— **Уведомления / всплывающие окна** — реакция на системные события

Пример: push-уведомление → тап по нему открывает нужный экран; уведомление от другого приложения не перекрывает критичные кнопки

— **Разрядка / подзарядка** — изменение состояния батареи

Пример: низкий заряд → приложение предлагает сохранить данные; подключение к зарядке → возобновляет фоновую синхронизацию

— **Сворачивание / разворачивание** — переход в фон и обратно

Пример: свернуть приложение → через 10 минут вернуться → сессия активна, данные на месте; проверить таймауты авторизации

☐ Интернет и сетевые условия

— **Тип соединения** — Wi-Fi, 3G, 4G, 5G, EDGE

Пример: переключиться с Wi-Fi на мобильные данные → загрузка продолжается без ошибки

— **Качество соединения** — стабильность при плохом сигнале

Пример: скорость 100 Кбит/с, потеря пакетов 10% → приложение показывает индикатор загрузки, не падает

18:42

— **Потеря связи** — внезапный обрыв сети

Пример: отключить интернет во время отправки формы → приложение сохраняет черновик, показывает «нет сети», предлагает повторить

— **Смена типа соединения** — адаптивность при переключении

Пример: выход из зоны Wi-Fi → автоматический переход на 4G → пользователь не замечает разрыва

☐ Дополнительные сценарии

— **Мультиязычность** — работа при разных языках системы

Пример: переключить язык устройства на арабский → интерфейс зеркалится (RTL), все строки переведены, нет «кракозябр»

— **Совместимость с аксессуарами** — умные часы, фитнес-браслеты, наушники

Пример: получить уведомление на часы → тап открывает приложение на телефоне; синхронизация шагов с браслетом

— **Мультимедиа** — воспроизведение аудио/видео внутри приложения

Пример: видео проигрывается в фоне при сворачивании; пауза при звонке, возобновление после

— **Интеграция с соцсетями** — авторизация, шеринг контента

Пример: войти через ВКонтакте → токен сохраняется; поделиться результатом → открывается нативный диалог шеринга

— **Безопасность** — защита данных, уязвимости

Пример: токены не хранятся в логах; трафик шифруется (HTTPS); биометрия (Face ID) работает корректно; нет утечек через clipboard

— **ISTQB Mobile Syllabus** — официальный источник углублённых знаний

Пример: скачать и изучить «ISTQB Mobile Application Testing Foundation Level» по ссылке: <https://www.rstqb.org/ru/istqb-downloads.html>

🌟 Что спрашивают на собеседовании в 2026:

? «Как тестировать приложение при плохом интернете?»

→ Использовать инструменты: Charles Proxy (throttling), Android Emulator Network Conditions, Xcode Network Link Conditioner. Проверять: индикаторы загрузки, повторные попытки, сохранение черновиков, понятные сообщения об ошибке.

? «Что проверять при обновлении приложения?»

→ Сохранение пользовательских данных и настроек, миграция локальной БД, совместимость старого кэша с новым кодом, корректность онбординга для новых функций, откат при неудачном обновлении.

? «Как тестировать геолокацию без реального перемещения?»

→ В эмуляторе: задать координаты или маршрут (Android: `adb emu geo fix`, iOS: Simulate Location). На реальном устройстве: использовать моки локации через инструменты разработчика или тестовые сборки.

? «Почему важно тестировать прерывания?»

→ Пользователи редко используют приложение в «тепличных» условиях. Звонок, уведомление, сворачивание — частые сценарии. Если приложение не восстанавливается — пользователь уходит к конкурентам.

? «Как проверить потребление батареи?»

→ На Android: `adb shell dumpsys batterystats`, Battery Historian. На iOS: Xcode → Energy Log. Тестировать сценарии: фоновая синхронизация, постоянное использование GPS, яркие анимации.

👉 Итог за 10 секунд:

- **Суть:** Мобильное тестирование = функционал + производительность + совместимость + реальные условия использования
- **Когда использовать:** На всех этапах: от ранних сборок до пост-релизного мониторинга
- **Частая ошибка:** Тестировать только в идеальных условиях (стабильный Wi-Fi, новое устройство, без прерываний)
- **Лучшая практика:** Комбинировать эмуляторы (быстро) + реальные устройства (точно) + облачные фермы (покрытие)
- **Исключение:** Внутренние корпоративные приложения можно тестировать на ограниченном парке устройств
- **Предостережение:** Никогда не выпускай без проверки на реальных устройствах — эмуляторы скрывают проблемы с памятью, батареей и сенсорами

#тестирование мобильных приложений

19 May 2026

14:07

Тестирование ПО



📺 Эмулятор Android Studio в тестировании

🔹 **Эмулятор (Android Emulator)** — программная копия реального Android-смартфона, запускаемая на компьютере. Позволяет тестировщикам проверять приложения в разных конфигурациях без покупки физических устройств.

#тестирование мобильных приложений

14:11

Эмулятор создаёт изолированную среду (виртуальную машину), которая воспроизводит работу операционной системы, процессора и сенсоров. В 2026 году он работает на 60–80% быстрее благодаря нативной поддержке ARM-чипов (Apple Silicon, Windows on ARM) и улучшенному графическому рендерингу.

- **Виды системных образов:** Stock (чистый Android без сервисов), Google APIs (с картами и уведомлениями), Play Store (с магазином и проверкой лицензий).
- **Настройка профиля:** задаются разрешение экрана, объём ОЗУ, версия ОС, тип процессора (x86_64/ARM64) и поддержка датчиков.
- **Принцип работы:** эмулятор подключается через ADB (Android Debug Bridge — инструмент управления и отладки), загружает системный образ и эмулирует сенсоры, сеть и местоположение через панель Extended Controls.
- **Роль QA:** ручная проверка адаптивности, локализации, тестирование на старых/новых версиях Android, прогон автотестов (Maestro, Appium) и симуляция плохой связи без выезда в поля.

🌟 Что спрашивают на собеседовании в 2026:

? Чем эмулятор отличается от реального устройства и когда его недостаточно? → ✓ Эмулятор копирует софт и базовое железо, но не заменяет физические датчики, реальный GPS и специфику вендорных оболочек (MIUI, OneUI).

? Как ускорить запуск эмулятора в CI/CD пайплайнах? → ✓ Использовать pre-booted Docker-образы, отключить GPU-рендеринг для headless-режима и применять Snapshots (мгновенные снимки состояния).

? Почему UI-тесты проходят в эмуляторе, но падают на телефоне? → ✓ Из-за разницы в частоте отрисовки кадров, плотности пикселей и скорости выполнения фоновых процессов. Решается динамическими ожиданиями (waits) и тестированием на 2–3 реальных девайсах.

? Какой стек автоматизации актуален в 2026 для эмуляторов? → ✓ Maestro (быстрый, минимальный код, нативная поддержка жестов) и Appium 2.x с плагином UiAutomator2 для сложных кроссплатформенных сценариев.

☛ Итог за 10 секунд:

- Суть: виртуальный Android-телефон для ручных проверок, отладки и автотестов.
 - Когда использовать: при проверке версий ОС, адаптивности, сетевых условий, локализации и в автоматических пайплайнах.
 - Частая ошибка: принимать результаты эмулятора за финальную истину без проверки на физическом устройстве.
 - Лучшая практика: использовать Cold Boot только при смене образа, далее работать с Quick Boot (Snapshots) для экономии времени.
- Исключение
- Предостережение: не тестировать камеру, NFC, Bluetooth, точный расход батареи, AR и работу с реальными SIM-картами. Функции либо имитируются, либо недоступны.

#Android Studio #тестирование мобильных приложений

18:40

Тестирование ПО

🔗 Эмуляторы и симуляторы: кратко для QA

◆ **Эмулятор (Emulator)** — программа, которая копирует и «железо», и ОС устройства. Работает медленнее, но точнее. Пример: Android Studio Emulator.

◆ **Симулятор (Simulator)** — программа, которая копирует только поведение ОС, без эмуляции процессора. Работает быстро, но менее точно. Пример: iOS Simulator в Xcode.

[Разбор темы]

- Эмулятор = виртуальная машина с полной имитацией архитектуры (ARM/x86). Подходит для тестов производительности, работы с железом, отладки драйверов.
- Симулятор = слой совместимости, запускающий код «как есть» на хост-машине. Идеален для быстрой проверки UI, логики, навигации.
- Ключевое различие: эмулятор отвечает на вопрос «Как это работает на реальном чипе?», симулятор — «Как это выглядит для пользователя?».

✦ Что спрашивают на собеседовании в 2026:

? Почему iOS-симулятор нельзя запустить на Windows? → ✓ Он компилирует код под архитектуру macOS и использует её системные библиотеки, а не эмулирует железо iPhone.

? Когда эмулятор обязательнее симулятора? → ✓ При тестировании работы с камерой, датчиками, разной плотностью пикселей, версиями Android и фрагментацией устройств.

? Можно ли доверять только симулятору при релизе? → ✓ Нет. Симулятор не покажет реальный расход батареи, нагрев, поведение при нехватке памяти и особенности вендорных прошивок.

☛ Итог за 10 секунд:

- Суть: эмулятор копирует устройство целиком, симулятор — только среду выполнения.
- Когда использовать: эмулятор — для глубоких и аппаратных тестов; симулятор — для быстрой

проверки UI и логики.

- Частая ошибка: считать, что «если работает в симуляторе — будет работать везде».
- Лучшая практика: комбинировать оба инструмента + финальная проверка на 2–3 реальных девайсах.

Исключение

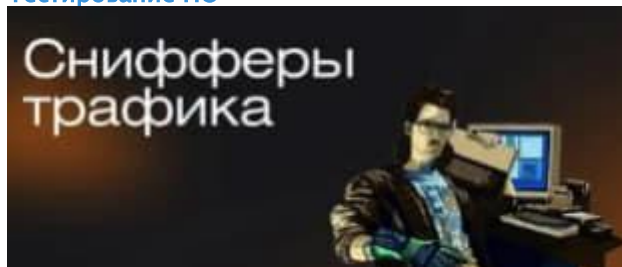
- Предостережение: не тестируйте в эмуляторах/симуляторах NFC, Bluetooth, точный геолокационный трекинг, AR и работу с физическими аксессуарами.

#эмулятор #симулятор #тестирование мобильных приложений

25 May 2026

14:11

Тестирование ПО



📌 **Снифферы (Sniffers)** — перехват и анализ сетевого трафика

Сниффер (Sniffer / Network Interceptor) — программа или устройство, которое перехватывает, логирует и анализирует сетевой трафик между клиентом и сервером, действуя как прокси-посредник.

14:20

❏ Как работает сниффер: модель «клиент — прокси — сервер»

- *Обычная модель:* клиент ↔ сервер (прямое соединение)
- *Модель со сниффером:* клиент ↔ прокси-сервер ↔ сервер

Принцип работы:

- Настройка устройства: указать в настройках Wi-Fi/мобильной сети адрес прокси (компьютера со сниффером) Пример: в настройках телефона → Wi-Fi → Proxy → Manual → IP: **192.168.1.100**, Port: 8888
- Установка сертификата: для перехвата HTTPS-трафика нужно установить доверенный сертификат сниффера на устройство Пример: скачать сертификат из Charles (Help → SSL Proxying → Save Root Certificate) → установить на телефон → включить доверие в настройках
- Перехват трафика: все запросы приложения идут через прокси → сниффер показывает их в реальном времени Пример: приложение шлёт **POST /api/login** → сниффер показывает заголовки, тело запроса, ответ сервера, время выполнения

❏ Для чего используют снифферы в тестировании

- Мониторинг сетевого трафика — наблюдение за взаимодействием приложения с сервером в реальном времени Пример: увидеть, какие запросы шлёт приложение при нажатии кнопки «Купить», какие параметры передаются, какой ответ приходит
- Модификация контента — изменение запросов и ответов «на лету» для тестирования Пример: изменить в запросе price: 100 на price: 1 → проверить, что сервер не принимает подделанную цену; или подменить ответ 200 OK на 500 Error → проверить обработку ошибок на фронтенде

- Эмуляция слабых сетевых условий — имитация плохого интернета для проверки устойчивости приложения Пример: в Charles: Proxy → Throttling → Preset: 3G → приложение должно показать индикатор загрузки, а не «зависнуть»
- Отладка и воспроизведение багов — сохранение запросов для передачи разработчикам Пример: найти баг → сохранить запрос как .har файл → приложить к баг-репорту → разработчик импортирует и воспроизведёт ту же ситуацию
- Проверка безопасности — анализ передачи чувствительных данных Пример: увидеть, что пароль передаётся в открытом виде (не через HTTPS) или токен «светится» в логах → зафиксировать уязвимость

📦 Популярные инструменты

- **Charles Proxy** — кроссплатформенный прокси для HTTP/HTTPS/WebSocket Пример: удобная визуализация, маппинг (подмена ответов), троттлинг сети, повторная отправка запросов
- **Fiddler Classic / Fiddler Everywhere** — прокси для Windows (Classic) и кроссплатформенный (Everywhere) Пример: мощный отладчик, скрипты на FiddlerScript, автоматизация через командную строку
- **Wireshark** — анализатор пакетов на низком уровне (не только HTTP) Пример: показывает весь сетевой трафик (TCP, UDP, DNS), полезен для отладки протоколов, но сложнее в использовании
- **Proxyman** — современный сниффер для macOS/iOS/Android Пример: красивый интерфейс, встроенная эмуляция устройств, удобная работа с сертификатами

★ Что спрашивают на собеседовании в 2026:

? «Как перехватить трафик мобильного приложения, если оно не доверяет пользовательским сертификатам?»

→ Это защита (SSL Pinning). Варианты: использовать тестовую сборку приложения с отключенным пиннингом, применить Frida для обхода проверки, или тестировать на уровне эмулятора с root-доступом.

? «В чём разница между Charles и Wireshark?»

→ Charles — прокси уровня приложения (видит только HTTP/HTTPS, удобен для веба и мобильных приложений). Wireshark — сниффер уровня пакетов (видит весь трафик: TCP, UDP, DNS), мощнее, но сложнее для анализа конкретных запросов.

? «Как протестировать обработку ошибки 500, если сервер её не возвращает?»

→ Использовать сниффер: в Charles — Tools → Breakpoints (остановить запрос → изменить ответ) или Map Local (подменить ответ файлом). В Fiddler — AutoResponder.

? «Почему после настройки прокси приложение не грузит данные?»

→ Возможные причины: не установлен/не доверен SSL-сертификат, приложение использует SSL Pinning, неверно указан IP/порт прокси, приложение не поддерживает прокси (редко).

14:20

📌 Итог за 10 секунд:

- **Суть:** Сниффер — посредник между приложением и сервером, который видит, записывает и может менять весь сетевой трафик
- **Когда использовать:** При отладке интеграций, поиске причин багов, тестировании обработки ошибок, проверке безопасности, эмуляции плохой сети
- **Частая ошибка:** Забыть установить или доверить сертификат → перехватывается только часть трафика или ничего не видно
- **Лучшая практика:** Сохранять проблемные запросы в **.har** для баг-репортов, использовать троттлинг для проверки оффлайн-режима
- **Исключение:** Приложения с жестким SSL Pinning могут блокировать перехват — использовать

тестовые сборки или обходные пути

- **Предостережение:** Не тестируй с включенным сниффером на реальных пользовательских данных
- можно случайно изменить или утечь чувствительную информацию

14:35



Git

Git — распределённая система контроля версий для отслеживания изменений в коде и координации работы команды.

◆ Основные понятия

- **Система контроля версий (VCS)** — инструмент для сохранения истории изменений файлов Пример: как автосохранение в игре + возможность откатиться к любому сохранению
- **Распределённая система** — у каждого разработчика полная копия репозитория с историей Пример: можно работать оффлайн, коммитить локально, потом синхронизироваться с сервером
- **Репозиторий (repo)** — хранилище проекта со всеми файлами и историей изменений Пример: локально — папка `.git`, удалённо — GitHub/GitLab/Bitbucket
- **Коммит (commit)** — снимок состояния файлов на момент сохранения с уникальным хэшем
Пример: `a1b2c3d` — автор, дата, сообщение «Fix login bug»

14:36

◆ Ключевые преимущества Git

- **Ветвление (branching)** — создание изолированных копий для новых фич Пример: `main` — стабильная версия, `feature/login` — разработка входа
- **Слияние (merging)** — объединение изменений из разных веток Пример: завершили фичу → мержим в `main` → код попадает в продакшен
- **Работа оффлайн** — коммиты, ветки, история доступны локально Пример: нет интернета → продолжаешь работать, потом пушишь изменения
- **Отказоустойчивость** — потеря сервера не страшна, история есть у всех Пример: упал GitHub → клонируешь репо у коллеги → работа продолжается

★ Что спрашивают на собеседовании в 2026:

? «Чем Git отличается от SVN?»

→ Git — распределённая (полная история у каждого), SVN — централизованная (история только на сервере). В Git можно коммитить оффлайн, в SVN — нет.

? «Что такое хэш коммита?»

→ Уникальный идентификатор (40 символов, SHA-1), например `a1b2c3d4...`. Используется для ссылки на коммит, проверки целостности, отката изменений.

? «Зачем нужны ветки?»

→ Для изоляции изменений: новая фича, фикс бага, эксперимент — всё в отдельной ветке, чтобы не ломать стабильную версию `main`.

☞ Итог за 10 секунд:

- **Суть:** Git — система для отслеживания изменений кода, работы в команде и отката к предыдущим версиям
- **Когда использовать:** Всегда при разработке ПО, даже в одиночку — для истории и безопасности
- **Частая ошибка:** Коммитить с сообщением «fix» или «update» → непонятно, что менялось

- **Лучшая практика:** Маленькие коммиты с понятными сообщениями, ветки под каждую задачу, регулярный пуш на удалённый репо
- **Исключение:** Бинарные файлы (картинки, видео) плохо отслеживаются в Git — использовать Git LFS или внешнее хранилище
- **Предостережение:** `git push --force` может перезаписать историю и сломать работу команды — использовать только в личных ветках

20:00

Тестирование ПО

🔗 Git Workflow

Git Workflow — стандартный процесс работы с Git: от получения кода до отправки изменений и слияния веток.

◆ Основные этапы работы

- **Клонирование репозитория** — создание локальной копии удалённого репо со всей историей
Команда: `git clone <url>` Пример: `git clone https://github.com/user/project.git`
- **Создание ветки** — изолированная линия разработки для новой фичи или фикса Команда: `git branch <branch_name>` Пример: `git branch feature/login`
- **Переключение ветки** — переход на другую ветку для работы Команда: `git checkout <branch_name>` или `git switch <branch_name>` Пример: `git checkout feature/login`
- **Добавление изменений** — подготовка изменённых файлов к коммиту (добавление в индекс)
Команда: `git add <file>` или `git add .` (все файлы) Пример: `git add src/login.js`
- **Коммит** — фиксация изменений в истории с сообщением Команда: `git commit -m "message"` Пример: `git commit -m "Add login form validation"`
- **Отправка изменений (push)** — загрузка локальных коммитов на удалённый репозиторий
Команда: `git push origin <branch_name>` Пример: `git push origin feature/login`
- **Слияние веток (merge)** — объединение изменений из ветки в основную Команда: `git merge <branch_name>` или через Pull Request на GitHub/GitLab Пример: `git checkout main` → `git merge feature/login`
- **Разрешение конфликтов** — ручное исправление ситуаций, когда Git не может автоматически слить изменения Действия: открыть файл с конфликтом → выбрать нужную версию → `git add` → `git commit`
- **Обновление локального репо (pull)** — синхронизация с удалённым репозиторием Команда: `git pull origin <branch_name>` Пример: `git pull origin main`

★ Что спрашивают на собеседовании в 2026:

? «В чём разница между `git fetch` и `git pull`?»

→ `git fetch` — только скачивает изменения с сервера, но не применяет. `git pull` = `git fetch` + `git merge` (скачивает и сразу применяет).

? «Что такое индекс (*staging area*)?»

→ Промежуточная область между рабочей директорией и репозиторием. Файлы, добавленные через `git add`, попадают в индекс перед коммитом.

? «Как разрешить конфликт слияния?»

→ 1) Открыть файл с конфликтом (помечен <<<<<<, =====, >>>>>>), 2) Выбрать нужную версию или объединить вручную, 3) `git add <file>`, 4) `git commit`.

🔗 Итог за 10 секунд:

- **Суть:** Git workflow — последовательность: clone → branch → add → commit → push → merge
- **Когда использовать:** При любой командной разработке для изоляции изменений и безопасного слияния
- **Частая ошибка:** Работать напрямую в `main` → сломать стабильную версию для всей команды
- **Лучшая практика:** Ветка под каждую задачу, частые коммиты, пуш в конце дня, pull перед началом работы
- **Исключение:** Срочный хотфикс можно сделать в `main`, но сразу создать ветку для отката при

проблемах

— **Предостережение:** Перед `git push` всегда делай `git pull`, чтобы избежать конфликтов и не перезаписать чужие изменения

■ QA-процесс с нуля: Пошаговое построение

QA-процесс (Quality Assurance Process) — система правил, артефактов и взаимодействий, обеспечивающая контроль качества продукта на всех этапах разработки.

12:30

ссылка



12:30



12:30



12:30



12:31

□ Этап 1: Изучение текущей ситуации

- Методология разработки — Agile, Scrum, Kanban или Waterfall? От этого зависит ритм и точки входа тестирования.
- Состав команды — кто уже в проекте: разработчики, аналитики, есть ли QA или ты первый?
- Наличие требований — есть ли ТЗ, user stories, макеты или всё держится в головах?
- Наличие тестовой документации — чек-листы, кейсы, автотесты или полный хаос?
- Наличие ПО — чем уже пользуются: Jira, TestRail, Postman, или всё в чатах и таблицах?
- Исследовательское тестирование — пока нет структуры, проверка идёт «на ощупь», что нормально на старте.

□ Этап 2: Стратегия и планирование

- Выбор тестовой документации — решаем, что ведём: чек-листы (быстро), тест-кейсы (подробно) или только автотесты.
- Организация требований — наводим порядок: эпики, user stories, acceptance criteria в едином месте.
- Выбор ПО — определяем стек: баг-трекер, TMS, API-клиенты, инструменты CI/CD.
- Участие в SDLC и настройка BTS — прописываем статусы бага (New → In Progress → Ready for QA → Done).
- Создание тест-плана/стратегии — фиксируем, что, как, когда и кем тестируем.
- Создание тестового окружения — готовим staging/qa-сервер, тестовые данные, доступы, моки внешних сервисов.

□ Этап 3: Написание документации и тестирование

Разработка чек-листов/наборов — покрываем критичные сценарии, «счастливый путь», негативы и граничные значения.

- Разработка регрессионных наборов — отбираем тесты, которые запускаем перед каждым релизом.
- Процедура работы с багами — правила оформления: шаги, ожидаемый/фактический результат, логи, скриншоты, приоритет.
- Введение матрицы трассировки — связываем требования ↔ тест-кейсы ↔ баги, чтобы ничего не потерялось.
- Интеграционное тестирование — проверяем стыки модулей, API, микросервисов.
- Пользовательское тестирование (UAT) — отдаём продукт бизнесу/заказчику на финальную

приёмку.

- Релизное тестирование — смоук, проверка миграций, бэкапов, готовность к выкатке.
- Работа с окружениями — синхронизация dev/stage/prod, очистка данных между прогонами.

□ Этап 4: Отчётность

- Отчёты по результатам — итог спринта/релиза: что протестировано, сколько багов найдено/закрыто, оставшиеся риски.
- Сбор метрик и анализ — количество дефектов, время на тест, покрытие требований, процент возвратов, стабильность билдов.

□ Этап 5: Дополнительно (культура качества)

- Участие в общих митингах — стендапы, планирование, груминг, ретро.
- Инфо-сессии о QA — объясняем команде, как правильно заводить баги и зачем нужны артефакты.
- Предложения и улучшения — оптимизация процессов не только в QA, но и в аналитике/разработке.
- Участие в демо-сессиях — показываем, что протестировано и готово к релизу.
- Менторинг и онбординг — обучаем новых QA, делимся знаниями с разработчиками и аналитиками.

★ Что спрашивают на собеседовании в 2026:

? «С чего начать построение QA-процесса, если в компании его нет?»

→ С аудита: понять методологию, команду, наличие требований и инструментов. Затем внедрить минимально жизнеспособный процесс: баг-трекер + чек-листы + чёткий жизненный цикл бага. Не усложнять сразу.

? «Как выбрать между чек-листами и тест-кейсами на старте?»

→ Чек-листы — быстрее, гибче, подходят для Agile. Тест-кейсы — детальнее, нужны для строгой регламентации или аутсорса. На старте лучше чек-листы + автотесты на критичный путь.

? «Зачем нужна матрица трассировки?»

→ Чтобы гарантировать, что каждое требование покрыто тестами, а каждый баг связан с требованием. Помогает при аудите и оценке рисков при изменениях.

12:31

☞ Итог за 10 секунд:

- Суть: QA-процесс — это система от аудита текущего состояния до отчётности и культуры качества
 - Когда использовать: При запуске нового проекта, смене методологии или масштабировании команды
 - Частая ошибка: Сразу внедрять тяжёлые процессы и тонны документации → команда сопротивляется, процесс умирает
 - Лучшая практика: Начинать с минимума (BTS + чек-листы + регламент багов), затем масштабировать по мере роста
 - Исключение: В стартапах на ранней стадии допустимо исследовательское тестирование без документации до первых релизов
- Предостережение: Не строй процесс «в вакууме» — согласуй с разработкой, аналитикой и бизнесом, иначе он будет оторван от реальности

28 May 2026

13:33

Тестирование ПО

■ Тестовые окружения и Jenkins — архитектура качества

Тестовое окружение (Testing Environment) — комплекс серверов, баз данных, сетей и конфигураций, где запускается приложение для проверки.

Deployment (Развертывание) — процесс доставки и установки готового ПО или обновлений на сервера.

□ Карта тестовых окружений

◆ Local (Локальное)

- Возможность самостоятельного развертывания приложения на своём ПК

Пример: запуск через Docker или localhost:127.0.0.1

Разработчики: unit-тестирование отдельных методов и функций

- Тестировщики: проверка изолированных функций, отладка запросов

◆ Dev (Development / Песочница)

- Среда для разработчиков до передачи функционала тестировщикам

Пример: dev.test.com, куда падает код после каждого коммита

- Если нет выделенного QA-окружения, может использоваться тестировщиками (плохая практика)

◆ QA / Test (Тестовое)

- Основная среда, используемая только тестировщиками

Пример: qa1.test.com, qa2.test.com (несколько сред для параллельной работы)

- Виды тестирования: smoke, critical path, new feature testing, регресс

- Данные: синтетические, регулярно очищаются или сбрасываются

◆ Stage / Pre-Release (Предпродакшн)

- Стабильная версия приложения, максимально приближённая к Prod

Пример: stage.test.com

- Финальное тестирование, полная регрессия, code freeze (код не меняется)

- Проверка готовности приложения к выпуску в продакшн

◆ UAT (User Acceptance Testing)

- Приёмочное тестирование группой конечных пользователей или заказчика

Пример: uat.test.com с реальными бизнес-сценариями и тестовыми аккаунтами

- Подтверждение, что продукт решает задачу бизнеса

◆ Prod (Production / Боевое)

- Реальная среда для конечных пользователей

Мониторинг работы приложения, минимальный smoke новых функций, анализ пост-релизных проблем

- Прямое тестирование запрещено (используются канареечные релизы и фич-флаги)

★ Что спрашивают на собеседовании в 2026:

? «Зачем нужно несколько QA-окружений (QA1, QA2)?»

→ Чтобы тестировать разные фичи параллельно без конфликтов. Пока одна ветка в регрессе, другая тестируется изолированно. Это ускоряет delivery в Agile.

? «Почему нельзя тестировать на Dev?»

→ На Dev код нестабилен, разработчики постоянно перезаписывают билды. Тесты будут падать не из-за багов, а из-за «шума» и частых деплоев.

? «Что такое Code Freeze на Stage и зачем он нужен?»

→ Запрет на внесение изменений в код перед релизом. Нужен, чтобы финальная регрессия проверяла именно ту версию, которая пойдёт в Prod, без внезапных правок.

? «Как Jenkins помогает QA?»

→ Автоматизирует рутину: сборку, деплой на тестовые среды, запуск автотестов после каждого коммита. QA тратит время на анализ рисков и сложных сценариев, а не на ручную установку билдов.

☞ Итог за 10 секунд:

— Суть: Окружения — это ступени качества от локальной машины до продакшена, Jenkins — «конвейер» между ними

Когда использовать: Всегда. Каждое окружение решает свою задачу в жизненном цикле ПО

— Частая ошибка: Тестировать на Dev или Prod, путать данные между средами, игнорировать code freeze

— Лучшая практика: Чёткое разделение сред, автоматический деплой через Jenkins, тестовые данные изолированы

— Исключение: В маленьких проектах Dev и QA могут совмещаться, но это риск нестабильности

— Предостережение: Никогда не лей тестовые данные в Prod и не деплой нестабильный билд на Stage

13:35



Jenkins — роль в процессе

◆ **Jenkins** — сервер автоматизации (CI/CD), который управляет потоком кода между окружениями.

Автоматизирует сборку, тестирование и деплой

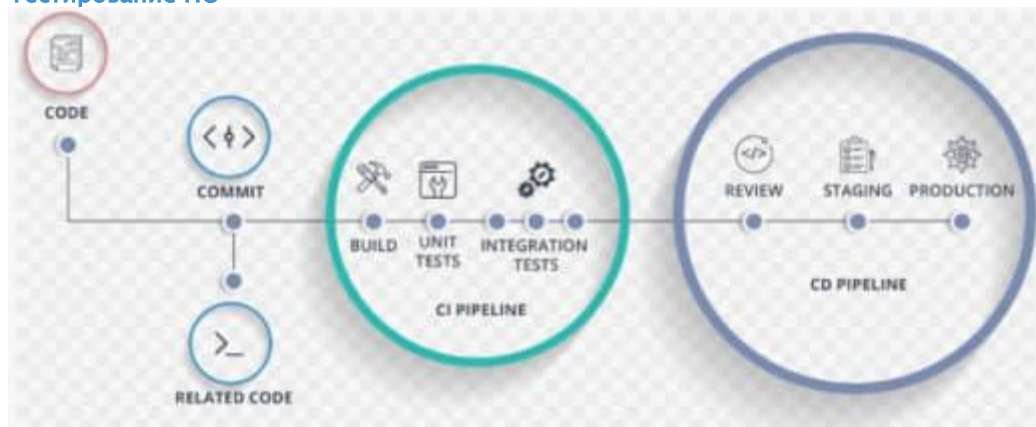
Пример: пуш в GIT → Jenkins собирает билд → запускает автотесты → деплоит на Dev → уведомляет команду

— Роль QA: настройка пайплайнов, запуск ручных/авто-прогонов по триггерам, анализ логов сборки, откат при падении

— Почему на схеме: связывает GIT, DB, Docker и все окружения в единый конвейер доставки

13:56

Тестирование ПО



🔗 CI/CD для тестировщика — автоматизация качества

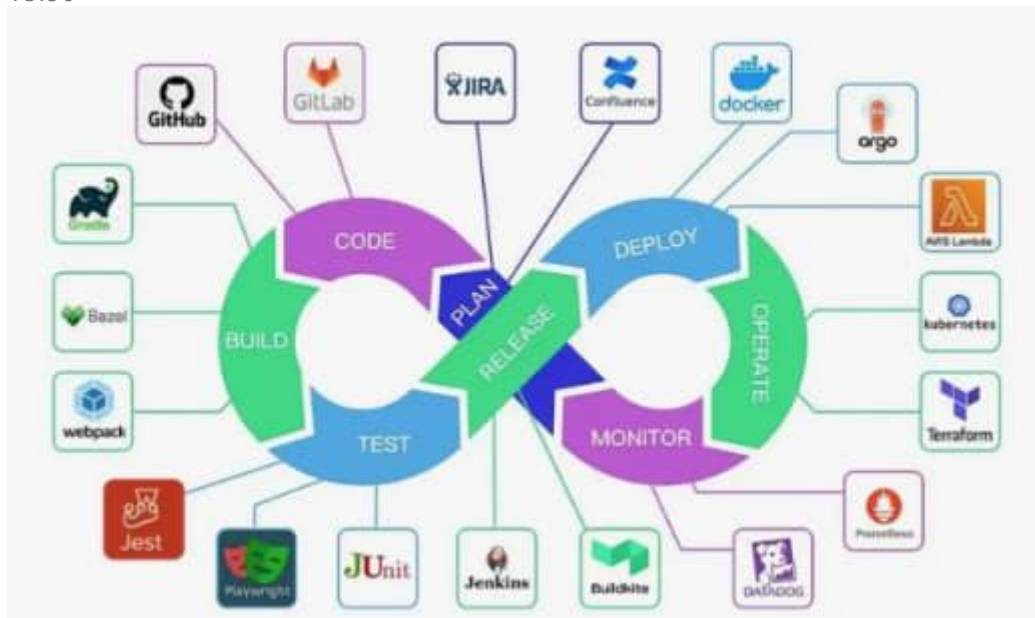
CI/CD (Continuous Integration / Continuous Delivery/Deployment) — технология автоматизации сборки,

тестирования и доставки кода от разработчика до пользователя.

Это подход, который автоматизирует процессы интеграции кода, тестирования и доставки. Для тестировщиков это означает переход от ручного тестирования в конце цикла к непрерывной автоматизированной валидации на протяжении всего процесса разработки.

DevOps — методология совместной работы разработки, тестирования и эксплуатации для ускорения и повышения качества доставки ПО.

13:56



14:16

Тестирование ПО

□ Этапы CI/CD процесса

◆ Continuous Integration (CI) — Непрерывная интеграция

- Коммит изменений в код (GitHub) Пример: разработчик делает `git push` → запускается процесс CI
- Запуск билда (сборки) Пример: код компилируется, собирается в исполняемый файл или Docker-образ
- Уведомление о готовности сборки

Пример: команда получает сообщение в чат: «Билд #1234 собран»

- Запуск юнит и интеграционных тестов

Пример: автоматически прогоняются тесты на функциональность и стыки модулей

- Уведомление о результатах тестов

Пример: «✓ Все тесты прошли» или «✗ 3 теста упали → смотри логи»

◆ Continuous Delivery (CD) — Непрерывная доставка

- Приемочные тесты (автоматические)

Пример: прогон E2E-сценариев на тестовом окружении

- Доставка билда на тестовое окружение (автоматически)

Пример: после успешных тестов билд деплоится на QA/Stage

- Доставка на Prod (в ручном режиме)

Пример: после апрува от команды/менеджера — ручной запуск деплоя на продакшн

◆ Continuous Deployment (CD) — Непрерывное развёртывание

- Приемочные тесты — аналогично Delivery

- Доставка на тестовое окружение — автоматически

- Доставка на Prod (автоматически) Пример: если все тесты зелёные — код попадает к пользователям без участия человека

□ Инструменты CI/CD по этапам

- Планирование – Confluence, Jira
- Кодирование – Git, GitHub, GitLab
- Сборка – Maven, Gradle, npm, sbt
- Тестирование – JUnit, Selenium, Playwright, pytest
- Релиз/Оркестрация – Jenkins, GitLab CI, GitHub Actions, CircleCI
- Развёртывание – Docker, Kubernetes, AWS, Ansible
- Мониторинг – Datadog, Prometheus, Grafana, Splunk

★ Что спрашивают на собеседовании в 2026:

? «В чём разница между Continuous Delivery и Continuous Deployment?»

→ **Delivery** — код готов к релизу, но деплой на Prod ручной (требуется апрув). **Deployment** — деплой на Prod полностью автоматический, если все тесты прошли. Delivery безопаснее для регулируемых отраслей, Deployment быстрее для стартапов.

? «Что делать, если пайплайн упал?»

→ 1) Посмотреть логи в Jenkins/GitLab CI, 2) Определить этап падения (сборка/тесты/деплой), 3) Если тесты — воспроизвести локально, 4) Если инфраструктура — уведомить DevOps, 5) Не мерджить новый код, пока пайплайн не зелёный.

? «Как тестировщик участвует в CI/CD?»

→ Пишет автотесты, которые запускаются в пайплайне; анализирует падения; настраивает триггеры (когда запускать тесты); добавляет шаги валидации качества (линтеры, безопасность); участвует в онбординге новых сред.

? «Зачем нужен мониторинг после деплоя?»

→ Чтобы ловить проблемы, которые не поймали тесты: нагрузка в проде, реальные данные, поведение пользователей. Инструменты: Datadog, Sentry, Grafana. QA настраивает алерты на ошибки 500, медленные запросы, аномалии.

☞ Итог за 10 секунд:

- **Суть:** CI/CD — автоматический конвейер: код → сборка → тесты → деплой; DevOps — культура совместной работы для его поддержки
- **Когда использовать:** Всегда в современной разработке — от старта до энтерпрайза
- **Частая ошибка:** Думать, что CI/CD — это «заботы разработчиков» → тестировщик теряет контроль над качеством релиза
- **Лучшая практика:** Встраивать автотесты в пайплайн на ранних этапах, настраивать уведомления о падениях, мониторить прод после деплоя
- **Исключение:** В очень маленьких проектах можно начать с ручного деплоя, но планировать автоматизацию с ростом команды
- **Предостережение:** Никогда не отключай тесты в пайплайне «чтобы быстрее» — это технический долг, который вернётся багами в проде

14:57

Тестирование ПО

👤 Роль тестировщика в CI/CD — архитектор качества

Роль тестировщика в CI/CD — участие в проектировании, настройке и поддержке автоматизированных тестовых процессов внутри конвейера непрерывной интеграции и доставки.

- **Проектирование автоматизированных тестовых наборов.** Тестировщики участвуют в создании тестовых сценариев, которые запускаются при каждом коммите.
- **Настройка этапов тестирования внутри пайплайна.** Тестировщики определяют, какие тесты и на каких этапах должны выполняться.
- **Анализ результатов тестов и предоставление быстрой обратной связи разработчикам.** Это позволяет быстрее выявлять проблемы и улучшать качество кода.
- **Поддержка тестовой инфраструктуры и обеспечение стабильности пайплайна.**
- **Оптимизация выполнения тестов для скорости и надёжности.**

◆ Ключевые преимущества для тестировщиков

- Более быстрые циклы обратной связи. Дефекты выявляются в течение минут после изменения кода.
- Сокращение ручного труда. Автоматизация повторяющихся задач тестирования.
- Улучшенное покрытие тестами. Запуск полных тестовых наборов при каждой сборке.
- Лучшая коллаборация. Наведение мостов между командами разработки и QA.
- Улучшенные метрики качества. Отслеживание трендов тестирования и выявление проблемных зон.

□ Типы тестов в CI/CD-пайплайне (пирамида тестов)

- Юнит-тесты — Проверяют отдельные компоненты системы на изолированном уровне.

Пример: тест функции расчёта скидки без подключения к БД

- Интеграционные тесты — проверка взаимодействия модулей/сервисов

Пример: тест, что сервис заказов корректно вызывает сервис оплаты

- End-to-End (E2E) тесты — Проверяют систему в целом с точки зрения пользователя.

Пример: «регистрация → добавление товара → оплата → получение заказа»

- Функциональные тесты — проверка бизнес-требований, Фокусируются на бизнес-требованиях и пользовательских сценариях.

Пример: «промокод применяется только к товарам из категории «Электроника»

- Нефункциональные тесты — производительность, безопасность, доступность и т. д.

Пример: нагрузочный тест: 1000 пользователей одновременно оформляют заказ

💡 *Оптимальная стратегия тестирования часто визуализируется в виде «пирамиды тестов», где юнит-тесты составляют основание (их больше всего), а E2E-тесты — вершину (их меньше всего).*

◆ Инструменты CI/CD для тестировщиков

- **Jenkins**. Кроссплатформенный инструмент с открытым исходным кодом на базе Java. Поддерживает создание конвейеров непрерывной доставки «как код» (Jenkinsfile), имеет обширную экосистему плагинов для интеграции с различными фреймворками тестирования.

- **GitLab CI**. Обеспечивает бесшовную интеграцию с репозиториями GitLab. Конфигурируется через файл

```
.gitlab-ci.yml
```

в корне проекта. Поддерживает Docker для изоляции тестовых окружений.

- **GitHub Actions**. Встроенный CI/CD в GitHub. Конфигурация через YAML-файлы в

```
.github/workflows/
```

- **CircleCI**. Облачное решение, оптимизированное для скорости и параллельного выполнения тестов.

- **Azure DevOps**. Комплексная платформа от Microsoft, включающая не только CI/CD, но и управление требованиями, тестами и релизами.

5 June 2026

16:12

SQL (Structured Query Language) — это язык для общения с реляционными базами данных (самые популярные в 2026 году: PostgreSQL, MySQL, Oracle).

#SQL

- ◆ **Реляционная база данных (РБД)** — это база, где данные хранятся в строгих таблицах, которые **взимосвязаны** (от англ. *relation* — связь, отношение) между собой по общим признакам.

Аналогия: Несколько Excel-таблиц, которые умеют автоматически ссылаться друг на друга. В одной

таблице у тебя "Клиенты", в другой — "Заказы". Они не свалены в одну кучу.

◆ **3 кита РБД, которые обязан знать QA:**

- **Таблица (Table):** Состоит из строк (записей) и столбцов (полей/атрибутов).
- **Primary Key (PK, Первичный ключ):** Уникальный идентификатор строки (например, `id`). Как номер паспорта — дубликатов быть не может.
- **Foreign Key (FK, Внешний ключ):** "Крючок" для связи. В таблице "Заказы" есть поле `client_id`, которое ссылается на `id` из таблицы "Клиенты".

◆ **Секрет для тестировщика (актуально на 2026 год):** Из-за жестких связей в РБД работает **целостность данных (Data Integrity)**.

- **Пример из практики:** Разработчик сделал кнопку "Удалить пользователя". Ты идешь в базу и пытаешься удалить юзера вручную, а база падает с ошибкой `Foreign Key Constraint`. Почему? Потому что у этого юзера остались "Заказы", и база защищает их от "сиротства".
- **Твоя задача:** Проверять, что фронтенд/API не просто выдают "Ошибка 500", а корректно обрабатывают такие конфликты базы данных и показывают юзеру понятное сообщение ("Сначала удалите все заказы").

Из-за этих связей тебе и понадобятся `JOIN` (объединения) — чтобы проверять, правильно ли разработчики "склеили" данные из разных таблиц для вывода на экран.

✂ **Итог за 10 секунд:**

- РБД = данные в таблицах, связанных между собой.
- Primary Key (PK) = уникальный ID строки (как паспорт).
- Foreign Key (FK) = ссылка из одной таблицы на другую (связь).
- QA проверяет целостность данных: что нельзя удалить "главную" запись, пока есть "дочерние".

#SQL

17:55

Тестирование ПО

◆ **СУБД (Система Управления Базами Данных)** — это программный движатель, который создает, хранит, защищает и управляет базами данных.

◆ **Зачем это тестировщику:**

- Ты подключаешься к СУБД через специальные программы-клиенты (в 2026 году стандарты индустрии: DBeaver, DataGrip, pgAdmin), чтобы читать и проверять данные.
- Нужно всегда уточнять у разработчиков, на какой СУБД работает проект. Абсолютные лидеры в 2026 году: PostgreSQL, MySQL, Oracle.
- У каждой СУБД есть свой диалект SQL. Запрос, который идеально сработает в MySQL, может выдать ошибку в PostgreSQL (например, при форматировании дат). Тестировщик должен писать запросы с учетом конкретной СУБД.

✂ **Итог за 10 секунд:**

- БД — это файлы с данными, СУБД — это программа, которая этими файлами управляет.
- СУБД берет на себя всю тяжелую работу: поиск, сортировку, защиту и связи.
- Топ СУБД в 2026 году: PostgreSQL (лидер), MySQL.
- У каждой СУБД свой диалект SQL, поэтому универсальных запросов на все случаи жизни не бывает.

#SQL #СУБД

18:13

Тестирование ПО

Виды баз данных

◆ Реляционные СУБД (RDBMS, Relational Database Management System)

- Это классика. Данные хранятся строго в таблицах (строки и столбцы).
- Поддерживают только простые типы данных (числа, строки, даты).
- Примеры: MySQL, SQLite, Oracle.

◆ Объектно-реляционные СУБД (ORDBMS, Object-Relational Database Management System)

- Это эволюция реляционных баз. Они взяли за основу таблицы, но добавили возможности объектного программирования.
- Позволяют хранить внутри ячеек таблицы сложные объекты: массивы, JSON-документы, пользовательские типы данных.
- Главный пример в 2026 году: PostgreSQL (технически это ORDBMS, хотя многие по привычке называют её просто реляционной).

◆ Главная разница – в гибкости хранения.

В классической RDBMS (как MySQL) ты не можешь просто так сохранить массив в колонке. В ORDBMS (как PostgreSQL) ты создаешь колонку типа `jsonb` или `array` и хранишь там сложные структуры, продолжая использовать привычный SQL.

◆ Какие еще виды бывают (NoSQL)

В современных проектах 2026 года используют не только SQL. Тестировщик обязан знать про NoSQL (Not Only SQL), где нет жестких таблиц:

- **Документные (MongoDB):** Данные хранятся в виде JSON-документов. У каждой записи может быть своя уникальная структура. Идеально для каталогов товаров с разными характеристиками.
- **Ключ-Значение (Redis):** Работает как словарь. Есть ключ (например, `user:123:session`) и значение. Невероятно быстрая, используется для кэша и хранения сессий.
- **Колоночные (ClickHouse):** Данные хранятся по столбцам, а не по строкам. Создана для аналитики, дашбордов и обработки огромных объемов данных.
- **Графовые (Neo4j):** Данные — это узлы и связи между ними. Идеально для соцсетей (поиск "друзей друзей") и рекомендательных систем.

◆ Зачем это тестировщику (QA)

- Современные приложения используют несколько баз сразу (например, PostgreSQL для пользователей, Redis для кэша, MongoDB для логов).
- Твоя задача на проекте — узнать у разработчиков, какая СУБД за что отвечает, чтобы писать правильные запросы и проверять данные в нужном месте.

✂ Итог за 10 секунд:

- RDBMS (MySQL) — строгие таблицы, только простые типы данных.
- ORDBMS (PostgreSQL) — таблицы + сложные типы (JSON, массивы).
- NoSQL (MongoDB, Redis) — нет жестких таблиц, хранят документы, кэш или графы.
- QA должен знать, какая СУБД за что отвечает в проекте, чтобы проверять данные в правильном месте.

#SQL #СУБД #виды СУБД

18:32

Тестирование ПО

◆ ANSI (American National Standards Institute — Американский национальный институт стандартов)

SQL-92 (Structured Query Language — язык структурированных запросов)

Это фундаментальный международный стандарт языка SQL, принятый в 1992 году. Он зафиксировал базовый синтаксис, чтобы любой SQL-запрос работал одинаково в любой базе данных.

◆ Отступления от стандарта (SQL Dialects — диалекты SQL)

- Разработчики СУБД (RDBMS — реляционных систем управления базами данных) добавляют свои

уникальные функции и синтаксис, чтобы их продукт был быстрее или удобнее.

— Из-за этого чистый SQL-92 на практике почти не встречается.

— **Примеры отступлений:**

— — В PostgreSQL есть мощные операторы для работы с JSON (JSONB), которых нет в стандарте.

— — В MySQL специфичная работа с автоинкрементом (AUTO_INCREMENT вместо стандартного GENERATED ALWAYS AS IDENTITY).

— — В Oracle (Oracle Database) свой процедурный диалект PL/SQL (Procedural Language / Structured Query Language), а в Microsoft SQL Server — T-SQL (Transact-SQL).

◆ **Почему это критично для QA (Quality Assurance — обеспечения качества)**

— Ошибка новичка: написать идеальный запрос по стандарту SQL-92 на локальной MySQL, а на продакшене (production — рабочем сервере) стоит PostgreSQL, и запрос падает с синтаксической ошибкой.

— Правило 2026 года: тестировать базы данных нужно строго в том окружении (environment), которое используется на продакшене.

— Если в команде есть Code Review (проверка кода) SQL-запросов, ты должна знать, какие функции являются специфичными для вашей СУБД, а какие — универсальными.

✂ **Итог за 10 секунд:**

— ANSI SQL-92 — это базовый международный стандарт синтаксиса SQL.

— На практике используют диалекты SQL (T-SQL, PL/pgSQL), так как СУБД добавляют свои уникальные функции.

— Запрос, работающий в одной СУБД, может не работать в другой из-за отступлений от стандарта.

— QA обязан писать и проверять запросы с учетом конкретной СУБД проекта, а не абстрактного стандарта.

[#SQL](#) [#СУБД](#) [#язык СУБД](#)

18:44

MySQL Workbench

MySQL Workbench (Официальная визуальная среда разработки и администрирования для MySQL).

— Это графический клиент (GUI — Graphical User Interface, графический пользовательский интерфейс), созданный компанией Oracle специально для работы с СУБД MySQL.

✂ **Итог за 10 секунд:**

— MySQL Workbench — это официальная графическая программа (GUI) для работы исключительно с базой данных MySQL.

— Позволяет писать SQL-запросы, рисовать схемы таблиц и настраивать сервер.

— Для QA это хороший инструмент проверки данных, но на практике часто уступает универсальным аналогам (DBeaver), если в проекте используется несколько разных СУБД.

[Ссылка для скачивания и установки](#). Работает бесплатно. Скачиваем тот, который больше весит.

[#SQL](#) [#MySQL](#)